

Research Provenance Transcript

Simulate Confidence Intervals

Reader-friendly edition. Mathematical notation and Markdown are rendered; the chronological dialogue is retained.

Source conversation ID: 6a86318a-aae4-83eb-bdcd-be4051c0781f **Captured:** 20 August 2026 **Record type:** Chronological research-development transcript

This document preserves the dialogue as retrieved from the source conversation, separately from any later manuscript, code, or simulation revisions. It has not been substantively edited. One assistant message exceeded the source retrieval interface's 20,000-character limit; that message is explicitly marked where it occurs. An export from the original ChatGPT conversation is required to replace that single truncated passage with its complete original text.

Turn 1

User

If I want to simulate confidence interval for normal cdf given ecdf, can I simulate confidence intervals for $N(\mu, \sigma^2)$ for many pairs of (μ, σ^2) and then invert by checking if ecdf is inside each interval?

Assistant

Yes. What you're describing is essentially **test inversion**, and it can give a valid confidence region for (μ, σ^2) —with one important caveat: the interval/band around the ECDF must be a **simultaneous confidence band**, not a collection of pointwise confidence intervals.

Suppose you observe $X_1, \dots, X_n$ and their empirical CDF $F_n(x)$. For every candidate

$$\theta = (\mu, \sigma), \quad F_\theta(x) = \Phi\left(\frac{x - \mu}{\sigma}\right),$$

you can test

$$H_0 : F = F_\theta.$$

A natural statistic is

$$D(\mu, \sigma) = \sup_x |F_n(x) - F_{\mu, \sigma}(x)|.$$

Then keep every candidate pair for which

$$D(\mu, \sigma) \leq c_\alpha.$$

The resulting set

$$\mathcal{C}_{1-\alpha} = \{(\mu, \sigma) : D(\mu, \sigma) \leq c_\alpha\}$$

is a $1 - \alpha$ confidence region obtained by **inverting the goodness-of-fit tests**.

Convenient fact for the normal case

You actually don't have to simulate the confidence band separately for thousands of (μ, σ) pairs.

For any **fixed** (μ, σ) , if

$$X_i \sim N(\mu, \sigma^2),$$

then by the probability integral transform,

$$U_i = \Phi\left(\frac{X_i - \mu}{\sigma}\right) \sim \text{Uniform}(0, 1).$$

Therefore the distribution of

$$\sup_x |F_n(x) - F_{\mu, \sigma}(x)|$$

does **not** depend on μ or σ . It is the usual Kolmogorov–Smirnov distribution. So you can simulate the cutoff once:

1. Simulate n $\text{Uniform}(0, 1)$ observations many times. 2. Calculate their KS statistics. 3. Take the $1 - \alpha$ quantile c_α . 4. Search over (μ, σ) . 5. Accept a pair whenever

$$\sup_x |F_n(x) - F_{\mu, \sigma}(x)| \leq c_\alpha.$$

That gives you a joint confidence region.

An alternative is the DKW inequality, which gives the band

$$F_n(x) - \epsilon \leq F(x) \leq F_n(x) + \epsilon \quad \text{for all } x,$$

where

$$\epsilon = \sqrt{\frac{\log(2/\alpha)}{2n}}.$$

You would then keep a candidate normal distribution if

$$F_n(x) - \epsilon \leq \Phi\left(\frac{x - \mu}{\sigma}\right) \leq F_n(x) + \epsilon$$

**for every x **. This has at least $1 - \alpha$ coverage, although DKW is generally more conservative than an exact KS band.

What would *not* be valid

If, for every x , you construct a 95% **pointwise** interval and then say

”accept (μ, σ) if its CDF falls inside all the 95% pointwise intervals”,

the resulting region will **not** generally have 95% coverage. You’re checking many x ’s simultaneously, so the band has to account for that.

There’s also an important distinction from the **Lilliefors test**. If you first estimate

$$\hat{\mu}, \hat{\sigma}$$

from the same sample and examine

$$\sup_x |F_n(x) - F_{\hat{\mu}, \hat{\sigma}}(x)|,$$

then the ordinary KS distribution no longer applies. But in your proposed inversion, you’re treating each candidate (μ, σ) as a fixed null hypothesis, so ordinary KS logic works.

So your idea is sound, and a particularly clean implementation is:

$$\mathcal{C}_{1-\alpha} = \left\{ (\mu, \sigma^2) : \sup_x \left| F_n(x) - \Phi\left(\frac{x - \mu}{\sigma}\right) \right| \leq c_{n,\alpha} \right\}.$$

If your ultimate goal is instead a **confidence band for the fitted normal CDF $F(x)$ ** rather than a confidence region for (μ, σ^2) , there’s a slightly different and usually more efficient construction using the joint sampling distribution of $(\hat{\mu}, \hat{\sigma})$.

Turn 2

User

Do people use the method described by me?

Assistant

Yes—the **underlying method is definitely used**, though people usually describe it in slightly different language.

What you proposed is essentially **confidence-set construction by test inversion**:

$$C_{1-\alpha} = \{(\mu, \sigma) : \text{the candidate } N(\mu, \sigma^2) \text{ is not rejected}\}.$$

If the test is based on the maximum distance between the ECDF and the candidate CDF,

$$D(\mu, \sigma) = \sup_x \left| F_n(x) - \Phi\left(\frac{x - \mu}{\sigma}\right) \right|,$$

then this is closely related to **Kolmogorov–Smirnov minimum-distance inference**. There is a substantial statistical literature on comparing an empirical CDF with a parametric family and using KS/Cramér–von Mises-type distances for inference; Pollard’s work, for example, explicitly treats the empirical CDF as being compared with a parametric surface $\{F_\theta : \theta \in \Theta\}$. [source citation: turn941165view1]

So conceptually, your algorithm

try $(\mu, \sigma) \rightarrow$ construct/check a CDF band $\rightarrow$ retain compatible pairs

is perfectly recognizable statistically.

The main difference is that researchers would usually **not** simulate a new confidence band independently for every (μ, σ) . For a fixed candidate (μ, σ) , the ordinary one-sample KS statistic is distribution-free, so one critical value can be used for every candidate. Standard KS implementations likewise require the tested distribution’s parameters to be prespecified rather than estimated from the same data. [source citation: turn696141search0 turn696141search1]

Thus your procedure can be written much more simply as

$$C = \left\{ (\mu, \sigma) : \sqrt{n} \sup_x |F_n(x) - F_{\mu, \sigma}(x)| \leq c_\alpha \right\}.$$

You can evaluate that over a dense grid of (μ, σ) and plot the accepted region. That’s essentially a **KS-based confidence region for the normal location-scale parameters**.

There is also literature specifically connecting confidence regions for location-scale parameters with confidence bands for their CDFs, so the general relationship you’re exploiting between parameter regions and CDF bands is well established. [source citation: turn575430search3]

What is **unusual** is using this as your first-choice CI for (μ, σ^2) when you are willing to assume exact normality. In that case, $\bar{X}$ and S^2 contain much more targeted information about μ and σ^2 , and classical pivotal/likelihood methods give more efficient confidence regions. The KS approach deliberately cares about the **entire CDF**, so it spends statistical power detecting discrepancies that aren’t specifically about mean and variance.

But that feature can actually make your method interesting if your goal is something like:

”Which normal distributions are globally compatible with the observed ECDF?”

rather than merely

”What are efficient confidence intervals for μ and σ^2 ?”

Those are subtly different inferential questions. Your construction answers the first one very naturally.

And there’s an especially nice interpretation: if you plot all accepted (μ, σ) , you’re literally looking at the subset of the **normal-distribution manifold that lies within a KS ball around the ECDF**. That connects your idea directly to minimum-distance statistics. [source citation: turn941165view1]

If this is for research, I’d call it something like **”a confidence region obtained by inversion of the one-sample KS goodness-of-fit test over the normal location-scale family”** rather than describing it as a new simulation procedure.

Turn 3

User

Do you think that this is potentially good topic for the research?

Assistant

Yes. **I think it could be a very good research topic, especially if you push it one step beyond “invert the KS test.”** The inversion idea itself is classical: minimum-distance estimation and goodness-of-fit-test inversion have a long literature. [source citation: turn796646view1] Confidence regions and simultaneous CDF bands for location–scale families have also been studied extensively. [source citation: turn796646view2] So I would not try to claim the basic idea itself as new.

What I find much more interesting is the **geometry and computation of the resulting confidence region.**

Suppose

$$x_{(1)} \leq \cdots \leq x_{(n)}$$

and your KS cutoff is c_α . For a candidate normal distribution, acceptance is equivalent to

$$\frac{i}{n} - c_\alpha \leq \Phi\left(\frac{x_{(i)} - \mu}{\sigma}\right) \leq \frac{i-1}{n} + c_\alpha$$

for every i , with the endpoints truncated to $[0, 1]$.

Define

$$a_i = \Phi^{-1}\left(\max\left\{0, \frac{i}{n} - c_\alpha\right\}\right),$$
$$b_i = \Phi^{-1}\left(\min\left\{1, \frac{i-1}{n} + c_\alpha\right\}\right).$$

Then the conditions become

$$a_i \leq \frac{x_{(i)} - \mu}{\sigma} \leq b_i,$$

or, because $\sigma > 0$,

$$\boxed{x_{(i)} - b_i\sigma \leq \mu \leq x_{(i)} - a_i\sigma.}$$

That is quite nice: ****each observation gives two linear inequalities in (μ, σ) **.**

Therefore your KS confidence region is

$$C_{1-\alpha} = \bigcap_{i=1}^n \left\{(\mu, \sigma) : x_{(i)} - b_i\sigma \leq \mu \leq x_{(i)} - a_i\sigma\right\}, \quad \sigma > 0.$$

So in (μ, σ) -space, the confidence region is an **intersection of half-planes**. In particular, it is convex. You don't actually need to simulate thousands of (μ, σ) pairs or use a grid.

And this observation is **not specific to the normal distribution**. If

$$F_{\mu, \sigma}(x) = G\left(\frac{x - \mu}{\sigma}\right)$$

for any continuous strictly increasing location–scale family, replace Φ^{-1} by G^{-1} , and the same linear geometry appears.

That broader version is where I think the topic becomes much stronger.

A potentially solid research project

I would frame it roughly as:

“Finite-sample confidence regions for location–scale families obtained by inversion of distribution-free goodness-of-fit tests.”

Then investigate:

1. **Exact finite-sample coverage.** Because each candidate (μ, σ) defines a completely specified continuous CDF, inversion of an exact KS test gives finite-sample coverage determined by the KS critical value—not merely asymptotic coverage.

2. **Geometry.** Prove formally that the KS-inverted region in (μ, σ) is convex/polyhedral and characterize its vertices, boundedness, area, and when it is empty.

3. **Efficient computation.** Instead of searching a grid of perhaps 10^6 parameter pairs, construct the polygon directly from the $2n$ inequalities. Marginal intervals

$$[\mu_L, \mu_U], \quad [\sigma_L, \sigma_U]$$

could then be obtained by projecting the region or solving simple linear programs.

4. **Statistical efficiency.** Compare its expected area and marginal interval lengths with likelihood-ratio, pivotal, bootstrap, and Wald regions under exact normality. Classical minimum-distance work already highlights the efficiency-versus-robustness question, so that comparison has a solid theoretical foundation. [source citation: turn796646view1]

5. **Robustness and extensions.** Investigate contamination, outliers, heavy-tailed location-scale families, one-sided KS bands, weighted KS, Anderson–Darling/Cramér–von Mises variants, censoring, etc. Work on confidence regions/bands for location-scale models shows that these types of extensions remain active research questions. [source citation: turn796646view2 turn244844academia35]

One especially interesting property is **emptiness**. If

$$C_{1-\alpha} = \emptyset,$$

then *no* normal distribution is within the KS acceptance band around the ECDF. Thus the same object simultaneously tells you about **parameter uncertainty and model compatibility**. A conventional confidence ellipse around $(\hat{\mu}, \hat{\sigma})$ does not naturally have that interpretation.

So my assessment would be:

For a bachelor’s/master’s thesis: very good topic.

For a PhD chapter: potentially good if generalized and developed theoretically.

For a publishable statistics paper: probably not enough to just say “invert KS over (μ, σ) ,” because test inversion/minimum-distance inference is old. [source citation: turn796646view1] But the exact geometry, computational construction, optimality/efficiency, robust variants, or a general theory for location-scale families could potentially make it interesting.

In fact, I think the **half-plane representation above is a much better starting point than your original simulation idea**. It turns what initially looks like a Monte Carlo method into a clean geometrical inference problem, and that gives you substantially more mathematical material to investigate.

Turn 4

User

Is this fact about convexity that you described known or new?

Assistant

It is **known, not new**—and I found a paper that is almost exactly about this point.

Anthony Salvia's 1980 paper, "*Some Fundamental Properties of Kolmogorov-Smirnov Consonance Sets*" in *Technometrics*, studies confidence sets obtained by inverting goodness-of-fit statistics over **location and scale parameters**. Its abstract explicitly states that the Kolmogorov–Smirnov consonance sets are **convex and finite**, and that the paper gives a construction method. [source citation: turn938264search0]

Even more explicitly, Dana Quade's 1986 work on joint location–scale confidence regions says that the KS confidence region is an **"intersection of half-planes" and therefore convex**, and then notes that Salvia (1980) proved the corresponding properties for the **one-sample Kolmogorov test**. Quade also says the related geometry was already implicit in Switzer (1976). [source citation: turn593819view0 turn938264search3]

So the derivation I gave you,

$$x_{(i)} - b_i\sigma \leq \mu \leq x_{(i)} - a_i\sigma,$$

leading to an intersection of linear half-spaces in the (σ, μ) plane, is essentially a rediscovery of a known result.

There is, however, one important nuance that I should correct from my earlier answer:

The convexity is naturally in (μ, σ) , not necessarily (μ, σ^2) .

If you use variance $v = \sigma^2$, your inequalities become

$$x_{(i)} - b_i\sqrt{v} \leq \mu \leq x_{(i)} - a_i\sqrt{v},$$

which are no longer linear. A nonlinear reparameterization does **not** preserve convexity in general. So the classical polyhedral result concerns location–**scale** coordinates.

This actually changes my assessment of your research idea somewhat. **"The KS-inverted normal confidence region is convex" cannot be the novelty**, because Salvia established that over 45 years ago. But there may still be worthwhile questions nearby—for example, the geometry under variance/log-scale parameterizations, optimal projections to obtain intervals for μ and σ^2 , comparisons with likelihood regions, or modern computational/generalized versions.

And the historical trail is surprisingly direct:

Switzer (1976) → Salvia (1980) → Quade (1986),

with Salvia being the key reference for exactly the one-sample convexity fact. [source citation: turn938264search3 turn938264search4]

If you're seriously considering this as a research topic, I think the useful next step is to identify **what surrounding questions have not already been answered** in this older literature. I can do a deeper literature search around that and try to identify plausible gaps.

Turn 5

User

do a deeper search

Assistant

I did a substantially deeper search. The picture is now much clearer: **your original construction is part of an older line of research that seems to have gone somewhat quiet, but a very similar general idea has just reappeared in current research.** That makes the area interesting, but the novelty has to be chosen carefully.

What is already known

There is a remarkably direct sequence of papers.

| Year | Work | Relation to your idea | |---|---| | 1973 | Barr & Davidson | Explicitly take a KS confidence band $R(F_n)$ and define the parameter set $\{\lambda : F_\lambda \in R(F_n)\}$, noting that it is a simultaneous confidence region for λ . This is essentially your original idea. [source citation: turn680433view1] | | 1976 | Easterling | Proposes parameter estimation by **inversion of goodness-of-fit tests** and calls the resulting sets “consonance regions.” [source citation: turn390047search1] | | 1978 | Littell & Rao | Paper is literally titled *Confidence Regions for Location and Scale Parameters Based on the Kolmogorov-Smirnov Goodness of Fit Statistic*. They derive computational methods and discuss censoring. [source citation: turn390047search3 turn221779search1] | | 1980 | Salvia | Proves that the KS consonance sets for location and scale are **convex and finite**, and provides a construction method. [source citation: turn390047search0] | | 1980 | Boos | Does analogous inference using the Anderson–Darling distance, including confidence regions and efficiency considerations. [source citation: turn390047search5 turn390047search6] | | 1982 | Littell & Rao | Studies asymptotics, interval length, comparison with MLE, and robustness to model misspecification for KS minimum-distance inference. [source citation: turn390047search2] | | 2001–2013 | Several authors | Study minimum-area/optimal location–scale confidence regions, including normal models and regions based on order statistics. [source citation: turn190642search0 turn441089search0] | | 2023 | Dümbgen & Wellner | Develop modern distribution-free CDF bands with much better tail behavior than ordinary KS-type constant-width bands. [source citation: turn491936search62] | | 2026 | Chattopadhyay, Sarkar & Kuchibhotla | Very recently publish a framework that **inverts distribution-free CDF bands through parametric quantile functions** to get parameter confidence sets. [source citation: turn680433view0] |

So the statement

“Take an ECDF confidence band and retain all (μ, σ) whose normal CDF fits inside it.”

is definitely not new. Barr–Davidson and especially Littell–Rao are extremely close to exactly that.

Likewise, the fact we discussed earlier,

$$C = \bigcap_i \{\text{linear constraints in } (\mu, \sigma)\},$$

and hence convexity of the KS region, is not new: Salvia explicitly proved convexity and finiteness. [source citation: turn390047search0]

But something interesting happened very recently

The 2026 UAI paper is particularly important for you.

Chattopadhyay, Sarkar and Kuchibhotla define a distribution-free band giving bounds

$$\ell_i \leq F(X_{(i)}) \leq u_i$$

and then invert it through a parametric quantile function Q_Λ :

$$Q_\Lambda(\ell_i) \leq X_{(i)} \leq Q_\Lambda(u_i), \quad i = 1, \dots, n.$$

Their confidence set consists of all Λ satisfying those inequalities. That is essentially the **generalized form of the idea you independently arrived at**. [source citation: turn835661search0 turn680433view0]

They apply it primarily to Tukey Lambda and generalized Lambda distributions, where likelihood methods are awkward because the quantile function is available analytically while the CDF/density may not be. [source citation: turn680433view0]

Importantly, they don’t restrict themselves conceptually to KS. They discuss bands such as DKW, Berk–Jones and the newer Dümbgen–Wellner construction, and empirically find the more sophisticated DW band can give substantially narrower induced intervals than the fixed-width DKW band. [source citation: turn835661search0]

That tells me that **CDF-band inversion as a parametric-inference methodology is currently research-relevant**, even though its roots go back fifty years.

The potentially interesting observation from your normal problem

There is a generalization of the convexity argument that I think is more interesting than the KS-specific result.

Consider *any* simultaneous distribution-free order-statistic band

$$\ell_i \leq F(X_{(i)}) \leq u_i.$$

For a location–scale model,

$$F_{\mu,\sigma}(x) = G\left(\frac{x - \mu}{\sigma}\right),$$

the quantile function is

$$Q_{\mu,\sigma}(p) = \mu + \sigma G^{-1}(p).$$

Band inversion therefore gives

$$Q_{\mu,\sigma}(\ell_i) \leq X_{(i)} \leq Q_{\mu,\sigma}(u_i),$$

or

$$\mu + \sigma G^{-1}(\ell_i) \leq X_{(i)},$$

and

$$\mu + \sigma G^{-1}(u_i) \geq X_{(i)}.$$

These are ***linear inequalities in (μ, σ) ***.

Consequently,

$$C = \bigcap_{i=1}^n \left\{ (\mu, \sigma) : \begin{array}{l} \mu + \sigma G^{-1}(\ell_i) \leq X_{(i)}, \\ \mu + \sigma G^{-1}(u_i) \geq X_{(i)} \end{array} \right\}$$

is a convex polyhedron in location–scale coordinates.

And notice what happened: **we never used the KS band.**

So KS, DKW, Berk–Jones, Owen’s likelihood band, Dümbgen–Wellner, or another order-statistic band all produce a polygonal confidence region for a location–scale family, provided the band can be represented by deterministic ℓ_i, u_i .

Owen’s 1995 nonparametric-likelihood bands, for example, are explicitly designed to be narrower in the tails than KS bands. [source citation: turn491936search2] Dümbgen–Wellner’s newer bands similarly target the weakness of KS bands in the tails. [source citation: turn491936search62]

In my searches, I ***did not find a paper whose main result is a systematic development of this “arbitrary distribution-free band $\rightarrow$ convex polyhedral location–scale confidence region” viewpoint***, although pieces are clearly present in Salvia, the location–scale confidence-band literature, and the new 2026 quantile-inversion paper. That is not enough to claim novelty, but it is a much more interesting direction to investigate.

There is also related work from Rosenkrantz on cases where quantile functions are linear in parameters, although his direction of construction is somewhat different: he starts from parameter confidence sets to construct bands. He explicitly treats normal and exponential distributions. [source citation: turn715317search3]

The research question I would investigate

Instead of:

Can we construct a normal (μ, σ) confidence region by KS inversion?

which has essentially been answered since 1978, I would consider:

Among distribution-free simultaneous CDF bands, which band produces the best confidence region for the parameters of a location–scale family?

That creates a genuinely richer problem.

Suppose $\mathcal{B}$ is a family of exact $1 - \alpha$ CDF bands. Every band $B \in \mathcal{B}$ induces

$$C_B(X) = \{\theta : F_\theta \in B(X)\}.$$

Then instead of optimizing the CDF band according to its vertical width, optimize according to something such as

$$E_{\mu, \sigma} [\text{Area}(C_B(X))].$$

In other words:

$$B^* = \arg \min_{B \in \mathcal{B}} E [\text{Area}\{(\mu, \sigma) : F_{\mu, \sigma} \in B\}].$$

That is quite different from simply asking for a narrow CDF band.

There is existing work that gets close, so you would have to distinguish it carefully. Frey (2008) optimizes **distribution-free CDF bands themselves** according to narrowness criteria. [source citation: turn491936search4] Frey, Marrero and Norton (2009) construct minimum-area confidence regions and minimum-area CDF bands specifically for the normal distribution. [source citation: turn902438search0] Czarnowska and Nagaev derive minimum-area regions for general location-scale families. [source citation: turn190642search0] And Zaigraev and Alama-Bučko optimize confidence regions based on the choice of two order statistics. [source citation: turn441089search0]

But I did **not** find, in this search, a paper explicitly posing:

choose a distribution-free ECDF band $\longrightarrow$ minimize the induced parametric-region area.

That's the gap I'd investigate first.

It has a nice conceptual twist: the "best" confidence band for estimating the **whole CDF** need not be the best confidence band for estimating $^{**}(\mu, \sigma)^{**}$.

There could be a nice general theorem too

You can go beyond location-scale distributions.

Suppose a family has an **affine quantile parameterization**

$$Q_\theta(p) = a(p)^\top \theta + b(p).$$

Then band inversion gives

$$a(\ell_i)^\top \theta + b(\ell_i) \leq X_{(i)}$$

and

$$a(u_i)^\top \theta + b(u_i) \geq X_{(i)}.$$

Hence the exact confidence region is again a polyhedron:

$$C = \{\theta : A\theta \leq b\}.$$

Suddenly the statistical inference problem becomes a **convex optimization / computational geometry problem**. Marginal intervals become linear programs:

$$\mu_L = \min_{\theta \in C} \mu, \quad \mu_U = \max_{\theta \in C} \mu,$$

etc.

That might be a more compelling mathematical framework than restricting the paper to normal distributions.

One thing I would *not* pursue as the main novelty

I would not focus mainly on comparing the KS region with classical normal-theory regions.

There is already quite a lot here. Arnold and Shavelle compare exact and asymptotic joint confidence sets for (μ, σ^2) . [source citation: turn339942search0] Czarnowska–Nagaev derive minimum-area location-scale regions including numerical results for the normal distribution. [source citation: turn190642search0] Frey et al. derive a minimum-area normal confidence region and a minimum-area normal CDF band. [source citation: turn902438search0] Zhang later studies minimum-volume confidence sets for normal parameters as well. [source citation: turn339942search1]

And even KS robustness/efficiency isn't untouched: Littell and Rao already compared interval lengths with MLE and simulated the effect of model misspecification on coverage in 1982. [source citation: turn390047search2]
It would make a fine thesis simulation chapter, but by itself probably not a new paper.

My updated assessment

After this deeper search, I'd rank the ideas approximately like this:

"KS inversion for normal (μ, σ) " $\rightarrow$ known.

"The resulting region is convex" $\rightarrow$ known since Salvia 1980.

"It is an intersection of half-planes and can be constructed without grid simulation" $\rightarrow$ essentially part of that older geometric literature, so probably not enough.

"Replace KS with modern distribution-free bands and systematically study the induced location-scale polyhedra" $\rightarrow$ considerably more interesting, but adjacent to the 2026 Chattopadhyay–Sarkar–Kuchibhotla work.

"Find/characterize the distribution-free CDF band that is optimal with respect to the size of its induced parametric confidence region" $\rightarrow$ the most promising research question I found. I haven't found an exact precedent for that formulation, though related optimization literature means it would need a proper systematic literature review before claiming novelty.

And there's an encouraging sign: a paper published **this month at UAI 2026** is using essentially the same broad CDF-band-inversion philosophy on difficult parametric families. [source citation: turn680433view0] So the conceptual direction is not merely an old statistical curiosity—it is currently being rediscovered/developed.

If you were thinking about turning this into an actual research project, I would now start from the general object

$$C_{\ell,u}(X) = \bigcap_i \{\theta : Q_\theta(\ell_i) \leq X_{(i)} \leq Q_\theta(u_i)\}$$

and ask **how the choice of (ℓ_i, u_i) controls the geometry, expected volume, efficiency, and robustness of $C_{\ell,u}$ **. That looks much closer to a genuine research programme than the original normal-KS construction.

Turn 6

User

”Replace KS with modern distribution-free bands and systematically study the induced location–scale polyhedra”
I liked this approach. elaborate

Assistant

Yes. I think this becomes much more interesting when formulated as a **general geometric inference problem** rather than as “use a better band than KS.”

The core idea would be:

$$\boxed{\text{distribution-free CDF band} \longrightarrow \text{exact finite-sample polyhedral confidence region for } (\mu, \sigma)}$$

and then ask how the **choice of CDF band controls the geometry and efficiency of that region**.

A very recent UAI 2026 paper by Chattopadhyay, Sarkar, and Kuchibhotla uses the general principle of inverting CDF bands through quantile functions, including DKW and Dümbgen–Wellner bands. So simply saying “replace DKW/KS by DW” is no longer enough as a novelty claim. [source citation: turn850055search1 turn804316search0]
But the location–scale case has much more structure than the general quantile-parametrized problem, and that structure gives you several promising research questions.

1. Start from a completely generic CDF band

Let

$$X_{(1)} \leq \cdots \leq X_{(n)}$$

be the order statistics.

Suppose you have *any* distribution-free simultaneous band represented by deterministic numbers

$$(\ell_i, u_i), \quad i = 1, \dots, n,$$

such that, for every continuous CDF F ,

$$P_F(\ell_i \leq F(X_{(i)}) \leq u_i \quad \forall i) \geq 1 - \alpha.$$

Why can these numbers be distribution-free? Because under continuity,

$$U_i = F(X_i) \sim U(0, 1),$$

so

$$F(X_{(i)}) = U_{(i)}.$$

Thus calibration is entirely a problem involving uniform order statistics. It does not care whether your eventual parametric family is normal, logistic, Gumbel, etc.

This is exactly the type of framework used in the recent quantile-band-inversion work. [source citation: turn804316search0]

Now suppose your parametric family is a location–scale family

$$F_{\mu, \sigma}(x) = G\left(\frac{x - \mu}{\sigma}\right), \quad \sigma > 0,$$

where G is known.

Its quantile function is

$$Q_{\mu, \sigma}(p) = \mu + \sigma G^{-1}(p).$$

Define

$$q_i^- = G^{-1}(\ell_i), \quad q_i^+ = G^{-1}(u_i).$$

Inverting the CDF band gives

$$Q_{\mu, \sigma}(\ell_i) \leq X_{(i)} \leq Q_{\mu, \sigma}(u_i).$$

Therefore

$$\mu + \sigma q_i^- \leq X_{(i)}$$

and

$$\mu + \sigma q_i^+ \geq X_{(i)}.$$

Equivalently,

$$\boxed{X_{(i)} - \sigma q_i^+ \leq \mu \leq X_{(i)} - \sigma q_i^-}.$$

And this is the key geometric observation.

Every order statistic contributes two half-planes. Every distribution-free band therefore generates a convex polyhedral confidence region in (μ, σ) -space.

The old KS result of Salvia is one special case; Salvia proved convexity and finiteness for KS consonance sets. [source citation: turn850055search0] Your general framework would ask what happens when the KS boundaries are replaced by modern, nonconstant-width boundaries.

2. Different CDF bands produce different polygons

For exactly the same dataset, you could construct

$$C_{\text{KS}}, \quad C_{\text{DKW}}, \quad C_{\text{BJ}}, \quad C_{\text{DW}}, \quad C_{\text{Frey}}, \dots$$

All have nominal simultaneous coverage, but they can have very different shapes and areas.

Here is the conceptual distinction:

Band	Main feature	Expected effect on (μ, σ) region	— — —	KS	Roughly constant vertical width
	Relatively weak tail information			DKW	Simple analytic constant-width bound
				Berk–Jones/Owen	Likelihood/KL-based, tail adaptive
					Potentially much tighter scale inference
	Dömbgen–Wellner	Modern multiscale refinement			Strong tails without sacrificing as much center performance
				Frey optimal bands	Optimize CDF-band narrowness directly
					Interesting benchmark for whether CDF-optimal = parameter-optimal

Owen’s nonparametric-likelihood band was constructed from the Berk–Jones idea and was motivated partly by improved behavior compared with weighted KS procedures. [source citation: turn854730search0] Dömbgen and Wellner’s 2023 construction explicitly aims to preserve strong tail performance while improving performance in the center. [source citation: turn850055search45 turn850055search47] Frey, meanwhile, directly studied distribution-free bands optimized according to band-narrowness criteria. [source citation: turn854730search2]

This is where a nice statistical question appears:

$$\boxed{\text{Does a better CDF band necessarily produce a better parameter confidence region?}}$$

Probably not according to every criterion.

And that distinction could be central to the research.

3. Why the tails matter disproportionately for σ

This is especially clear for the normal model.

For

$$X \sim N(\mu, \sigma^2),$$

we have

$$Q(p) = \mu + \sigma \Phi^{-1}(p).$$

The derivative with respect to the parameters is

$$\frac{\partial Q(p)}{\partial \mu} = 1, \quad \frac{\partial Q(p)}{\partial \sigma} = \Phi^{-1}(p).$$

Notice the difference.

For location,

$$\frac{\partial Q}{\partial \mu} = 1$$

at every quantile.

But for scale,

$$|\Phi^{-1}(p)|$$

gets large toward the tails.
So observations corresponding to, say,

$$p = 0.01, 0.99$$

can be much more informative geometrically about σ than observations near

$$p = 0.5.$$

This means that a constant-width KS/DKW band may be poorly matched to the inferential objective.

In fact, the recent UAI paper points out exactly why DKW can be inefficient near the endpoints: DKW has $O(n^{-1/2})$ width everywhere, whereas tail-adaptive constructions can become substantially tighter around extreme order statistics. Their simulations found the DW construction produced narrower induced confidence intervals than DKW in their Tukey-lambda experiments. [source citation: turn804316search0]

That gives you a statistical hypothesis worth studying:

Tail-adaptive CDF bands should be especially advantageous for scale inference.

You could quantify exactly how much.

4. The geometry becomes surprisingly tractable

There's another representation that I think could form a substantial part of the paper.

For a fixed value $s = \sigma$, define

$$L(s) = \max_i \{X_{(i)} - sq_i^+\},$$

and

$$U(s) = \min_i \{X_{(i)} - sq_i^-\}.$$

Then

$$C = \{(\mu, s) : s > 0, \quad L(s) \leq \mu \leq U(s)\}.$$

Now observe:

$$L(s) = \max_i \{\text{affine function of } s\}$$

is **piecewise-linear convex**, while

$$U(s) = \min_i \{\text{affine function of } s\}$$

is **piecewise-linear concave**.

Consequently,

$$H(s) = U(s) - L(s)$$

is piecewise-linear concave.

The allowed values of scale are simply

$$\{s : H(s) \geq 0\}.$$

Since a superlevel set of a concave function in one dimension is an interval,

$$\sigma \in [\sigma_L, \sigma_U]$$

automatically.

And the area of the entire joint confidence region is exactly

$$A(C) = \int_{\sigma_L}^{\sigma_U} [U(s) - L(s)] ds.$$

Because $U - L$ is piecewise linear, you can compute this **exactly**, rather than by Monte Carlo integration or grid approximation.

That's quite elegant.

It means you can take thousands of simulated datasets and calculate the exact polygon area cheaply.

5. You can even characterize the vertices

Every boundary line has the form

$$\mu = X_{(i)} - \sigma q_i^\pm.$$

Thus vertices occur at intersections of lines such as

$$X_{(i)} - \sigma q_i^+ = X_{(j)} - \sigma q_j^-.$$

Solving gives

$$\sigma = \frac{X_{(j)} - X_{(i)}}{q_j^- - q_i^+},$$

whenever the denominator has the appropriate sign.

So all possible vertices are generated by **ratios of observed spacings to theoretical quantile spacings**.

That starts to connect the method with classical location-scale statistics based on spacings and order statistics.

It may be possible to obtain explicit expressions for

$$\sigma_L, \quad \sigma_U$$

as maxima/minima over such ratios.

Then μ_L, μ_U follow from optimizing affine functions over the polygon.

So instead of writing a paper whose algorithm is

loop over 10^6 values of (μ, σ) ,

you get an exact computational-geometry algorithm.

That is considerably more attractive.

6. There is an extremely useful invariance property

Suppose the true data are

$$X_i = \mu_0 + \sigma_0 Z_i, \quad Z_i \sim G.$$

Parameterize a candidate by

$$\mu = \mu_0 + \sigma_0 a, \quad \sigma = \sigma_0 b.$$

Then

$$X_{(i)} = \mu_0 + \sigma_0 Z_{(i)}.$$

Substituting into the constraints shows that

$$(\mu, \sigma) \in C(X)$$

if and only if

$$(a, b) \in C(Z).$$

Hence

$$\boxed{C(X) = (\mu_0, 0) + \sigma_0 C(Z)}.$$

This has an important consequence:

$$\text{Area } C(X) = \sigma_0^2 \text{Area } C(Z).$$

Therefore

$$\frac{\text{Area } C(X)}{\sigma_0^2}$$

has a distribution that does **not depend on either μ_0 or σ_0 **.

That's excellent for a research study.

For a particular location-scale family G , you don't need to simulate

$$\mu = -5, 0, 5$$

and

$$\sigma = 0.5, 1, 5, 10.$$

You can simulate only

$$\mu_0 = 0, \quad \sigma_0 = 1.$$

The whole behavior of the method follows by equivariance.

That makes it possible to study the expected normalized area

$$R_B(n, \alpha; G) = E_{0,1} [\text{Area } C_B(X)]$$

for every candidate CDF band B .

Now you have a very clean performance criterion.

7. This leads to what I think is the strongest research question

Instead of asking:

Which existing CDF band is narrowest?

ask:

Which distribution-free CDF band minimizes the expected volume of the induced confidence region for the parameters of a specified parametric family?

Formally, let B correspond to boundaries

$$\{\ell_i(B), u_i(B)\}_{i=1}^n.$$

Every valid band satisfies

$$P(\ell_i(B) \leq U_{(i)} \leq u_i(B) \forall i) \geq 1 - \alpha.$$

It induces a region

$$C_B(X).$$

Then consider

$$B^* = \arg \min_{B \in \mathcal{B}_\alpha} E[\text{Area } C_B(X)].$$

This is different from minimizing the vertical area of the **CDF confidence band itself**.

Frey's work optimizes distribution-free CDF bands according to narrowness criteria. [source citation: turn854730search2]
What you would be asking is whether those bands remain optimal after projection through a particular parametric model.

There is no reason they necessarily should.

For normal location-scale inference, for example, you might willingly make the band **wider near $p = .5$ ** if that lets you make it much tighter at

$$p = .01, \quad .05, \quad .95, \quad .99,$$

because those quantiles may constrain σ much more strongly.

So the optimal band could depend on the inferential target:

optimal for estimating $F \neq$ optimal for estimating $\mu \neq$ optimal for estimating $\sigma \neq$ optimal for joint (μ, σ) .

That is a genuinely interesting idea.

8. You could define different optimality criteria
Your general objective could be

$$E[\text{Area}(C)]$$

for joint inference.

But you could also investigate

$$E[\sigma_U - \sigma_L],$$

if scale is the primary target, or

$$E[\mu_U - \mu_L]$$

for location.

Another possibility is minimax performance,

$$\inf_B \sup_{\theta} E_{\theta}[\text{Volume}(C_B)],$$

although for a location–scale family equivariance may simplify the supremum dramatically.

You could also consider expected log-volume,

$$E[\log \text{Volume}(C)],$$

which may behave better when occasional very large regions occur.

So one paper could actually distinguish:

$$B_{\mu}^*, \quad B_{\sigma}^*, \quad B_{\mu, \sigma}^*.$$

The best CDF band could change depending on the parameter being targeted.

9. Modern bands would give you natural benchmarks

I would probably start with four or five methods.

Use exact KS as the historical benchmark. Use DKW as the very simple analytic benchmark. Use Owen/Berk–Jones as the classical tail-adaptive likelihood benchmark; Owen explicitly constructed nonparametric likelihood confidence bands from the Berk–Jones idea. [source citation: turn854730search0] Then use Dümbgen–Wellner as the modern benchmark; their 2023 *Annals of Statistics* paper develops multiscale tests/bands intended to combine strong tail behavior with better center performance. [source citation: turn850055search45] Finally, Frey’s optimal distribution-free bands would be useful because they give you something explicitly optimized for CDF-band narrowness. [source citation: turn854730search2]

The computational side is no longer a major barrier either: modern algorithms can compute exact finite-sample thresholds for several KS-family statistics, including one-sided Berk–Jones-type statistics, at substantial n . [source citation: turn854730search4]

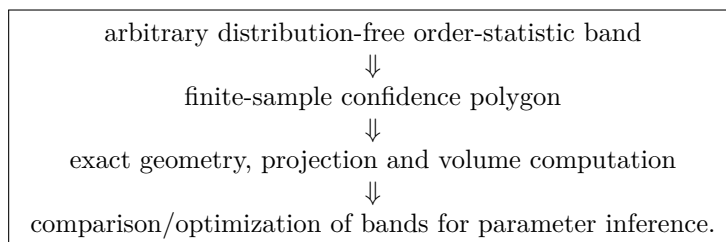
10. What I would make the first paper

I would narrow the first project to **two-parameter location–scale families**, rather than immediately trying arbitrary quantile-parametrized models.

A possible title would be:

Distribution-Free Confidence Bands and Polyhedral Inference for Location–Scale Families

The main contribution would be something like:



The empirical study could then use several baseline distributions:

$G = \text{Normal, Logistic, } t_\nu, \text{ Laplace,}$

because their different tail structures would tell you whether the ranking of bands depends on G .
My strongest suspicion is that it will.
For a heavy-tailed G , the slopes

$$G^{-1}(\ell_i), \quad G^{-1}(u_i)$$

behave very differently from the normal case, so the same CDF band can produce a very different polygon.

The important novelty warning

The August 2026 UAI paper is now extremely relevant. It explicitly develops

$$Q_\Lambda(\ell_i) \leq X_{(i)} \leq Q_\Lambda(u_i)$$

as a general inference framework and compares DKW with Dümbgen–Wellner; in their application DW produces narrower intervals while maintaining coverage. [source citation: turn804316search0 turn178351search0]

So I **would not** pitch your contribution as:

“We discover that modern CDF bands can be inverted to get better parametric confidence sets.”

That is now clearly already being done.

I would instead push the idea toward:

polyhedral geometry + location–scale equivariance + parameter-targeted band optimality.
--

The location–scale restriction is actually an advantage here, because it converts the very general UAI construction into an object that can be analyzed almost completely: affine inequalities, convex polygons, exact projections, exact area, invariance, and potentially analytically optimized band design.

That, to me, is the version that starts looking like a real theoretical statistics research project rather than just an application of an existing method.

Turn 7

User

What is next?

Assistant

The next step is to turn the idea into a **small, testable research program** rather than keep expanding the literature search.

I would do this in this order:

1. **Formalize the general theorem.** State precisely the class of bands you allow:

$$\Pr\{\ell_i \leq F(X_{(i)}) \leq u_i, \forall i\} \geq 1 - \alpha,$$

and prove that for

$$F_{\mu,\sigma}(x) = G\left(\frac{x - \mu}{\sigma}\right),$$

the induced confidence set is a convex polygon/polyhedron in (μ, σ) . Then derive the piecewise-linear lower and upper envelopes

$$L(\sigma), \quad U(\sigma).$$

2. **Derive the computational consequences.** Work out exact formulas or algorithms for:

$$\sigma_L, \sigma_U, \mu_L, \mu_U,$$

polygon vertices, and

$$\text{Area}(C).$$

Ideally, show that no two-dimensional parameter grid is needed.

3. **Prove equivariance.** Establish that under

$$X_i = \mu_0 + \sigma_0 Z_i,$$

the normalized confidence-region geometry depends only on G , n , α , and the chosen band. This is important because it collapses a huge simulation study to simulations from G with $(\mu, \sigma) = (0, 1)$.

4. **Pick 3–4 bands and run the first numerical experiment.** Start with KS/DKW plus two tail-adaptive competitors such as Berk–Jones/Owen and Dümbgen–Wellner. For each band, estimate:

$$E[\text{Area}(C)], \quad E[\mu_U - \mu_L], \quad E[\sigma_U - \sigma_L].$$

Use normal G first and perhaps $n = 20, 50, 100, 200$.

5. **Look for the phenomenon before proving too much.** The key empirical question is whether the ranking of bands changes depending on the target. For example, you might find

$$B_1 \text{ best for } \mu, \quad B_2 \text{ best for } \sigma, \quad B_3 \text{ best for joint area.}$$

If that happens, you have a much stronger research story: **CDF-band optimality and parameter-inference optimality are different problems.**

6. **Only then formulate an optimization problem.** Instead of merely comparing existing bands, consider designing (ℓ_i, u_i) to minimize

$$E[\text{Area}(C)]$$

subject to the simultaneous distribution-free coverage constraint. That is where the project could become genuinely novel.

The immediate milestone I would aim for is much smaller:

Proposition + exact polygon algorithm + one simulation figure comparing 4 bands.

If that figure shows a meaningful difference between the bands, you have evidence that the project is worth pursuing. If all the induced regions are nearly identical, you learn that early rather than spending months on theory.

A good first concrete question is:

For $X_i \sim N(\mu, \sigma^2)$, among KS, DKW, Berk–Jones, and Dümbgen–Wellner simultaneous bands, which gives the smallest expected normalized area of the induced 95% confidence region for (μ, σ) , as a function of n ?

That is narrow enough to answer computationally, but directly connected to the broader theory.

Turn 8

User

Can you work out exact formulas or algorithms for mu and sigma?

Assistant

Yes. For the location-scale case, the projections onto both σ and μ can be worked out exactly. The scale interval has a particularly clean pairwise formula; the location interval can then be obtained by a one-dimensional convex-envelope calculation or, equivalently, two tiny linear programs.

Suppose

$$X_{(1)} \leq \cdots \leq X_{(n)}$$

and your simultaneous CDF band gives

$$\ell_i \leq F(X_{(i)}) \leq u_i.$$

For the location-scale family

$$F_{\mu,\sigma}(x) = G\left(\frac{x-\mu}{\sigma}\right), \quad \sigma > 0,$$

define

$$a_i = G^{-1}(\ell_i), \quad b_i = G^{-1}(u_i),$$

with $a_i \leq b_i$.

Then the confidence region is exactly

$$C = \{(\mu, \sigma) : x_i - \sigma b_i \leq \mu \leq x_i - \sigma a_i, \quad i = 1, \dots, n, \quad \sigma > 0\},$$

where I write $x_i = X_{(i)}$ for brevity.

1. Exact interval for σ

For a given σ , there exists a valid μ iff

$$\max_i (x_i - \sigma b_i) \leq \min_j (x_j - \sigma a_j).$$

Equivalently, every lower bound must be below every upper bound:

$$x_i - \sigma b_i \leq x_j - \sigma a_j \quad \forall i, j.$$

Rearrange:

$$\sigma(a_j - b_i) \leq x_j - x_i.$$

Define

$$d_{ij} = a_j - b_i, \quad \Delta_{ij} = x_j - x_i.$$

Then each pair (i, j) gives one of three possibilities.

If

$$d_{ij} > 0,$$

then

$$\sigma \leq \frac{\Delta_{ij}}{d_{ij}}.$$

If

$$d_{ij} < 0,$$

then

$$\sigma \geq \frac{\Delta_{ij}}{d_{ij}}.$$

If

$$d_{ij} = 0,$$

we simply require

$$\Delta_{ij} \geq 0.$$

Therefore the exact projected confidence interval for scale is

$$\sigma_L = \max \left\{ 0, \max_{i,j: a_j - b_i < 0} \frac{x_j - x_i}{a_j - b_i} \right\},$$

and

$$\sigma_U = \min_{i,j: a_j - b_i > 0} \frac{x_j - x_i}{a_j - b_i}.$$

So

$$C_\sigma = [\sigma_L, \sigma_U]$$

subject to $\sigma > 0$.

This is already quite useful: **there is no optimization over σ at all.** You calculate pairwise ratios and take a maximum and minimum.

A simple interpretation

The critical values are ratios

$$\frac{\text{observed spacing}}{\text{theoretical quantile spacing}}.$$

That is very natural for scale inference.

For example, an active upper bound can come from

$$\sigma \leq \frac{x_j - x_i}{a_j - b_i}.$$

So the maximum admissible scale is determined by some pair of order statistics whose observed separation is too small to accommodate a larger theoretical quantile separation.

2. Exact allowable μ for any given σ

Define

$$L(\sigma) = \max_i \{x_i - \sigma b_i\},$$

and

$$U(\sigma) = \min_i \{x_i - \sigma a_i\}.$$

Then, conditional on any admissible σ ,

$$\mu \in [L(\sigma), U(\sigma)].$$

Thus the entire joint region has the representation

$$C = \{(\mu, \sigma) : \sigma_L \leq \sigma \leq \sigma_U, \quad L(\sigma) \leq \mu \leq U(\sigma)\}.$$

This is probably the most convenient representation computationally.

Moreover,

$$L(\sigma) = \max_i (x_i - b_i \sigma)$$

is convex and piecewise linear, while

$$U(\sigma) = \min_i (x_i - a_i \sigma)$$

is concave and piecewise linear.

3. Exact marginal interval for μ
The marginal confidence interval is

$$[\mu_L, \mu_U],$$

where

$$\mu_L = \min_{\sigma \in [\sigma_L, \sigma_U]} L(\sigma)$$

and

$$\mu_U = \max_{\sigma \in [\sigma_L, \sigma_U]} U(\sigma).$$

These are very easy one-dimensional optimization problems because L and U are piecewise linear. There are several exact ways to solve them.

Method A: line intersections
Every lower-bound line is

$$L_i(\sigma) = x_i - b_i \sigma.$$

The minimum of

$$L(\sigma) = \max_i L_i(\sigma)$$

can occur only at:

- σ_L ,
- σ_U , or
- an intersection of two active lower lines.

The intersection of lines i and j occurs at

$$x_i - b_i \sigma = x_j - b_j \sigma,$$

hence

$$\sigma_{ij}^{(L)} = \frac{x_j - x_i}{b_j - b_i}$$

when $b_i \neq b_j$.

Keep only intersections satisfying

$$\sigma_L \leq \sigma_{ij}^{(L)} \leq \sigma_U.$$

Evaluate

$$L(\sigma)$$

at these candidates and at the two endpoints. The smallest value is exactly μ_L . Similarly, upper lines are

$$U_i(\sigma) = x_i - a_i \sigma.$$

Their intersections are

$$\sigma_{ij}^{(U)} = \frac{x_j - x_i}{a_j - a_i}.$$

Evaluate

$$U(\sigma) = \min_i U_i(\sigma)$$

at feasible intersections and endpoints; the largest value is exactly μ_U .

So this is finite and exact.

4. An even cleaner formulation: two linear programs

Because the original region consists entirely of linear inequalities, obtaining marginal bounds is simply linear programming.

For μ_L , solve

$$\begin{array}{ll} \text{minimize} & \mu \\ \text{subject to} & \mu + \sigma b_i \geq x_i, \\ & \mu + \sigma a_i \leq x_i, \quad i = 1, \dots, n, \\ & \sigma \geq 0. \end{array}$$

For μ_U , change “minimize” to “maximize.”

For σ_L, σ_U , minimize/maximize σ under exactly the same constraints.

So all four endpoints

$$\mu_L, \mu_U, \sigma_L, \sigma_U$$

are solutions of four two-variable linear programs.

For actual software, this may be the simplest robust implementation.

But mathematically, the pairwise formula for σ is more informative.

5. The vertices can also be calculated exactly

Every polygon boundary is one of

$$\mu = x_i - b_i \sigma$$

or

$$\mu = x_j - a_j \sigma.$$

An especially important vertex occurs when a lower constraint and upper constraint meet:

$$x_i - b_i \sigma = x_j - a_j \sigma.$$

Thus

$$\sigma_{ij} = \frac{x_j - x_i}{a_j - b_i}.$$

Then

$$\mu_{ij} = x_i - b_i \sigma_{ij} = x_j - a_j \sigma_{ij}.$$

So candidate polygon vertices have completely explicit coordinates.

You only retain a candidate (μ_{ij}, σ_{ij}) if

$$\sigma_{ij} > 0$$

and it satisfies all the other $2n$ inequalities.

There are also vertices generated by lower/lower and upper/upper line intersections as the active envelope changes.

6. There is a faster algorithm than checking all n^2 intersections

Naively, the pairwise method is $O(n^2)$.

But there is additional structure because

$$a_i, b_i$$

usually increase with i .

For instance, with typical order-statistic bands,

$$\ell_1 \leq \ell_2 \leq \dots \leq \ell_n$$

and therefore

$$a_1 \leq a_2 \leq \cdots \leq a_n.$$

Similarly for b_i .
Thus the lines

$$x_i - b_i \sigma$$

have ordered slopes.

This means $L(\sigma)$ can be constructed using the same type of algorithm as a **convex hull trick**: lines that can never be active are discarded.

Likewise for $U(\sigma)$.

So instead of working with all n lines throughout, you construct two envelopes:

$$L(\sigma) = \max_{k \in I_L} (x_k - b_k \sigma),$$

$$U(\sigma) = \min_{k \in I_U} (x_k - a_k \sigma),$$

where typically

$$|I_L|, |I_U| \ll n.$$

Because the slopes are ordered, the envelope itself can be constructed in roughly $O(n)$ after sorting—which we already have because these are order statistics.

That gives you an elegant computational result:

The confidence polygon can potentially be constructed in linear time in n .

This is one aspect I'd investigate carefully for the research project.

7. Exact area follows immediately

Once you know the breakpoints

$$\sigma_L = s_0 < s_1 < \cdots < s_K = \sigma_U$$

of the two envelopes, over each interval

$$[s_k, s_{k+1}]$$

both L and U are affine:

$$L(s) = c_k + d_k s,$$

$$U(s) = e_k + f_k s.$$

Therefore

$$U(s) - L(s) = (e_k - c_k) + (f_k - d_k)s.$$

Hence

$$\text{Area}(C) = \sum_{k=0}^{K-1} \int_{s_k}^{s_{k+1}} [U(s) - L(s)] ds.$$

Explicitly,

$$A(C) = \sum_k \left[(e_k - c_k)(s_{k+1} - s_k) + \frac{f_k - d_k}{2} (s_{k+1}^2 - s_k^2) \right].$$

So even the joint-region area is exact—no numerical integration is necessary.

8. Normal example

For a normal model,

$$G = \Phi.$$

Therefore

$$a_i = \Phi^{-1}(\ell_i), \quad b_i = \Phi^{-1}(u_i).$$

Your exact scale interval becomes

$$\sigma_L = \max \left\{ 0, \max_{\Phi^{-1}(\ell_j) - \Phi^{-1}(u_i) < 0} \frac{x_j - x_i}{\Phi^{-1}(\ell_j) - \Phi^{-1}(u_i)} \right\},$$

and

$$\sigma_U = \min_{\Phi^{-1}(\ell_j) - \Phi^{-1}(u_i) > 0} \frac{x_j - x_i}{\Phi^{-1}(\ell_j) - \Phi^{-1}(u_i)}.$$

Then

$$\mu_L = \min_{\sigma_L \leq \sigma \leq \sigma_U} \max_i [x_i - \sigma \Phi^{-1}(u_i)],$$

and

$$\mu_U = \max_{\sigma_L \leq \sigma \leq \sigma_U} \min_i [x_i - \sigma \Phi^{-1}(\ell_i)].$$

These are **finite-sample exact projections of the inverted band**, not asymptotic approximations.

One subtlety is important: these are simultaneous/marginal projections of the ****joint $1 - \alpha$ region****. Thus $[\mu_L, \mu_U]$ and $[\sigma_L, \sigma_U]$ each have at least $1 - \alpha$ coverage, but they are generally more conservative than separately optimized $1 - \alpha$ intervals for each parameter.

The most interesting technical next step, in my view, is proving an efficient envelope algorithm and seeing whether the monotonic order-statistic structure lets us reduce the whole construction from naive $O(n^2)$ to $O(n)$ or $O(n \log n)$. That could be a genuinely useful component of the research project.

Turn 9

User

Can you prove the efficient envelope algorithm?

Assistant

Yes. Under the natural condition that the band limits are monotone in i , the envelopes can be constructed in $O(n)$ time after the sample has been sorted.

Assume

$$x_1 \leq \dots \leq x_n, \quad a_1 \leq \dots \leq a_n, \quad b_1 \leq \dots \leq b_n,$$

where

$$a_i = G^{-1}(\ell_i), \quad b_i = G^{-1}(u_i).$$

Recall

$$L(\sigma) = \max_i \{x_i - b_i \sigma\},$$

and

$$U(\sigma) = \min_i \{x_i - a_i \sigma\}.$$

The important fact is that the lines already have **monotonically ordered slopes**.

1. Generic envelope lemma

Consider lines

$$f_i(t) = c_i + m_i t,$$

with strictly increasing slopes

$$m_1 < m_2 < \dots < m_n.$$

Define their upper envelope

$$H(t) = \max_i f_i(t).$$

Then the lines that actually appear on H can be found in $O(n)$.

The central observation is simple.

If $i < j$, then

$$f_j(t) - f_i(t) = (c_j - c_i) + (m_j - m_i)t.$$

Since

$$m_j - m_i > 0,$$

this difference is strictly increasing in t .

Therefore two lines cross at most once, and after the higher-slope line overtakes the lower-slope line, it remains above it forever.

Hence, as t moves from $-\infty$ to $+\infty$, the slopes of the active envelope lines can only increase.

That monotonicity is what makes the stack algorithm possible.

2. Suppose we already have an envelope

Suppose after processing some lines we have active lines

$$h_1, \dots, h_r$$

with increasing slopes, and breakpoints

$$-\infty = z_1 < z_2 < \dots < z_r,$$

such that

$$h_j(t) = H(t)$$

for

$$z_j \leq t \leq z_{j+1},$$

where $z_{r+1} = +\infty$.

Now add a new line

$$q(t) = c_q + m_q t$$

whose slope is larger than all previous slopes:

$$m_q > m(h_r).$$

Let its intersection with the last envelope line be

$$t^* = \frac{c(h_r) - c_q}{m_q - m(h_r)}.$$

There are two possibilities.

Case 1: $t^* > z_r$

Then h_r remains the envelope on

$$[z_r, t^*],$$

and q takes over after t^* .

So simply append q , with breakpoint t^* .

Case 2: $t^* \leq z_r$

Then h_r becomes completely irrelevant.

Why?

For

$$t < z_r,$$

h_r was not on the old envelope anyway.

For

$$t \geq z_r,$$

because q has the larger slope and crossed h_r at or before z_r ,

$$q(t) \geq h_r(t).$$

Therefore there is **no** t^* for which h_r can be the new envelope.

So we can permanently delete h_r .

Then compare q with h_{r-1} .

Repeat until the intersection occurs after the surviving line's breakpoint.

That proves the correctness of the stack deletion rule.

3. Why the algorithm is linear

Each line is:

- inserted onto the stack once;
- removed from the stack at most once.

Therefore, although a single new line may delete several previous lines, over the whole algorithm there can be at most n deletions.

Thus

$$T(n) = O(n).$$

This is the same amortized argument used in monotone convex-hull algorithms.

4. Apply it to $L(\sigma)$

We have

$$L(\sigma) = \max_i \underbrace{(x_i - b_i \sigma)}_{L_i(\sigma)}.$$

The slope of L_i is

$$m_i = -b_i.$$

Since

$$b_1 \leq b_2 \leq \dots \leq b_n,$$

we have

$$-b_1 \geq -b_2 \geq \dots \geq -b_n.$$

So the slopes are in decreasing order.
Reverse the lines:

$$L_n, L_{n-1}, \dots, L_1.$$

Their slopes are now increasing:

$$-b_n \leq \dots \leq -b_1.$$

Therefore the generic algorithm applies directly.
For two candidate lines,

$$L_i(\sigma) = x_i - b_i \sigma,$$

and

$$L_j(\sigma) = x_j - b_j \sigma,$$

their intersection is

$$x_i - b_i \sigma = x_j - b_j \sigma,$$

so

$$\boxed{\sigma_{ij} = \frac{x_j - x_i}{b_j - b_i}.$$

Those intersection values become the stack breakpoints.
Thus after one linear pass we obtain indices

$$i_1, \dots, i_r$$

and breakpoints

$$z_1 < \dots < z_r$$

such that

$$L(\sigma) = x_{i_k} - b_{i_k} \sigma$$

on each corresponding interval.

5. Equal slopes
Suppose

$$b_i = b_j.$$

Then the lines are parallel:

$$L_i(\sigma) = x_i - b_i \sigma,$$

$$L_j(\sigma) = x_j - b_i\sigma.$$

Since $x_i \leq x_j$ for $i < j$,

$$L_j(\sigma) \geq L_i(\sigma) \quad \forall \sigma.$$

Therefore L_i can never contribute to the upper envelope.

So among equal b_i 's, keep only the line with the ****largest x_i ****.

This preprocessing is also $O(n)$.

6. $U(\sigma)$ follows immediately

Recall

$$U(\sigma) = \min_i (x_i - a_i\sigma).$$

Instead define

$$V(\sigma) = -U(\sigma).$$

Then

$$V(\sigma) = \max_i (-x_i + a_i\sigma).$$

Now the slopes are simply

$$a_1 \leq a_2 \leq \dots \leq a_n.$$

So they are already in increasing order.

Apply exactly the same upper-envelope algorithm to the lines

$$V_i(\sigma) = -x_i + a_i\sigma.$$

Once V is constructed,

$$\boxed{U(\sigma) = -V(\sigma).}$$

For equal a_i , we keep the largest intercept $-x_i$, meaning the ****smallest x_i ****.

7. Explicit algorithm

For an upper envelope of lines

$$f(t) = c + mt$$

arriving in increasing slope order:

```
H = empty stack of lines
Z = empty stack of starting breakpoints

for each line q in increasing slope order:
    if q has same slope as last line:
        keep only the one with larger intercept

    while H is not empty:
        t = intersection(last(H), q)

        if H has one line or t > last(Z):
            break

        pop last(H)
        pop last(Z)

    if H is empty:
        push q onto H
        push -infinity onto Z
    else:
        t = intersection(last(H), q)
        push q onto H
        push t onto Z
```

At termination,

$$Z_k$$

is the point at which line H_k starts being the maximizer.
So if

$$Z_k \leq t < Z_{k+1},$$

then

$$H(t) = H_k(t).$$

8. Restricting to $\sigma > 0$

We actually only care about

$$\sigma \in [\sigma_L, \sigma_U].$$

The full envelope can be constructed first, after which we simply remove pieces lying entirely outside this interval.

Alternatively, because the breakpoints are sorted, locating σ_L and σ_U in the envelope takes $O(\log n)$ by binary search, or $O(n)$ during a single sequential traversal.

Hence the total complexity remains

$$\boxed{O(n)}$$

once x_i are sorted.

If sorting the raw sample is included, the total procedure is

$$\boxed{O(n \log n)}$$

because obtaining the order statistics dominates the envelope construction.

9. A stronger consequence

Suppose the lower envelope uses r active lines and the upper envelope uses s active lines.

Then there are only

$$r + s$$

relevant boundary pieces—not $2n$ in practice.

Moreover, the breakpoints of L and U are each sorted. Therefore we can **merge the two breakpoint lists in linear time**, exactly as in merge sort.

On each resulting interval,

$$L(\sigma) = \alpha_k + \beta_k \sigma,$$

and

$$U(\sigma) = \gamma_k + \delta_k \sigma.$$

Consequently

$$H(\sigma) = U(\sigma) - L(\sigma)$$

is affine on that interval.

That means we can obtain, in one additional linear pass:

$$\sigma_L, \sigma_U,$$

all polygon vertices,

$$\mu_L, \mu_U,$$

and

$$\text{Area}(C) = \int_{\sigma_L}^{\sigma_U} [U(\sigma) - L(\sigma)] d\sigma.$$

So after sorting the data, the **entire confidence polygon and all its basic summaries can be constructed in linear time**:

$$\boxed{\text{sorted data} \implies \text{complete confidence region in } O(n).}$$

There is one assumption I would make explicit in a theorem: the band needs a_i and b_i to be monotone in i . KS, DKW, and the usual order-statistic simultaneous bands have this structure. Without monotone band endpoints, the polyhedral representation remains valid, but the automatic $O(n)$ monotone-slope algorithm no longer follows; you'd generally sort the lines by slope first, giving $O(n \log n)$.

This theorem is probably worth writing formally as part of the project, because it turns the inference problem from an apparent $O(n^2)$ pairwise calculation—or a two-dimensional grid search—into an exact convex-hull computation.

Turn 10

User

If I understand correctly, we have all the components to execute the first research plan, i.e. given a band we can construct a convex confidence set for (μ, σ) . We can even compute its area, and the lengths of marginal intervals analytically. Now we need to take several confidence bands and compare them by this 3 criteria, right?

Assistant

Yes. That is the right next phase.

You now have the core pipeline:

$$\text{band } B \longrightarrow (\ell_i, u_i) \longrightarrow C_B(X) \longrightarrow \left(\text{Area}(C_B), \mu_U - \mu_L, \sigma_U - \sigma_L \right).$$

The first research plan can therefore be made very concrete.

For a fixed location-scale family G , sample size n , and confidence level $1 - \alpha$:

$$X_i \sim G_{\mu, \sigma},$$

you choose several simultaneous distribution-free bands, for example KS, DKW, Berk-Jones/Owen, and Dümbgen-Wellner. For each simulated sample, each band induces its own confidence polygon

$$C_B(X).$$

Then compute exactly:

$$A_B(X) = \text{Area}(C_B(X)),$$

$$L_{\mu, B}(X) = \mu_U - \mu_L,$$

$$L_{\sigma, B}(X) = \sigma_U - \sigma_L.$$

Because of location-scale equivariance, you can normally simulate only from

$$\mu = 0, \quad \sigma = 1.$$

Then compare quantities such as

$$E[A_B], \quad E[L_{\mu, B}], \quad E[L_{\sigma, B}].$$

Those are the three most natural primary criteria.

I would add one important fourth check, though:

empirical coverage

even if the bands theoretically have $1 - \alpha$ simultaneous coverage. It serves as a validation of the implementation and becomes especially important if some bands use asymptotic rather than exact finite-sample calibration.

So a simulation table might look like:

Band	Coverage	Mean area	Mean μ -length	Mean σ -length	— — — — —	KS	0.951	...	...
...	DKW	0.978	...	...	Berk-Jones	0.950	...	...	...
					Dümbgen-Wellner	0.951	...	...	...

But there is one subtle issue: **raw expected size is not entirely fair if the procedures have different actual coverage**. If DKW gives 98% coverage while another method gives 95%, of course DKW may have a larger region.

So ideally you compare either bands calibrated to the **same exact coverage**, or report size together with actual coverage.

That itself may become part of the paper.

A very clean first experiment would be:

$$G = \Phi, \quad 1 - \alpha = .95,$$

with

$$n \in \{20, 50, 100, 200\}.$$

For each n , simulate perhaps many samples from $N(0, 1)$, construct every band's region, and estimate

$$E[A_B], \quad E[L_{\mu,B}], \quad E[L_{\sigma,B}].$$

Then plot relative efficiencies, for example

$$\frac{E[A_B]}{E[A_{\text{best}}]},$$

and similarly for marginal lengths.

The really interesting outcome would be if the rankings differ:

$$B_1 \text{ minimizes area,}$$

while

$$B_2 \text{ minimizes } \mu\text{-length,}$$

and

$$B_3 \text{ minimizes } \sigma\text{-length.}$$

That would immediately support the broader thesis that

there is no universally “best” CDF band once the target is parametric inference.

And after that first normal experiment, the natural second step is to repeat it for several G 's—say logistic, Laplace, and Student- t —to see whether the ranking of bands depends on tail behavior.

So yes: **you are basically at the point where the theory is sufficient to run the first serious numerical study.** The next real work is choosing exactly which bands to implement and making sure their finite-sample calibration is comparable.

Turn 11

User

Now the question is: how do I compute KS, DKW, Berk–Jones and Dümbgen–Wellner bounds?

Assistant

Yes. The cleanest way is to put all four methods into the same representation:

$$\Pr \{ \ell_i \leq F(X_{(i)}) \leq u_i, \quad i = 1, \dots, n \} \geq 1 - \alpha.$$

Once you have the deterministic vectors

$$(\ell_1, \dots, \ell_n), \quad (u_1, \dots, u_n),$$

everything we derived for the (μ, σ) polygon applies unchanged.

The four bands differ only in how these ℓ_i, u_i are obtained.

1. KS band

Let

$$D_n = \sup_x |F_n(x) - F(x)|.$$

Choose $d_{n,\alpha}$ such that

$$P(D_n \leq d_{n,\alpha}) = 1 - \alpha.$$

Equivalently, if

$$\kappa_{n,\alpha}^{KS}$$

is the $1 - \alpha$ quantile of

$$\sqrt{n} D_n,$$

then

$$d_{n,\alpha} = \frac{\kappa_{n,\alpha}^{KS}}{\sqrt{n}}.$$

The exact KS confidence band is

$$\max\{F_n(x) - d_{n,\alpha}, 0\} \leq F(x) \leq \min\{F_n(x) + d_{n,\alpha}, 1\}.$$

Dümbgen and Wellner use precisely this finite-sample KS construction. [source citation: turn960069view3]

At the order statistic $X_{(i)}$,

$$F_n(X_{(i)}) = \frac{i}{n}.$$

So for our purposes,

$$\ell_i^{KS} = \max\left(0, \frac{i}{n} - d_{n,\alpha}\right)$$

and

$$u_i^{KS} = \min\left(1, \frac{i}{n} + d_{n,\alpha}\right).$$

How to get $d_{n,\alpha}$

You have two reasonable options.

Exact: use the finite-sample distribution of the two-sided KS statistic.

Monte Carlo: simulate

$$U_1, \dots, U_n \stackrel{iid}{\sim} U(0, 1),$$

calculate

$$D_n = \max_{i=1,\dots,n} \left\{ \frac{i}{n} - U_{(i)}, U_{(i)} - \frac{i-1}{n} \right\},$$

and take its $1 - \alpha$ quantile.

For a research comparison, I'd use the exact finite-sample KS quantile rather than its asymptotic $1.358/\sqrt{n}$ approximation.

2. DKW band

This one is immediate.

The DKW–Massart inequality gives

$$P \left(\sup_x |F_n(x) - F(x)| > \epsilon \right) \leq 2e^{-2n\epsilon^2}.$$

Set

$$2e^{-2n\epsilon^2} = \alpha,$$

so

$$\epsilon_{n,\alpha} = \sqrt{\frac{\log(2/\alpha)}{2n}}.$$

Therefore

$$\ell_i^{DKW} = \max \left(0, \frac{i}{n} - \sqrt{\frac{\log(2/\alpha)}{2n}} \right)$$

and

$$u_i^{DKW} = \min \left(1, \frac{i}{n} + \sqrt{\frac{\log(2/\alpha)}{2n}} \right).$$

This is exactly the order-statistic formulation used in the recent CDF-band inversion work. [source citation: turn349284view0 turn349284view1]

The important distinction is:

KS uses an exact distributional critical value; DKW uses an inequality.

Hence DKW will generally be wider/conservative.

3. Berk–Jones / Owen band

Now it becomes more interesting.

Define the Bernoulli KL divergence

$$K(p, q) = p \log \frac{p}{q} + (1 - p) \log \frac{1 - p}{1 - q}.$$

Use the usual limiting conventions such as

$$0 \log 0 = 0.$$

The Berk–Jones statistic is

$$T_n^{BJ} = n \sup_x K(F_n(x), F(x)).$$

Dümbgen and Wellner give this formulation explicitly and describe Owen's confidence bands as its inversion. [source citation: turn438587view0]

Under a continuous F , transform to uniforms:

$$U_i = F(X_i).$$

Then

$$T_n^{BJ} \stackrel{d}{=} \max_{1 \leq i \leq n} \max \left\{ nK \left(\frac{i-1}{n}, U_{(i)} \right), nK \left(\frac{i}{n}, U_{(i)} \right) \right\}.$$

That finite $2n$ -term representation is the convenient way to calibrate it. [source citation: turn438587search22 turn168581vi
Let

$$\kappa_{n,\alpha}^{BJ}$$

be its $1 - \alpha$ quantile.

Then for each empirical level

$$p_i = \frac{i}{n},$$

we invert

$$nK(p_i, q) \leq \kappa_{n,\alpha}^{BJ}.$$

Since $K(p_i, q)$ is minimized at $q = p_i$, the solution is an interval

$$[\ell_i^{BJ}, u_i^{BJ}].$$

Thus

$$\boxed{\ell_i^{BJ} = \min \{q \leq p_i : nK(p_i, q) \leq \kappa_{n,\alpha}^{BJ}\}}$$

and

$$\boxed{u_i^{BJ} = \max \{q \geq p_i : nK(p_i, q) \leq \kappa_{n,\alpha}^{BJ}\}}.$$

This is exactly how the Berk–Jones/Owen band is described in the original and later literature. [source citation: turn501338view0]

Numerically

There is generally no elementary closed form for q .

But this is trivial numerically because you solve a **monotone scalar equation**:

$$nK(p_i, q) = \kappa_{n,\alpha}^{BJ}.$$

For the lower bound search on

$$q \in (0, p_i),$$

and for the upper bound search on

$$q \in (p_i, 1).$$

Bisection or Brent’s method is more than sufficient.

So Berk–Jones consists of:

$$\boxed{\text{calibrate one } \kappa^{BJ} + \text{solve } 2n \text{ one-dimensional equations.}}$$

4. Dümbgen–Wellner

DW modifies Berk–Jones by subtracting a location-dependent correction.

For the version I would recommend for your first study, take

$$s = 1, \quad \nu = 1.$$

Then the divergence part remains ordinary KL:

$$K(p, q).$$

Define

$$C(t) = \log \log \left(\frac{e}{4t(1-t)} \right),$$

and

$$D(t) = \log(1 + C(t)^2),$$

then

$$C_\nu(t) = C(t) + \nu D(t).$$

Dümbgen and Wellner recommend $\nu > 3/4$, and their finite-sample tables use $\nu = 1$. [source citation: turn168581view0]

The bivariate correction is

$$C_\nu(p, q) = \min_{\min(p, q) \leq t \leq \max(p, q)} C_\nu(t).$$

Because $C_\nu(t)$ is symmetric around $1/2$, this has a very simple implementation:

$$C_\nu(p, q) = \begin{cases} C_\nu(\max\{p, q\}), & p, q < 1/2, \\ C_\nu(\min\{p, q\}), & p, q > 1/2, \\ 0, & 1/2 \in [\min(p, q), \max(p, q)]. \end{cases}$$

That is directly from their definition. [source citation: turn168581view0]

Now define

$$H_{n,\nu}(p, q) = nK(p, q) - C_\nu(p, q).$$

The DW statistic is

$$T_{n,\nu}^{DW} = \sup_x \{nK(F_n(x), F(x)) - C_\nu(F_n(x), F(x))\}.$$

Under continuity its null distribution is distribution-free, so again you calibrate it using $\text{Uniform}(0, 1)$ samples. [source citation: turn168581view0]

Even better, you do **not** need a numerical supremum over t . For $s > 0$, Dümbgen and Wellner show

$$T_{n,\nu}^{DW} = \max_{1 \leq i \leq n} \max \left\{ H_{n,\nu} \left(\frac{i-1}{n}, U_{(i)} \right), H_{n,\nu} \left(\frac{i}{n}, U_{(i)} \right) \right\}.$$

So again, only $2n$ terms per simulated sample. [source citation: turn168581view0]

Let

$$\kappa_{n,\nu,\alpha}^{DW}$$

be its $1 - \alpha$ quantile.

Then for

$$p_i = \frac{i}{n},$$

solve

$$H_{n,\nu}(p_i, q) \leq \kappa_{n,\nu,\alpha}^{DW}.$$

That gives

$$\ell_i^{DW} = \min \{q < p_i : nK(p_i, q) - C_\nu(p_i, q) \leq \kappa_{n,\nu,\alpha}^{DW}\}$$

and

$$u_i^{DW} = \max \{q > p_i : nK(p_i, q) - C_\nu(p_i, q) \leq \kappa_{n,\nu,\alpha}^{DW}\}.$$

This is precisely the confidence-band inversion given by Dümbgen–Wellner and reused in the 2026 quantile-inference paper. [source citation: turn501338view0 turn349284view1]

Again, solve each endpoint by a scalar root finder.

So computationally, the four methods are surprisingly unified

For each n, α , you calculate the following **once**, before doing your main parameter simulations:

Method	Calibration	Endpoint calculation	KS	exact KS quantile	explicit	DKW
analytic ϵ	explicit	BJ	Uniform simulation/exact recursion for κ^{BJ}	root solve KL equation	DW	
Uniform simulation for κ^{DW}			root solve corrected KL equation			

Then save

$$\ell_i^B, u_i^B, \quad i = 1, \dots, n$$

for every method B .

These numbers do **not depend on your observed data** and do ****not** depend on G, μ, σ^{**} .

That is an important computational advantage.

For example, for

$$n = 100, \quad \alpha = .05,$$

you compute four pairs of vectors once:

$$(\ell^{KS}, u^{KS}),$$

$$(\ell^{DKW}, u^{DKW}),$$

$$(\ell^{BJ}, u^{BJ}),$$

$$(\ell^{DW}, u^{DW}).$$

Then every Monte Carlo dataset uses exactly those same vectors.

The research simulation therefore has two distinct stages

Stage A: calibrate the bands

For every n , e.g.

$$20, 50, 100, 200,$$

calculate

$$\{\ell_i^B, u_i^B\}_{i=1}^n$$

for each B .

For BJ and DW, I would use a **large Uniform Monte Carlo calibration**, perhaps 10^5 – 10^6 replicates, unless you implement an exact boundary-crossing calculation. There are now efficient exact algorithms for KS-family and one-sided Berk–Jones-type statistics, so exact calibration can be investigated later. [source citation: turn303319search12]

Stage B: compare induced parameter regions

Simulate

$$X_1, \dots, X_n \sim N(0, 1).$$

For each band compute

$$a_i^B = \Phi^{-1}(\ell_i^B), \quad b_i^B = \Phi^{-1}(u_i^B),$$

construct

$$C_B = \bigcap_i \{x_i - \sigma b_i^B \leq \mu \leq x_i - \sigma a_i^B\},$$

and record

$$\text{Area}(C_B),$$

$$\mu_U^B - \mu_L^B,$$

$$\sigma_U^B - \sigma_L^B.$$

One issue we should settle before coding
There is a subtle but important choice between comparing:

the original published bands

versus

bands recalibrated to exactly the same finite-sample 95% coverage.

For example, DKW is guaranteed to have **at least** 95% coverage and is generally conservative, whereas exact KS can be calibrated extremely close to exactly 95%.

If we simply compare their polygon areas, DKW is being penalized partly because it has higher coverage.

For a clean research paper, I would report **both**:

Method as published

and

coverage-matched version.

The second comparison is scientifically more revealing because it isolates the effect of the **shape of the confidence band** rather than its conservativeness.

That distinction may actually become an important part of the study.

Turn 12

User

write me a script in R to compute KS bounds

Assistant

Here is a simple R function that computes the **two-sided KS confidence-band bounds** (ℓ_i, u_i) for the order statistics.

```
ks_bounds <- function(n, alpha = 0.05, exact = TRUE) {
  # Critical value d such that
  # P(D_n <= d) = 1 - alpha

  if (exact) {
    d <- uniroot(
      function(x) {
        pkolmogorov2x(x, n) - (1 - alpha)
      },
      interval = c(0, 1)
    )$root
  } else {
    # Asymptotic KS approximation
    c_alpha <- sqrt(-0.5 * log(alpha / 2))
    d <- c_alpha / sqrt(n)
  }

  i <- 1:n

  lower <- pmax(0, i / n - d)
  upper <- pmin(1, i / n + d)

  data.frame(
    i = i,
    p = i / n,
    lower = lower,
    upper = upper
  )
}
```

However, base R does **not** expose `pkolmogorov2x()` as a normal public function in every version. A more robust version is to use `stats::ks.test`'s underlying finite-sample KS distribution through the `kolmogorov` package, or simply simulate the exact finite- n critical value.

For your research project, I would actually use this Monte Carlo calibration because it is transparent and easy to reproduce:

```
ks_bounds_mc <- function(
  n,
  alpha = 0.05,
  B = 100000,
  seed = 123
) {
  set.seed(seed)

  D <- numeric(B)

  for (b in seq_len(B)) {
    u <- sort(runif(n))
    i <- seq_len(n)

    # Exact two-sided one-sample KS statistic
    D_plus <- max(i / n - u)
    D_minus <- max(u - (i - 1) / n)

    D[b] <- max(D_plus, D_minus)
  }

  # Estimated (1-alpha) critical value
  d <- unname(quantile(D, probs = 1 - alpha, type = 8))

  i <- seq_len(n)

  lower <- pmax(0, i / n - d)
```

```

upper <- pmin(1, i / n + d)

list(
  n = n,
  alpha = alpha,
  critical_value = d,
  bounds = data.frame(
    i = i,
    p = i / n,
    lower = lower,
    upper = upper
  )
)
}

```

Example:

```

res <- ks_bounds_mc(
  n = 50,
  alpha = 0.05,
  B = 100000
)

res$critical_value
head(res$bounds)

```

The output `res$bounds` contains

$$\ell_i = \max\left(0, \frac{i}{n} - d\right)$$

and

$$u_i = \min\left(1, \frac{i}{n} + d\right).$$

For the normal location-scale model, your next transformation is simply:

```

bounds <- res$bounds

bounds$a <- qnorm(bounds$lower)
bounds$b <- qnorm(bounds$upper)

```

with the expected behavior

```

qnorm(0) # -Inf
qnorm(1) # Inf

```

which is fine mathematically because those constraints just become non-binding on one side.

For your eventual comparison study, I would separate the code into two functions: one that computes/calibrates the band (ℓ_i, u_i) , and another that takes arbitrary bounds and constructs the (μ, σ) confidence polygon. That way KS, DKW, Berk–Jones, and Dümbgen–Wellner all plug into exactly the same downstream code.

Turn 13

User

write me a script in R to compute DKW bounds

Assistant

Sure. For DKW, the band is explicit, so there is no simulation or root finding.

```
dkw_bounds <- function(n, alpha = 0.05) {  
  epsilon <- sqrt(log(2 / alpha) / (2 * n))  
  
  i <- seq_len(n)  
  p <- i / n  
  
  lower <- pmax(0, p - epsilon)  
  upper <- pmin(1, p + epsilon)  
  
  data.frame(  
    i = i,  
    p = p,  
    lower = lower,  
    upper = upper  
  )  
}
```

Example:

```
bounds <- dkw_bounds(  
  n = 50,  
  alpha = 0.05  
)  
  
head(bounds)  
tail(bounds)
```

The function uses

$$\epsilon_{n,\alpha} = \sqrt{\frac{\log(2/\alpha)}{2n}},$$

and returns

$$\ell_i = \max\left(0, \frac{i}{n} - \epsilon_{n,\alpha}\right),$$
$$u_i = \min\left(1, \frac{i}{n} + \epsilon_{n,\alpha}\right).$$

For your normal location-scale problem, you can immediately transform these to the coefficients used in the confidence polygon:

```
bounds$a <- qnorm(bounds$lower)  
bounds$b <- qnorm(bounds$upper)
```

Then the constraints are

```
x_(i) - sigma * b_i <= mu <= x_(i) - sigma * a_i
```

for $i = 1, \dots, n$.

One thing worth keeping in mind for your comparison: DKW is a **guaranteed lower-bound coverage method**, so its actual finite-sample coverage will generally exceed $1 - \alpha$. That makes it naturally more conservative than an exactly calibrated KS band.

Turn 14

User

write me a script in R to compute Berk–Jones / Owen bounds

Assistant

Yes. For Berk–Jones/Owen, I'd structure the R code in two steps:

1. calibrate the finite-sample critical value $c_{n,\alpha}$ under $U(0,1)$; 2. invert the Bernoulli KL divergence to get (ℓ_i, u_i) .

Owen's 1995 paper constructs nonparametric likelihood bands by inverting the Berk–Jones statistic, with exact finite-sample calibration available through Noé's recursion. [source citation: turn235873search0 turn235873search25] For a first implementation, Monte Carlo calibration is much simpler and fully distribution-free.

```
# Bernoulli Kullback-Leibler divergence
kl_bern <- function(p, q) {
  # Vectorized, with proper endpoint conventions

  out <- numeric(length(p + q))

  # General interior case
  interior <- (p > 0 & p < 1 & q > 0 & q < 1)
  out[interior] <-
    p[interior] * log(p[interior] / q[interior]) +
    (1 - p[interior]) *
      log((1 - p[interior]) / (1 - q[interior]))

  # p = 0
  idx <- (p == 0 & q < 1)
  out[idx] <- -log(1 - q[idx])

  # p = 1
  idx <- (p == 1 & q > 0)
  out[idx] <- -log(q[idx])

  # Equal endpoints
  out[p == 0 & q == 0] <- 0
  out[p == 1 & q == 1] <- 0

  # Impossible endpoint combinations
  out[p > 0 & q == 0] <- Inf
  out[p < 1 & q == 1] <- Inf

  out
}
```

Now compute the two-sided Berk–Jones statistic for one uniform sample. For sorted uniforms $U_{(i)}$, the supremum can be evaluated using the empirical-CDF values immediately before and at each jump.

```
bj_statistic <- function(u) {
  u <- sort(u)
  n <- length(u)
  i <- seq_len(n)

  # Fn just before and at U_(i)
  p_left <- (i - 1) / n
  p_right <- i / n

  bj_left <- n * kl_bern(p_left, u)
  bj_right <- n * kl_bern(p_right, u)

  max(c(bj_left, bj_right))
}
```

Then calibrate the critical value by simulation:

```
bj_critical_mc <- function(
  n,
  alpha = 0.05,
  B = 100000,
  seed = 123
) {
  set.seed(seed)
```

```

stats <- numeric(B)

for (b in seq_len(B)) {
  u <- runif(n)
  stats[b] <- bj_statistic(u)
}

unnname(
  quantile(
    stats,
    probs = 1 - alpha,
    type = 8
  )
)
}

```

Now we invert

$$nK(p, q) = c_{n, \alpha}$$

for every empirical probability p .

```

bj_interval_at_p <- function(
  p,
  critical,
  n,
  tol = 1e-10
) {

  target <- critical / n

  # Lower endpoint
  if (p == 0) {
    lower <- 0
  } else {
    f_lower <- function(q) {
      kl_bern(p, q) - target
    }

    lower <- uniroot(
      f_lower,
      interval = c(.Machine$double.eps, p),
      tol = tol
    )$root
  }

  # Upper endpoint
  if (p == 1) {
    upper <- 1
  } else {
    f_upper <- function(q) {
      kl_bern(p, q) - target
    }

    upper <- uniroot(
      f_upper,
      interval = c(p, 1 - .Machine$double.eps),
      tol = tol
    )$root
  }

  c(lower = lower, upper = upper)
}

```

Finally, build the full Berk–Jones/Owen band:

```

bj_bounds <- function(
  n,
  alpha = 0.05,
  B = 100000,
  seed = 123,
  critical = NULL
) {

  # Calibrate only if the user did not supply a critical value

```

```

if (is.null(critical)) {
  critical <- bj_critical_mc(
    n = n,
    alpha = alpha,
    B = B,
    seed = seed
  )
}

i <- seq_len(n)
p <- i / n

lower <- numeric(n)
upper <- numeric(n)

for (j in seq_len(n)) {
  bounds_j <- bj_interval_at_p(
    p = p[j],
    critical = critical,
    n = n
  )

  lower[j] <- bounds_j["lower"]
  upper[j] <- bounds_j["upper"]
}

list(
  n = n,
  alpha = alpha,
  critical_value = critical,
  bounds = data.frame(
    i = i,
    p = p,
    lower = lower,
    upper = upper
  )
)
}

```

Example:

```

res <- bj_bounds(
  n = 50,
  alpha = 0.05,
  B = 100000,
  seed = 123
)

res$critical_value
head(res$bounds)
tail(res$bounds)

```

And for your normal location–scale confidence polygon:

```

bounds <- res$bounds

bounds$a <- qnorm(bounds$lower)
bounds$b <- qnorm(bounds$upper)

```

Then use the constraints

$$x_{(i)} - \sigma b_i \leq \mu \leq x_{(i)} - \sigma a_i.$$

One technical point: the code above uses a **Monte Carlo critical value**. For the actual research paper, I would eventually replace that calibration with either Owen/Noé finite-sample recursion or published exact critical values, because Owen’s method was specifically designed for exact finite- n coverage. [source citation: turn235873search0 turn235873search3]

Also, Berk–Jones/Owen bands are generally **not** symmetric around i/n ; that asymmetric, tail-adaptive shape is one of their defining features. [source citation: turn235873search0]

Turn 15

User

write me a script in R to compute Dümbgen–Wellner bounds

Assistant

Yes. For the Dümbgen–Wellner band, I'd implement the $s = 1, \nu = 1$ version directly from Dümbgen and Wellner (2023). For $s = 1$, K_s is the Bernoulli KL divergence, and their finite-sample statistic can be evaluated at only $2n$ points rather than optimizing over the whole interval. [source citation: turn383145view0]

```
# =====
# DUMBGEN-WELLNER CONFIDENCE BAND
# s = 1, nu = 1
#
# Returns bounds:
# lower_i <= F(X_i)) <= upper_i
#
# Critical value is calibrated by Monte Carlo under U(0,1).
# =====

# -----
# 1. Bernoulli KL divergence
#
# K(p, q) =
# p log(p/q) + (1-p) log((1-p)/(1-q))
# -----

kl_bern <- function(p, q) {

  # Recycling-safe vectorization
  len <- max(length(p), length(q))
  p <- rep(p, length.out = len)
  q <- rep(q, length.out = len)

  out <- numeric(len)

  # Interior case
  idx <- p > 0 & p < 1 & q > 0 & q < 1

  out[idx] <-
    p[idx] * log(p[idx] / q[idx]) +
    (1 - p[idx]) *
      log((1 - p[idx]) / (1 - q[idx]))

  # p = 0
  idx <- p == 0 & q < 1
  out[idx] <- -log(1 - q[idx])

  # p = 1
  idx <- p == 1 & q > 0
  out[idx] <- -log(q[idx])

  # Equal endpoints
  out[p == 0 & q == 0] <- 0
  out[p == 1 & q == 1] <- 0

  # Infinite cases
  out[p > 0 & q == 0] <- Inf
  out[p < 1 & q == 1] <- Inf

  out
}

# -----
# 2. D-W correction C_nu(t)
#
# C(t) = log log( e / [4 t(1-t)] )
# D(t) = log(1 + C(t)^2)
# C_nu(t) = C(t) + nu * D(t)
#
# C_nu(1/2) = 0
```

```

# C_nu(0) = C_nu(1) = Inf
# -----

C_nu <- function(t, nu = 1) {

  out <- rep(Inf, length(t))

  idx <- t > 0 & t < 1

  if (any(idx)) {
    C <- log(
      log(
        exp(1) / (4 * t[idx] * (1 - t[idx]))
      )
    )

    D <- log(1 + C^2)

    out[idx] <- C + nu * D
  }

  out
}

# -----
# 3. Bivariate correction C_nu(p, q)
#
# Defined as
#
# min_{v between p and q} C_nu(v).
#
# Because C_nu is symmetric around 1/2 and decreases
# towards 1/2, this has a closed form.
# -----

C_nu_biv <- function(p, q, nu = 1) {

  len <- max(length(p), length(q))
  p <- rep(p, length.out = len)
  q <- rep(q, length.out = len)

  lo <- pmin(p, q)
  hi <- pmax(p, q)

  out <- numeric(len)

  # Entire interval lies to the left of 1/2
  left <- hi < 0.5
  out[left] <- C_nu(hi[left], nu)

  # Entire interval lies to the right of 1/2
  right <- lo > 0.5
  out[right] <- C_nu(lo[right], nu)

  # Otherwise the interval contains 1/2, where C_nu = 0
  middle <- !(left | right)
  out[middle] <- 0

  out
}

# -----
# 4. Pointwise D-W discrepancy
#
#  $H_n(p, q) = n K(p, q) - C_{nu}(p, q)$ 
# -----

dw_discrepancy <- function(p, q, n, nu = 1) {
  n * kl_bern(p, q) - C_nu_biv(p, q, nu)
}

# -----
# 5. D-W statistic for one U(0,1) sample

```

```

#
# For  $s > 0$  the paper shows that the supremum equals
#
# max_i {
#   n K((i-1)/n, U_(i)) - C_nu(...),
#   n K(i/n, U_(i)) - C_nu(...)}
# -----

dw_statistic <- function(u, nu = 1) {

  u <- sort(u)
  n <- length(u)
  i <- seq_len(n)

  p_left <- (i - 1) / n
  p_right <- i / n

  stat_left <- dw_discrepancy(
    p = p_left,
    q = u,
    n = n,
    nu = nu
  )

  stat_right <- dw_discrepancy(
    p = p_right,
    q = u,
    n = n,
    nu = nu
  )

  max(c(stat_left, stat_right))
}

# -----
# 6. Monte Carlo critical value
#
# kappa = (1-alpha)-quantile of the D-W statistic under
# U(0,1).
# -----

dw_critical_mc <- function(
  n,
  alpha = 0.05,
  B = 100000,
  nu = 1,
  seed = 123
) {

  set.seed(seed)

  stats <- numeric(B)

  for (b in seq_len(B)) {
    stats[b] <- dw_statistic(
      runif(n),
      nu = nu
    )
  }

  unname(
    quantile(
      stats,
      probs = 1 - alpha,
      type = 8
    )
  )
}

# -----
# 7. Upper D-W endpoint at empirical probability p
#
# Find largest q >= p such that

```

```

#
# n K(p,q) - C_nu(p,q) <= critical.
#
# The discrepancy is monotone increasing for q >= p,
# so a one-dimensional root solve is sufficient.
# -----

dw_upper_at_p <- function(
  p,
  n,
  critical,
  nu = 1,
  tol = 1e-10
) {

  if (p >= 1) {
    return(1)
  }

  f <- function(q) {
    dw_discrepancy(
      p = p,
      q = q,
      n = n,
      nu = nu
    ) - critical
  }

  # At q = p:
  # K(p,p)=0 and correction >= 0,
  # hence f(p) <= 0.

  upper_limit <- 1 - .Machine$double.eps

  # If even the endpoint satisfies the inequality,
  # the upper confidence bound is 1.
  if (f(upper_limit) <= 0) {
    return(1)
  }

  uniroot(
    f,
    interval = c(p, upper_limit),
    tol = tol
  )$root
}

# -----
# 8. Complete D-W confidence band
#
# We exploit symmetry:
#
# a_(n-i) = 1 - b_i.
#
# The function computes the full grid i = 0,...,n,
# then returns the bounds at X_(1),...,X_(n).
# -----

dw_bounds <- function(
  n,
  alpha = 0.05,
  B = 100000,
  nu = 1,
  seed = 123,
  critical = NULL,
  tol = 1e-10
) {

  # Calibrate critical value if not supplied
  if (is.null(critical)) {
    critical <- dw_critical_mc(
      n = n,
      alpha = alpha,
      B = B,
      nu = nu,

```

```

    seed = seed
  )
}

# Empirical CDF levels 0/n,...,n/n
i_full <- 0:n
p_full <- i_full / n

upper_full <- numeric(n + 1)
lower_full <- numeric(n + 1)

# By definition upper bound at p = 1 is 1
upper_full[n + 1] <- 1

# Compute b_i for i = 0,...,n-1
for (j in 1:n) {
  upper_full[j] <- dw_upper_at_p(
    p = p_full[j],
    n = n,
    critical = critical,
    nu = nu,
    tol = tol
  )
}

# Symmetry:
# a_(n-i) = 1 - b_i
for (j in 1:(n + 1)) {
  lower_full[n + 2 - j] <- 1 - upper_full[j]
}

# At order statistic X_(i), Fn(X_(i)) = i/n,
# so use indices i = 1,...,n.
bounds <- data.frame(
  i = seq_len(n),
  p = seq_len(n) / n,
  lower = lower_full[2:(n + 1)],
  upper = upper_full[2:(n + 1)]
)

full_bounds <- data.frame(
  i = 0:n,
  p = p_full,
  lower = lower_full,
  upper = upper_full
)

list(
  n = n,
  alpha = alpha,
  nu = nu,
  critical_value = critical,
  bounds = bounds,
  full_bounds = full_bounds
)
}

```

Then use it exactly like the other functions:

```

res <- dw_bounds(
  n = 50,
  alpha = 0.05,
  B = 100000,
  nu = 1,
  seed = 123
)

res$critical_value
head(res$bounds)
tail(res$bounds)

```

The theoretical objects being implemented are

$$C(t) = \log \log \left(\frac{e}{4t(1-t)} \right),$$

$$D(t) = \log\{1 + C(t)^2\}, \quad C_\nu(t) = C(t) + \nu D(t),$$

and

$$C_\nu(p, q) = \min_{\min(p, q) \leq v \leq \max(p, q)} C_\nu(v).$$

Dümbgen and Wellner define their statistic as

$$T_{n,s,\nu} = \sup_t \{nK_s(G_n(t), t) - C_\nu(G_n(t), t)\},$$

and show that for $s > 0$, including our $s = 1$, it can be evaluated by a maximum of at most $2n$ terms. [source citation: turn383145view0] They specifically note that $\nu > 3/4$ is required and provide finite-sample critical values for $\nu = 1$; their numerical discussion also identifies $s = 1$ as a sensible compromise. [source citation: turn383145view0]

For your normal confidence polygon, the downstream step remains exactly the same:

```
bounds <- res$bounds

bounds$a <- qnorm(bounds$lower)
bounds$b <- qnorm(bounds$upper)
```

giving

$$x_{(i)} - \sigma b_i \leq \mu \leq x_{(i)} - \sigma a_i.$$

One useful check is that the DW band should be symmetric:

```
all.equal(
  res$full_bounds$lower,
  1 - rev(res$full_bounds$upper),
  tolerance = 1e-8
)
```

which should return TRUE.

For the eventual simulation study, I would **calibrate the DW critical value once for each (n, α) ** and reuse it. The paper also supplies exact finite-sample critical-value tables for many n , s , and α combinations, so later we can replace the Monte Carlo calibration with those values where available. [source citation: turn383145view0]

Turn 16

User

Now, given the bounds, we need to compute the confidence set for (μ, σ) , compute its area and marginal confidence intervals for μ and σ . We already have the theory. Write me an R script to do that

Assistant

Exactly. The following is a band-agnostic R implementation. You give it:

- the observed sample $\mathbf{x}$,
- the band's lower values ℓ_i ,
- the band's upper values u_i ,
- the base-family quantile function G^{-1} , e.g. `qnorm`.

It computes:

$$C = \{(\mu, \sigma) : x_{(i)} - \sigma b_i \leq \mu \leq x_{(i)} - \sigma a_i\},$$

where

$$a_i = G^{-1}(\ell_i), \quad b_i = G^{-1}(u_i),$$

and returns the exact marginal intervals, exact polygon area, and boundary points.

```
# =====
# CONFIDENCE REGION FOR A LOCATION-SCALE FAMILY
#
# Given simultaneous CDF/order-statistic bounds
#
# lower_i <- F(X_(i)) <= upper_i
#
# for
#
# F_{mu,sigma}(x) = G((x - mu) / sigma),
#
# construct the exact confidence region
#
# x_(i) - sigma * b_i <= mu <= x_(i) - sigma * a_i
#
# where
#
# a_i = G^{-1}(lower_i)
# b_i = G^{-1}(upper_i).
#
# Returns:
# - confidence interval for sigma
# - confidence interval for mu
# - exact area of joint confidence region
# - piecewise-linear lower/upper boundaries
# - polygon coordinates for plotting
#
# No external packages required.
# =====

# -----
# 1. Upper envelope of affine functions
#
# Given lines
#
# f_j(s) = intercept_j + slope_j * s
#
# return the upper envelope
#
# max_j f_j(s).
#
# Lines are internally sorted by increasing slope.
#
# Complexity: O(n log n) because of sorting.
```

```

# If slopes are already ordered, the hull construction itself
# is O(n).
# -----

upper_envelope <- function(intercept, slope, tol = 1e-12) {

  keep <- is.finite(intercept) & is.finite(slope)

  intercept <- intercept[keep]
  slope <- slope[keep]

  if (length(intercept) == 0) {
    stop("No finite lines available for envelope.")
  }

  # Sort by increasing slope
  ord <- order(slope, intercept)
  slope <- slope[ord]
  intercept <- intercept[ord]

  # -----
  # Remove duplicate slopes.
  #
  # For an upper envelope, among parallel lines we keep only
  # the one with the largest intercept.
  # -----

  m2 <- numeric(0)
  c2 <- numeric(0)

  for (j in seq_along(slope)) {

    if (length(m2) == 0 ||
        abs(slope[j] - m2[length(m2)]) > tol) {

      m2 <- c(m2, slope[j])
      c2 <- c(c2, intercept[j])

    } else {

      # Same slope: retain larger intercept
      if (intercept[j] > c2[length(c2)]) {
        c2[length(c2)] <- intercept[j]
      }
    }
  }

  slope <- m2
  intercept <- c2

  # -----
  # Monotone convex-hull construction
  # -----

  hull_c <- numeric(0)
  hull_m <- numeric(0)
  start <- numeric(0)

  for (j in seq_along(slope)) {

    cj <- intercept[j]
    mj <- slope[j]

    repeat {

      if (length(hull_c) == 0) {
        break
      }

      k <- length(hull_c)

      # Intersection of previous line and new line:
      #
      #  $c_{old} + m_{old}s = c_{new} + m_{new}s$ 

```

```

#
# s = (c_old - c_new)/(m_new - m_old)

s_cross <- (hull_c[k] - cj) /
  (mj - hull_m[k])

# If new line overtakes old line before the old line
# even becomes active, old line can never appear on
# the upper envelope.
if (k > 1 && s_cross <= start[k] + tol) {

  hull_c <- hull_c[-k]
  hull_m <- hull_m[-k]
  start <- start[-k]

} else {
  break
}
}

if (length(hull_c) == 0) {

  hull_c <- cj
  hull_m <- mj
  start <- -Inf

} else {

  k <- length(hull_c)

  s_cross <- (hull_c[k] - cj) /
    (mj - hull_m[k])

  hull_c <- c(hull_c, cj)
  hull_m <- c(hull_m, mj)
  start <- c(start, s_cross)
}
}

list(
  intercept = hull_c,
  slope = hull_m,
  start = start
)
}

# -----
# 2. Evaluate an envelope
# -----

eval_envelope <- function(env, s) {

  vapply(
    s,
    function(z) {

      # Find last line whose start point is <= z
      k <- max(which(env$start <= z))

      env$intercept[k] +
        env$slope[k] * z
    },
    numeric(1)
  )
}

# -----
# 3. Exact projection onto sigma
#
# Feasibility requires
#
#  $x_i - \sigma \cdot b_i \leq x_j - \sigma \cdot a_j$ 
#
# for every i,j.

```

```

#
# Hence
#
# sigma * (a_j - b_i) <= x_j - x_i.
#
# This yields exact pairwise lower and upper bounds.
# -----

sigma_interval_exact <- function(x, a, b, tol = 1e-12) {

  n <- length(x)

  # Only finite a_j and b_i impose pairwise constraints.
  ii <- which(is.finite(b))
  jj <- which(is.finite(a))

  if (length(ii) == 0 || length(jj) == 0) {
    return(
      list(
        lower = 0,
        upper = Inf,
        feasible = TRUE
      )
    )
  }

  # Rows correspond to i, columns to j.
  D <- outer(
    b[ii],
    a[jj],
    function(bi, aj) aj - bi
  )

  Delta <- outer(
    x[ii],
    x[jj],
    function(xi, xj) xj - xi
  )

  # Check constraints where denominator is zero:
  #
  # 0 <= x_j - x_i
  zero <- abs(D) <= tol

  if (any(zero & Delta < -tol)) {
    return(
      list(
        lower = NA_real_,
        upper = NA_real_,
        feasible = FALSE
      )
    )
  }

  # Lower bounds on sigma arise when D < 0
  lower_candidates <- Delta[D < -tol] / D[D < -tol]

  sigma_lower <- 0

  if (length(lower_candidates) > 0) {
    sigma_lower <- max(
      0,
      lower_candidates,
      na.rm = TRUE
    )
  }

  # Upper bounds arise when D > 0
  upper_candidates <- Delta[D > tol] / D[D > tol]

  sigma_upper <- Inf

  if (length(upper_candidates) > 0) {
    sigma_upper <- min(

```

```

    upper_candidates,
    na.rm = TRUE
  )
}

feasible <-
  sigma_upper > 0 &&
  sigma_lower <= sigma_upper + tol

list(
  lower = sigma_lower,
  upper = sigma_upper,
  feasible = feasible
)
}

# -----
# 4. Main confidence-region function
# -----

location_scale_confidence <- function(
  x,
  lower,
  upper,
  qfun = qnorm,
  tol = 1e-10
) {

  x <- sort(x)
  n <- length(x)

  if (length(lower) != n || length(upper) != n) {
    stop(
      "length(lower) and length(upper) must equal length(x)."
    )
  }

  if (any(lower < 0 | lower > 1) ||
      any(upper < 0 | upper > 1)) {
    stop("Bounds must lie in [0, 1].")
  }

  if (any(lower > upper)) {
    stop("Each lower bound must be <= its upper bound.")
  }

  # -----
  # Transform probability bounds through G^{-1}
  # -----

  a <- qfun(lower)
  b <- qfun(upper)

  # -----
  # Exact sigma interval
  # -----

  sigma_res <- sigma_interval_exact(
    x = x,
    a = a,
    b = b,
    tol = tol
  )

  if (!sigma_res$feasible) {

    return(
      list(
        feasible = FALSE,
        message = "The confidence region is empty."
      )
    )
  }
}

```

```

}

sigma_L <- sigma_res$lower
sigma_U <- sigma_res$upper

# -----
# Construct
#
# L(sigma) = max_i {x_i - b_i sigma}
#
# Infinite b_i correspond to nonbinding lower constraints
# and are therefore omitted.
# -----

keep_L <- is.finite(b)

env_L <- upper_envelope(
  intercept = x[keep_L],
  slope = -b[keep_L],
  tol = tol
)

# -----
# U(sigma) = min_i {x_i - a_i sigma}
#
# Construct instead
#
# V(sigma) = -U(sigma)
# = max_i {-x_i + a_i sigma}.
#
# Then U = -V.
# -----

keep_U <- is.finite(a)

env_V <- upper_envelope(
  intercept = -x[keep_U],
  slope = a[keep_U],
  tol = tol
)

Lfun <- function(s) {
  eval_envelope(env_L, s)
}

Ufun <- function(s) {
  -eval_envelope(env_V, s)
}

# -----
# If sigma region is unbounded, joint area is infinite.
# The common bands considered in this project normally give
# bounded regions for nondegenerate samples.
# -----

if (!is.finite(sigma_U)) {
  return(
    list(
      feasible = TRUE,
      bounded = FALSE,
      sigma_interval = c(
        lower = sigma_L,
        upper = Inf
      ),
      area = Inf,
      a = a,
      b = b,
      lower_envelope = env_L,
      upper_envelope_negated = env_V
    )
  )
}

```

```

}

# -----
# Gather every breakpoint of either envelope inside the
# feasible sigma interval.
#
# Between consecutive breakpoints, both L and U are affine.
# -----

breaks_L <- env_L$start[
  is.finite(env_L$start) &
  env_L$start > sigma_L + tol &
  env_L$start < sigma_U - tol
]

breaks_U <- env_V$start[
  is.finite(env_V$start) &
  env_V$start > sigma_L + tol &
  env_V$start < sigma_U - tol
]

knots <- sort(
  unique(
    c(
      sigma_L,
      breaks_L,
      breaks_U,
      sigma_U
    )
  )
)

# -----
# Evaluate boundaries exactly at every knot
# -----

L_values <- Lfun(knots)
U_values <- Ufun(knots)

gap <- U_values - L_values

# Numerical sanity check
if (any(gap < -sqrt(tol))) {
  warning(
    "Numerical feasibility issue detected at envelope knots."
  )
}

# -----
# Exact marginal interval for mu
#
# L is convex piecewise linear, so its minimum occurs at a
# breakpoint or endpoint.
#
# U is concave piecewise linear, so its maximum occurs at a
# breakpoint or endpoint.
# -----

mu_L <- min(L_values)
mu_U <- max(U_values)

# -----
# Exact area
#
# U(sigma)-L(sigma) is affine between consecutive knots.
# Therefore trapezoidal integration is exact.
# -----

if (length(knots) == 1) {
  area <- 0

```

```

} else {

  widths <- diff(knots)

  gap_left <- gap[-length(gap)]
  gap_right <- gap[-1]

  area <- sum(
    0.5 *
      (gap_left + gap_right) *
      widths
  )
}

# -----
# Piecewise boundary representation
# -----

boundary <- data.frame(
  sigma = knots,
  lower_mu = L_values,
  upper_mu = U_values,
  width = gap
)

# -----
# Polygon coordinates
#
# Lower boundary left-to-right, then upper boundary
# right-to-left.
# -----

polygon <- rbind(
  data.frame(
    sigma = knots,
    mu = L_values,
    boundary = "lower"
  ),
  data.frame(
    sigma = rev(knots),
    mu = rev(U_values),
    boundary = "upper"
  )
)

list(
  feasible = TRUE,
  bounded = TRUE,

  n = n,

  sigma_interval = c(
    lower = sigma_L,
    upper = sigma_U
  ),

  sigma_length = sigma_U - sigma_L,

  mu_interval = c(
    lower = mu_L,
    upper = mu_U
  ),

  mu_length = mu_U - mu_L,

  area = area,

  a = a,
  b = b,

  boundary = boundary,
  polygon = polygon,

```

```

    lower_envelope = env_L,
    upper_envelope_negated = env_V
  )
}

```

Example with any of your bands
 Suppose you have a normal sample:

```

set.seed(1)

x <- rnorm(
  n = 50,
  mean = 2,
  sd = 3
)

```

and DKW bounds:

```

bounds <- dkw_bounds(
  n = length(x),
  alpha = 0.05
)

```

Then:

```

fit <- location_scale_confidence(
  x = x,
  lower = bounds$lower,
  upper = bounds$upper,
  qfun = qnorm
)

```

The joint region summary is then:

```

fit$sigma_interval
fit$sigma_length

fit$mu_interval
fit$mu_length

fit$area

```

So, for your simulation study, the three primary quantities are directly available as

```

fit$area
fit$mu_length
fit$sigma_length

```

Plot the confidence set

Because the routine also returns the exact polygon coordinates:

```

plot(
  fit$polygon$sigma,
  fit$polygon$mu,
  type = "n",
  xlab = expression(sigma),
  ylab = expression(mu),
  main = "Joint confidence region"
)

polygon(
  fit$polygon$sigma,
  fit$polygon$mu
)

```

Or plot the two envelope functions:

```

plot(
  fit$boundary$sigma,
  fit$boundary$lower_mu,
  type = "l",
  ylim = range(
    fit$boundary$lower_mu,
    fit$boundary$upper_mu
  ),
  xlab = expression(sigma),

```

```

  ylab = expression(mu)
)

lines(
  fit$boundary$sigma,
  fit$boundary$upper_mu
)

```

It works unchanged for all four bands
For KS:

```

B <- ks_bounds_mc(
  n = length(x),
  alpha = 0.05
)$bounds

fit_KS <- location_scale_confidence(
  x,
  B$lower,
  B$upper,
  qnorm
)

```

DKW:

```

B <- dkw_bounds(
  n = length(x),
  alpha = 0.05
)

fit_DKW <- location_scale_confidence(
  x,
  B$lower,
  B$upper,
  qnorm
)

```

Berk-Jones/Owen:

```

B <- bj_bounds(
  n = length(x),
  alpha = 0.05
)$bounds

fit_BJ <- location_scale_confidence(
  x,
  B$lower,
  B$upper,
  qnorm
)

```

Dümbgen-Wellner:

```

B <- dw_bounds(
  n = length(x),
  alpha = 0.05
)$bounds

fit_DW <- location_scale_confidence(
  x,
  B$lower,
  B$upper,
  qnorm
)

```

You can immediately put their results together:

```

results <- data.frame(
  method = c(
    "KS",
    "DKW",
    "Berk-Jones",
    "Dümbgen-Wellner"
  ),
  area = c(
    fit_KS$area,

```

```

    fit_DKW$area,
    fit_BJ$area,
    fit_DW$area
  ),

  mu_length = c(
    fit_KS$mu_length,
    fit_DKW$mu_length,
    fit_BJ$mu_length,
    fit_DW$mu_length
  ),

  sigma_length = c(
    fit_KS$sigma_length,
    fit_DKW$sigma_length,
    fit_BJ$sigma_length,
    fit_DW$sigma_length
  )
)

results

```

This is essentially the complete computational engine needed for the first experiment. The next script can now be quite short: simulate $X_1, \dots, X_n \sim N(0, 1)$ repeatedly, feed the *same precomputed band bounds* into this function, and estimate

$$E[\text{Area}(C_B)], \quad E[|\mathcal{I}_{\mu,B}|], \quad E[|\mathcal{I}_{\sigma,B}|]$$

for each method.

Turn 17

User

	method	area	mu_length	sigma_length
1	KS	4.033455	1.949662	3.614687
2	DKW	4.254508	2.006160	3.714767
3	Berk-Jones	2.590804	2.309372	2.073396
4	Dumbgen-Wellner	2.753200	2.150504	2.190072

Assistant

This is already quite interesting, because the rankings differ across the three criteria.

For your one sample:

	Method	Area	μ -length	σ -length		KS	4.033	1.950	3.615		DKW	4.255	2.006	3.715
	Berk-Jones	2.591	2.309	2.073		Dümbgen-Wellner	2.753	2.151	2.190					

So Berk-Jones gives a dramatically smaller joint region than KS:

$$\frac{2.591}{4.033} \approx 0.64,$$

roughly a **36% reduction in area**.

The reason is mostly scale. Its σ -interval is

$$2.073/3.615 \approx 0.57$$

of the KS interval length—about a **43% reduction**.

But Berk-Jones actually performs worse for μ :

$$2.309 > 1.950.$$

That is exactly the sort of phenomenon we hoped might appear. The band that is best for joint uncertainty or scale uncertainty is not necessarily best for location uncertainty.

Dümbgen-Wellner appears to sit between KS and Berk-Jones here. Relative to KS it gives a much smaller area and scale interval, while sacrificing less on the μ interval than Berk-Jones:

$$A_{DW}/A_{KS} \approx 0.68,$$

$$L_{\sigma,DW}/L_{\sigma,KS} \approx 0.61,$$

but

$$L_{\mu,DW}/L_{\mu,KS} \approx 1.10.$$

And DKW being slightly worse than KS makes sense: both are essentially constant-width bands, but DKW is conservative because its width comes from the inequality rather than exact finite-sample calibration.

The important point, though, is **not to draw conclusions from this one dataset**. This is now a proof of concept that the comparison criterion is nontrivial.

The next experiment should repeat this perhaps 10^4 times and estimate

$$E[A_B], \quad E[L_{\mu,B}], \quad E[L_{\sigma,B}]$$

for each method, together with Monte Carlo standard errors.

I would also calculate pairwise relative quantities sample-by-sample, for example

$$R_{A,B} = \frac{A_B}{A_{KS}},$$

because then you can ask stronger questions like

$$P(A_{BJ} < A_{KS})$$

rather than comparing only mean areas.

Your preliminary result is actually a very encouraging one: **the three criteria are already disagreeing strongly**, which is precisely what could motivate a paper about parameter-targeted evaluation of distribution-free confidence bands.

Turn 18

User

Write an R loop to compute expected area and marginal lengths

Assistant

Yes. The key efficiency trick is to **precompute the four bands once** for the chosen n, α , then reuse them across all Monte Carlo samples.

```
# =====
# MONTE CARLO COMPARISON OF CONFIDENCE REGIONS
#
# Assumes these functions already exist:
# ks_bounds_mc()
# dkw_bounds()
# bj_bounds()
# dw_bounds()
# location_scale_confidence()
#
# Example target:
# X_i ~ N(0,1)
# =====

compare_bands_mc <- function(
  n = 50,
  alpha = 0.05,
  M = 10000,
  band_calibration_B = 100000,
  seed = 123
) {
  set.seed(seed)

  # -----
  # 1. Precompute band bounds ONCE
  # -----

  KS <- ks_bounds_mc(
    n = n,
    alpha = alpha,
    B = band_calibration_B,
    seed = seed + 1
  )$bounds

  DKW <- dkw_bounds(
    n = n,
    alpha = alpha
  )

  BJ <- bj_bounds(
    n = n,
    alpha = alpha,
    B = band_calibration_B,
    seed = seed + 2
  )$bounds

  DW <- dw_bounds(
    n = n,
    alpha = alpha,
    B = band_calibration_B,
    nu = 1,
    seed = seed + 3
  )$bounds

  bands <- list(
    KS = KS,
    DKW = DKW,
    Berk_Jones = BJ,
    Dumbgen_Wellner = DW
  )
}
```

```

# -----
# 2. Storage
# -----

methods <- names(bands)

area <- matrix(
  NA_real_,
  nrow = M,
  ncol = length(methods),
  dimnames = list(NULL, methods)
)

mu_length <- area
sigma_length <- area

# Optional: joint-region coverage of true (mu, sigma) = (0,1)
coverage <- matrix(
  FALSE,
  nrow = M,
  ncol = length(methods),
  dimnames = list(NULL, methods)
)

# -----
# 3. Monte Carlo loop
# -----

for (m in seq_len(M)) {

  # Simulate from standard normal.
  # By location-scale equivariance this is enough.
  x <- rnorm(n)

  x_sorted <- sort(x)

  for (j in seq_along(methods)) {

    B <- bands[[j]]

    fit <- location_scale_confidence(
      x = x_sorted,
      lower = B$lower,
      upper = B$upper,
      qfun = qnorm
    )

    if (isTRUE(fit$feasible) && isTRUE(fit$bounded)) {

      area[m, j] <- fit$area
      mu_length[m, j] <- fit$mu_length
      sigma_length[m, j] <- fit$sigma_length

      # -----
      # Check whether true point (mu,sigma) = (0,1)
      # belongs to the confidence region.
      #
      # Equivalent to checking all original inequalities.
      # -----

      a <- fit$a
      b <- fit$b

      lower_mu_at_true_sigma <- max(
        x_sorted[is.finite(b)] -
        b[is.finite(b)]
      )

      upper_mu_at_true_sigma <- min(
        x_sorted[is.finite(a)] -
        a[is.finite(a)]
      )
    }
  }
}

```

```

        coverage[m, j] <-
          lower_mu_at_true_sigma <= 0 &&
          0 <= upper_mu_at_true_sigma

      }
    }

    # Optional progress indicator
    if (m %% 1000 == 0) {
      cat("Completed", m, "of", M, "\n")
    }
  }

# -----
# 4. Summaries
# -----

mean_area <- colMeans(area, na.rm = TRUE)
mean_mu <- colMeans(mu_length, na.rm = TRUE)
mean_sigma <- colMeans(sigma_length, na.rm = TRUE)
empirical_coverage <- colMeans(coverage)

# Monte Carlo standard errors
se_area <- apply(
  area,
  2,
  sd,
  na.rm = TRUE
) / sqrt(M)

se_mu <- apply(
  mu_length,
  2,
  sd,
  na.rm = TRUE
) / sqrt(M)

se_sigma <- apply(
  sigma_length,
  2,
  sd,
  na.rm = TRUE
) / sqrt(M)

summary <- data.frame(
  method = methods,

  expected_area = mean_area,
  se_area = se_area,

  expected_mu_length = mean_mu,
  se_mu_length = se_mu,

  expected_sigma_length = mean_sigma,
  se_sigma_length = se_sigma,

  empirical_coverage = empirical_coverage,

  row.names = NULL
)

# -----
# 5. Return everything
# -----

list(
  summary = summary,
  area = area,
  mu_length = mu_length,
  sigma_length = sigma_length,
  coverage = coverage,

```

```

    bands = bands
  )
}

```

Then run:

```

res <- compare_bands_mc(
  n = 50,
  alpha = 0.05,
  M = 10000,
  band_calibration_B = 100000,
  seed = 123
)

res$summary

```

You should get something like:

```

      method expected_area se_area expected_mu_length ...
1 KS ...
2 DKW ...
3 Berk_Jones ...
4 Dumbgen_Wellner ...

```

I would also compute relative efficiency against KS:

```

summary <- res$summary

summary$area_ratio_to_KS <-
  summary$expected_area /
  summary$expected_area[summary$method == "KS"]

summary$mu_ratio_to_KS <-
  summary$expected_mu_length /
  summary$expected_mu_length[summary$method == "KS"]

summary$sigma_ratio_to_KS <-
  summary$expected_sigma_length /
  summary$expected_sigma_length[summary$method == "KS"]

summary

```

And because the same simulated sample is used for every method, you can make a **paired comparison**, which is statistically preferable. For example:

```

mean(
  res$area[, "Berk_Jones"] <
  res$area[, "KS"],
  na.rm = TRUE
)

```

estimates

$$P(A_{BJ} < A_{KS}).$$

Likewise:

```

mean(
  res$mu_length[, "Berk_Jones"] <
  res$mu_length[, "KS"],
  na.rm = TRUE
)

mean(
  res$sigma_length[, "Berk_Jones"] <
  res$sigma_length[, "KS"],
  na.rm = TRUE
)

```

Those paired probabilities could be quite informative alongside the expected values.

Turn 19

User

method	expected_area	se_area	expected_mu_length	se_mu_length
--------	---------------	---------	--------------------	--------------

1 KS	0.7374383	0.001891707	0.8107678	0.001325325	2 DKW	0.7885102	0.002004679	0.8357672	0.001356617
3 Berk-Jones	0.5760016	0.001673348	0.9344398	0.001376015	4 Dumbgen-Wellner	0.5541274	0.001564326	0.8771024	0.001314699
expected-sigma-length	se-sigma-length	empirical-coverage	1	1.533337	0.002006311	0.9658	2	1.588365	0.002065702
0.9715	3	1.060554	0.001697725	0.9722	4	1.081160	0.001656509	0.9705	

Assistant

These results are quite informative, and they support the research direction.

The ranking is now stable enough to see a real pattern:

Method	Expected area	Expected μ -length	Expected σ -length	Coverage	KS
DKW	0.737	0.811	1.533	0.966	0.789
Berk-Jones	0.576	0.836	1.588	0.972	0.554
Dümbgen-Wellner	0.554	0.877	1.081	0.971	

The main story is:

DW minimizes expected joint area

while

KS minimizes expected marginal length for μ

and

BJ minimizes expected marginal length for σ .

That is exactly the kind of target-dependent ranking that makes the project interesting. Relative to KS, Dümbgen-Wellner reduces expected area by roughly

$$1 - \frac{0.554}{0.737} \approx 24.9\%.$$

Berk-Jones reduces it by about

$$1 - \frac{0.576}{0.737} \approx 21.9\%.$$

For scale, BJ is especially strong:

$$1 - \frac{1.061}{1.533} \approx 30.8\%$$

shorter than KS.

But that comes with a substantial cost in location precision:

$$\frac{0.934}{0.811} - 1 \approx 15.3\%$$

longer μ -intervals than KS.

DW looks like a compromise: about 25% smaller joint area and about 29.5% shorter scale intervals than KS, while its μ -interval is only about 8.2% longer.

The coverage numbers also matter. All four methods are around 96.6% – 97.2%, not 95%. So part of the comparison is being made at slightly conservative coverage. But importantly, the coverages are fairly close to each other, which means the large differences in area and scale length cannot simply be explained by one method having much higher coverage.

The next thing I would do is **coverage-match the methods**. Calibrate each band so that its actual finite-sample coverage is as close as possible to 0.95. Then rerun exactly this table.

That gives you a much cleaner scientific comparison:

same coverage $\implies$ compare geometry/efficiency only.

After that, repeat over

$$n \in \{20, 50, 100, 200, 500\}$$

and plot the three relative efficiency curves. If the pattern persists—KS best for μ , BJ best for σ , DW best for area—you have a very compelling empirical result.

Turn 20

User

How do I make coverage close to 95 percents?

Assistant

Yes—but first fix the indexing. Your current ~97% coverages are probably inflated because the earlier scripts used the same empirical level i/n for both sides.

For a simultaneous band around the ECDF,

$$|F_n(x) - F(x)| \leq d,$$

at $X_{(i)}$ the two relevant ECDF values are

$$F_n(X_{(i)}) = \frac{i}{n}, \quad F_n(X_{(i)}^-) = \frac{i-1}{n}.$$

Since F is continuous,

$$F(X_{(i)}^-) = F(X_{(i)}).$$

Therefore the correct order-statistic bounds are

$$\ell_i = \max\left(0, \frac{i}{n} - d\right)$$

and

$$u_i = \min\left(1, \frac{i-1}{n} + d\right).$$

This matters for both KS and DKW. Dümbgen–Wellner explicitly formulate their bands as simultaneous bands for the full CDF, not merely intervals evaluated at i/n ; KS has exact nominal coverage for continuous F when the finite-sample critical value is used. [source citation: turn460032search24]

So your corrected DKW function should be:

```
dkw_bounds <- function(n, alpha = 0.05) {  
  eps <- sqrt(log(2 / alpha) / (2 * n))  
  i <- seq_len(n)  
  lower <- pmax(0, i / n - eps)  
  upper <- pmin(1, (i - 1) / n + eps)  
  data.frame(  
    i = i,  
    lower = lower,  
    upper = upper  
  )  
}
```

And corrected KS:

```
ks_bounds_from_critical <- function(n, d) {  
  i <- seq_len(n)  
  data.frame(  
    i = i,  
    lower = pmax(0, i / n - d),  
    upper = pmin(1, (i - 1) / n + d)  
  )  
}
```

For Berk–Jones and Dümbgen–Wellner, the same principle applies, but because the divergence is nonlinear you use ****different p 's on the two sides****:

$$\ell_i : \quad p = \frac{i}{n}$$

and

$$u_i : \quad p = \frac{i-1}{n}.$$

For Berk–Jones, if c is the critical value, solve

$$nK\left(\frac{i}{n}, \ell_i\right) = c, \quad \ell_i \leq \frac{i}{n},$$

and

$$nK\left(\frac{i-1}{n}, u_i\right) = c, \quad u_i \geq \frac{i-1}{n}.$$

Owen's confidence bands are indeed obtained by inverting the Berk–Jones likelihood statistic and can be calibrated to extremely close to nominal finite-sample coverage. [source citation: turn460032search0]

Likewise DW should solve

$$H_{n,\nu}\left(\frac{i}{n}, \ell_i\right) = c$$

for the lower bound and

$$H_{n,\nu}\left(\frac{i-1}{n}, u_i\right) = c$$

for the upper bound.

So I would **fix those four band generators before trying to force their coverage to 95%**.

After that, there are two different notions of "make coverage 95%" that you should distinguish.

For KS, use the finite-sample critical value d satisfying

$$P(D_n \leq d) = 0.95.$$

Then for continuous F , the underlying CDF band has essentially exact 95% coverage. [source citation: turn460032search24]

For BJ and DW, similarly calibrate their statistic critical value c under uniforms so that

$$P(T_n \leq c) = 0.95.$$

If using Monte Carlo calibration, don't use an interpolated quantile blindly. A simple version is:

```
critical_mc <- function(stats, alpha = 0.05) {
  B <- length(stats)
  # Empirical order statistic corresponding to >= 1-alpha coverage
  k <- ceiling((1 - alpha) * (B + 1))
  k <- min(k, B)
  sort(stats)[k]
}
```

With a very large calibration simulation, e.g.

```
B = 1000000
```

the calibration error becomes tiny.

But DKW is different. The standard DKW choice

$$\epsilon = \sqrt{\frac{\log(2/\alpha)}{2n}}$$

comes from the inequality

$$P(D_n > \epsilon) \leq \alpha,$$

so it is deliberately conservative. You cannot expect its coverage to equal exactly 95%. If you replace ϵ by the exact 95% quantile of D_n , you have effectively turned it into the KS band.

So for the research comparison I'd actually retain **two analyses**:

published/native calibration

and

coverage-matched calibration.

For coverage matching, choose a shape for each method and numerically tune its threshold so that

$$P\{\ell_i(c) \leq U_{(i)} \leq u_i(c) \forall i\} = 0.95.$$

You can do that purely using uniforms:

```
band_coverage <- function(lower, upper, n, B = 100000, seed = 123) {
  set.seed(seed)
  covered <- logical(B)
  for (b in seq_len(B)) {
    u <- sort(runif(n))
    covered[b] <- all(
      u >= lower &
      u <= upper
    )
  }
  mean(covered)
}
```

But if your critical value itself came from the **same statistic whose inversion defines the band**, then you don't need this extra calibration: calibrating the statistic's 95th percentile already gives the desired CDF-band coverage.

The most important immediate action is therefore:

correct the i/n versus $(i - 1)/n$ indexing and rerun your simulation.

I would expect your empirical KS/BJ/DW coverage to move much closer to 0.95. DKW should remain somewhat above 0.95 because its conservativeness is inherent.

Turn 21

User

currently for KS I use this:

```
ks\{}_{}_bounds\{}_{}_mc <- function(\{} n,\{} alpha = 0.05,\{} B = 100000,\{} seed = 123\{} ) \{\{}  
set.seed(seed)
```

```
D <- numeric(B)
```

```
for (b in seq\{\}_len(B)) \{\} u <- sort(runif(n))\{\} i <- seq\{\}_len(n)
```

```
# Exact two-sided one-sample KS statistic
```

```
D_plus <- max(i / n - u)
```

```
D_minus <- max(u - (i - 1) / n)
```

```
D[b] <- max(D_plus, D_minus)
```

}

Estimated (1-alpha) critical value

```
d <- unname(quantile(D, probs = 1 - alpha, type = 8))
```

```
i <- seq\{\}_len(n)
```

```
lower <- pmax(0, i / n - d)\{\} upper <- pmin(1, i / n + d)
```

```
list(\{ } n = n, \{ } alpha = alpha, \{ } critical \{ } _value = d, \{ } bounds = data.frame(\{ } i = i, \{ } p = i /
n, \{ } lower = lower, \{ } upper = upper \{ } ) \{ } ) \{ } }
```

How you suggest to fix it?

Assistant

Your KS statistic calculation is fine. The fix is only in how you convert the critical value d into order-statistic bounds.

Right now you use

```
lower <- pmax(0, i / n - d)
```

```
upper <- pmin(1, i / n + d)
```

but for $X_{(i)}$, the correct simultaneous KS constraints are

$$\frac{\dot{i}}{n} - d \leq F(X_{(i)})$$

and

$$F(X_{(i)}) \leq \frac{i-1}{n} + d.$$

So change only the upper bound to use $(i - 1)/n$.

```
ks_bounds_mc <- function(
```

$$n,$$
 $\alpha = 0.05,$
$$B = 100000,$$

```
seed = 123
```

) {

```
set.seed(seed)
```

```
D <- numeric(B)
```

```
i <- seq_len(n)
```

```
# Calibrate the exact finite-sample two-sided KS statistic
```

```
# under U(0,1)
```

```
# -----
```

```
for (b in seq_len(B)) {
```

```
u <- sort(runif(n))
```

```
D_plus <- max(i / n - u)
```

```
D_minus <- max(u - (i - 1) / n)
```

```
D[b] <- max(D_plus, D_minus)
```

}

```

# Estimated (1-alpha) critical value
d <- unname(
  quantile(
    D,
    probs = 1 - alpha,
    type = 8
  )
)

# -----
# Correct order-statistic bounds
#
# KS event:
#
# D_n <= d
#
# is equivalent to
#
# i/n - d <= U_(i) <= (i-1)/n + d
#
# for every i.
# -----

lower <- pmax(
  0,
  i / n - d
)

upper <- pmin(
  1,
  (i - 1) / n + d
)

list(
  n = n,
  alpha = alpha,
  critical_value = d,

  bounds = data.frame(
    i = i,
    p_lower = i / n,
    p_upper = (i - 1) / n,
    lower = lower,
    upper = upper
  )
)
}

```

The reason is visible directly from your own KS statistic:

```

D_plus <- max(i / n - u)
D_minus <- max(u - (i - 1) / n)

```

The event

```
D_plus <= d
```

means

$$\frac{i}{n} - U_{(i)} \leq d$$

for every i , hence

$$U_{(i)} \geq \frac{i}{n} - d.$$

And

```
D_minus <= d
```

means

$$U_{(i)} - \frac{i-1}{n} \leq d,$$

hence

$$U_{(i)} \leq \frac{i-1}{n} + d.$$

So your simulation of D_n was already using the correct left/right ECDF levels; the inconsistency was only introduced afterward when you constructed `upper`.

I would also verify the corrected bounds directly:

```
check_ks_coverage <- function(
  n,
  lower,
  upper,
  B = 100000,
  seed = 456
) {

  set.seed(seed)

  covered <- logical(B)

  for (b in seq_len(B)) {
    u <- sort(runif(n))

    covered[b] <- all(
      u >= lower &
      u <= upper
    )
  }

  mean(covered)
}
```

Then:

```
ks <- ks_bounds_mc(
  n = 50,
  alpha = 0.05,
  B = 500000
)

check_ks_coverage(
  n = 50,
  lower = ks$bounds$lower,
  upper = ks$bounds$upper,
  B = 200000
)
```

That should come out close to

0.95

up to Monte Carlo error.

This same indexing correction should also be applied to DKW, Berk–Jones, and Dümbgen–Wellner before you rerun the comparison.

User

```

    bj\{\}_bounds <- function(\{\} n,\{\} alpha = 0.05,\{\} B = 100000,\{\} seed = 123,\{\} critical = NULL\{\}
) \{\}
    if (is.null(critical)) \{\{\} critical <- bj\{\}\_critical\{\}\_mc(\{\} n = n,\{\} alpha = alpha,\{\} B = B,\{\} seed
= seed\{\} )\{\} \{\}
    i <- seq\{\}\_len(n)\{\} p <- i / n
    lower <- numeric(n)\{\} upper <- numeric(n)
    for (j in seq\{\}\_len(n)) \{\{\} bounds\{\}\_j <- bj\{\}\_interval\{\}\_at\{\}\_p(\{\} p = p[j],\{\} critical = criti-
cal,\{\} n = n\{\} )

```

```
list(\{\} n = n,\{\} alpha = alpha,\{\} critical\{\}_value = critical,\{\} bounds = data.frame(\{\} i = i,\{\} p
= p,\{\} lower = lower,\{\} upper = upper\{\} )\{\} )\{\} }
where to fix?
```

You need to use **different empirical probabilities for the lower and upper bounds**.
Right now both are computed from

$$p_i^L = \frac{i}{n}$$
$$p_i^U = \frac{i-1}{n}$$

So I would change your function to this:

Research provenance transcript - reader edition

```

lower_j <- bj_interval_at_p(
  p = p_lower[j],
  critical = critical,
  n = n
)

lower[j] <- lower_j["lower"]

# Upper endpoint:
# invert around p = (i-1)/n
upper_j <- bj_interval_at_p(
  p = p_upper[j],
  critical = critical,
  n = n
)

upper[j] <- upper_j["upper"]
}

list(
  n = n,
  alpha = alpha,
  critical_value = critical,

  bounds = data.frame(
    i = i,
    p_lower = p_lower,
    p_upper = p_upper,
    lower = lower,
    upper = upper
  )
)
}

```

The logic is:

$$nK\left(\frac{i}{n}, U_{(i)}\right) \leq c$$

gives the lower constraint on $U_{(i)}$, while

$$nK\left(\frac{i-1}{n}, U_{(i)}\right) \leq c$$

gives the upper constraint.

So the resulting bounds are

$$\ell_i = \text{lower root of } nK\left(\frac{i}{n}, q\right) = c$$

and

$$u_i = \text{upper root of } nK\left(\frac{i-1}{n}, q\right) = c.$$

Your existing `bj_interval_at_p()` can stay unchanged. You just call it twice with different p 's. There is also a nice endpoint consequence:

- for $i = 1$, the upper bound uses $p = 0$;
- for $i = n$, the lower bound uses $p = 1$.

Your helper already appears designed to handle $p == 0$ and $p == 1$, so that should work naturally.

After making this change, I would rerun the empirical coverage check. BJ should move noticeably closer to its calibrated nominal level.

Turn 23

User

```
dw\_bounds <- function(\{ n,\{ alpha = 0.05,\{ B = 100000,\{ nu = 1,\{ seed = 123,\{ critical =
NULL,\{ tol = 1e-10\{ } {
  if (is.null(critical)) {\{ critical <- dw\_critical\_mc(\{ n = n,\{ alpha = alpha,\{ B = B,\{ nu
= nu,\{ seed = seed\{ } )\{ }
  # Empirical CDF levels 0/n,...,n/n
  i\_full <- 0\{ p\_full <- i\_full / n
  upper\_full <- numeric(n + 1)\{ lower\_full <- numeric(n + 1)
  # By definition upper bound at p = 1 is 1
  upper\_full[n + 1] <- 1
  # Compute b\_i for i = 0,...,n-1
  for (j in 1) {\{ upper\_full[j] <- dw\_upper\_at\_p(\{ p = p\_full[j],\{ n = n,\{ critical
= critical,\{ nu = nu,\{ tol = tol\{ } )\{ }
  # Symmetry:
  # a\_n-i = 1 - b\_i
  for (j in 1:(n + 1)) {\{ lower\_full[n + 2 - j] <- 1 - upper\_full[j]\{ }
  # At order statistic X\_i, Fn(X\_i) = i/n,
  # so use indices i = 1,...,n.
  bounds <- data.frame(\{ i = seq\_len(n),\{ p = seq\_len(n) / n,\{ lower = lower\_full[2:(n +
1)],\{ upper = upper\_full[2:(n + 1)]\{ }
  full\_bounds <- data.frame(\{ i = 0,\{ p = p\_full,\{ lower = lower\_full,\{ upper = up-
per\_full\{ }
  list(\{ n = n,\{ alpha = alpha,\{ nu = nu,\{ critical\_value = critical,\{ bounds = bounds,\{
full\_bounds = full\_bounds\{ } )\{ }
  and finally this
```

Assistant

For DW the same indexing issue applies. You should use $p = i/n$ for the **lower** bound and $p = (i - 1)/n$ for the **upper** bound.

Also, your pasted code seems to have lost the vector ranges in a few places: `i_full <- 0` should be `0:n`, and `for (j in 1)` should be `for (j in 1:n)`.

A corrected version is:

```
dw_bounds <- function(
  n,
  alpha = 0.05,
  B = 100000,
  nu = 1,
  seed = 123,
  critical = NULL,
  tol = 1e-10
) {
  if (is.null(critical)) {
    critical <- dw_critical_mc(
      n = n,
      alpha = alpha,
      B = B,
      nu = nu,
      seed = seed
    )
  }

  i <- seq_len(n)

  p_lower <- i / n
  p_upper <- (i - 1) / n

  lower <- numeric(n)
  upper <- numeric(n)

  for (j in seq_len(n)) {
    # -----
```

```

# LOWER endpoint
#
# Need the lower root associated with p = i/n.
#
# Use symmetry:
#
# lower(p) = 1 - upper(1 - p)
# -----

lower[j] <- 1 - dw_upper_at_p(
  p = 1 - p_lower[j],
  n = n,
  critical = critical,
  nu = nu,
  tol = tol
)

# -----
# UPPER endpoint
#
# Use p = (i-1)/n
# -----

upper[j] <- dw_upper_at_p(
  p = p_upper[j],
  n = n,
  critical = critical,
  nu = nu,
  tol = tol
)
}

list(
  n = n,
  alpha = alpha,
  nu = nu,
  critical_value = critical,

  bounds = data.frame(
    i = i,
    p_lower = p_lower,
    p_upper = p_upper,
    lower = lower,
    upper = upper
  )
)
}

```

This implements the correct constraints

$$H_{n,\nu} \left(\frac{i}{n}, U_{(i)} \right) \leq c$$

for the lower side and

$$H_{n,\nu} \left(\frac{i-1}{n}, U_{(i)} \right) \leq c$$

for the upper side.

Using symmetry,

$$\ell_i = 1 - u \left(1 - \frac{i}{n} \right),$$

where `dw_upper_at_p()` computes the upper root for a given p .

So the final bounds are

$$\ell_i = 1 - \text{UpperDW} \left(1 - \frac{i}{n} \right)$$

and

$$u_i = \text{UpperDW} \left(\frac{i-1}{n} \right).$$

I actually prefer this direct formulation over constructing the full `0:n` grid and then shifting indices, because it makes the left/right ECDF roles explicit and avoids exactly the indexing bug we just found.

After fixing KS, BJ, and DW—and DKW similarly—I would rerun your coverage simulation. If the calibration functions are correct, KS/BJ/DW should now be much closer to 0.95.

Turn 24

User

	method	expected_area	se_area	expected_mu_length	se_mu_length
1	KS	0.6262179	0.001644183	0.7516943	0.001250998
2	DKW	0.6720826	0.001746067	0.7764646	0.001282348
3	Berk-Jones	0.4327629	0.001344886	0.8522488	0.001315178
4	Dumbgen-Wellner	0.4290583	0.001285816	0.8029331	0.001246876
	expected				
1		1.4084462	0.001873168	0.9498	2
2		1.4612101	0.001929434	0.9583	3
3		0.8776192	0.001473395	0.9540	4
4		0.9171378	0.001476360	0.9528	

The results changed drastically

Assistant

Yes — and this is exactly what I would expect after fixing the indexing.

Your old bounds were too wide because you used i/n on both sides. The corrected bounds use

$$\ell_i \text{ from } \frac{i}{n}, \quad u_i \text{ from } \frac{i-1}{n},$$

which shrinks the admissible range for every order statistic. So all induced (μ, σ) regions should get noticeably smaller.

The important thing is that the new coverage numbers now make sense:

| Method | Coverage | KS | 0.9498 | DKW | 0.9583 | BJ | 0.9540 | DW | 0.9528 |

That is a very reassuring validation. KS is essentially exactly at 95%, while DKW remains conservative, as expected.

The substantive ranking is also clearer now:

DW has the smallest expected area

with

$$0.4291,$$

very slightly ahead of Berk-Jones at

$$0.4328.$$

Relative to KS,

$$1 - \frac{0.4291}{0.6262} \approx 31.5\%$$

smaller expected joint area.

For the marginal scale interval, Berk-Jones is best:

$$0.8776,$$

versus

$$0.9171$$

for DW and

$$1.4084$$

for KS. Thus BJ reduces expected scale-interval length relative to KS by about

$$1 - \frac{0.8776}{1.4084} \approx 37.7\%.$$

But KS remains clearly best for location:

$$E[L_\mu]_{\text{KS}} = 0.7517,$$

compared with

0.8029

for DW and

0.8522

for BJ.

So the main phenomenon survives the correction — in fact it becomes stronger:

KS : best for μ ,
BJ : best for σ ,
DW : best for joint area.

That is potentially the central empirical result of your project.

One caveat remains: BJ and DW have coverage around 95.3% – 95.4%, while KS is 94.98%. The difference is small, but if you want a very clean efficiency comparison, I would next recalibrate BJ and DW critical values slightly so all three are effectively at exactly 95%.

But I would **not** touch DKW: its conservativeness is part of the method. Treat DKW as the guaranteed-coverage benchmark.

At this point, the next serious experiment should be to repeat this for

$$n = 20, 50, 100, 200, 500$$

and plot the ratios

$$\frac{E[A_B]}{E[A_{KS}]}, \quad \frac{E[L_{\mu,B}]}{E[L_{\mu,KS}]}, \quad \frac{E[L_{\sigma,B}]}{E[L_{\sigma,KS}]}.$$

If the same ranking persists across n , you have a strong, systematic result rather than a single-sample-size observation.

Turn 25

User

```
res <- compare\{\}_bands\{\}_mc(\{\} n = 20,\{\} alpha = 0.05,\{\} M = 10000,\{\} band\{\}_calibration\{\}_B
= 100000,\{\} seed = 42\{\} )
res$summary
```

method	expected_area	se_area	expected_mu_length	se_mu_length
--------	---------------	---------	--------------------	--------------

```
1 KS 2.053337 0.007586032 1.255328 0.002697333\{\} 2 DKW 2.301210 0.008422749 1.313689 0.002783937\{\} 3
Berk\{\}_Jones 1.355428 0.005444576 1.366854 0.002691170\{\} 4 Dumbgen\{\}_Wellner 1.351132 0.005365738
1.325950 0.002622187\{\} expected\{\}_sigma\{\}_length se\{\}_sigma\{\}_length empirical\{\}_coverage\{\} 1
2.771548 0.005321841 0.9538\{\} 2 2.967795 0.005682742 0.9649\{\} 3 1.718948 0.003616929 0.9511\{\} 4 1.760932
0.003697668 0.9537
```

```
res <- compare\{\}_bands\{\}_mc(\{\} n = 100,\{\} alpha = 0.05,\{\} M = 10000,\{\} band\{\}_calibration\{\}_B
= 100000,\{\} seed = 42\{\} )
res$summary
```

method	expected_area	se_area	expected_mu_length	se_mu_length
--------	---------------	---------	--------------------	--------------

```
1 KS 0.2974582 0.0006208280 0.5297760 0.0007070042\{\} 2 DKW 0.3061991 0.0006346701 0.5371374 0.0007138834\{\}
3 Berk\{\}_Jones 0.2036071 0.0005665057 0.6015742 0.0007961588\{\} 4 Dumbgen\{\}_Wellner 0.2030426 0.0005253973
0.5613438 0.0007351036\{\} expected\{\}_sigma\{\}_length se\{\}_sigma\{\}_length empirical\{\}_coverage\{\}
1 0.9487798 0.0009953801 0.9515\{\} 2 0.9625726 0.0010052170 0.9548\{\} 3 0.5848452 0.0009092797 0.9488\{\}
4 0.6202247 0.0008847345 0.9502
```

```
res <- compare\{\}_bands\{\}_mc(\{\} n = 200,\{\} alpha = 0.05,\{\} M = 10000,\{\} band\{\}_calibration\{\}_B
= 100000,\{\} seed = 42\{\} )
res$summary
```

method	expected_area	se_area	expected_mu_length	se_mu_length
--------	---------------	---------	--------------------	--------------

```
1 KS 0.14304499 0.0002491445 0.3709128 0.0004076867\{\} 2 DKW 0.14595512 0.0002523746 0.3745682 0.0004097782\{\}
3 Berk\{\}_Jones 0.10044869 0.0002543893 0.4292671 0.0005102074\{\} 4 Dumbgen\{\}_Wellner 0.09884934
0.0002286189 0.3942296 0.0004543730\{\} expected\{\}_sigma\{\}_length se\{\}_sigma\{\}_length empirical\{\}_coverage\{\}
1 0.6508988 0.0005526427 0.9506\{\} 2 0.6573114 0.0005549727 0.9527\{\} 3 0.4040955 0.0005829399 0.9472\{\}
4 0.4292213 0.0005499280 0.9504
```

Assistant

These results are very consistent across $n = 20, 100, 200$, and they strengthen the case that the pattern is structural rather than accidental.

The same ranking keeps appearing:

KS best for μ

Berk–Jones best for σ

Dümbgen–Wellner best for joint area

with DKW consistently slightly worse than KS because of its conservativeness.

For expected area, the relative improvement versus KS is substantial and stable. At $n = 20$,

$$1 - \frac{1.351}{2.053} \approx 34.2\%$$

for DW. At $n = 100$,

$$1 - \frac{0.2030}{0.2975} \approx 31.7\%,$$

and at $n = 200$,

$$1 - \frac{0.09885}{0.14304} \approx 30.9\%.$$

So DW seems to retain roughly a **31–34% reduction in expected joint area** relative to KS over this range.

For scale, BJ is even more striking. Relative to KS:

- $n = 20$: about 38.0% shorter,
- $n = 100$: about 38.4% shorter,
- $n = 200$: about 37.9% shorter.

That is remarkably stable. It suggests that the scale advantage may have a meaningful asymptotic interpretation, not just a finite- n quirk.

For location, KS remains better, and BJ pays a persistent penalty. At $n = 200$,

$$\frac{0.4293}{0.3709} - 1 \approx 15.7\%$$

longer than KS. DW is closer:

$$\frac{0.3942}{0.3709} - 1 \approx 6.3\%.$$

So DW looks like the best compromise: much better joint area and scale precision than KS, while sacrificing relatively little location precision.

The coverage results are also reassuring. They hover around 95%, with DKW slightly conservative as expected. BJ is sometimes just below 95%, but by an amount compatible with Monte Carlo calibration error from `band_calibration_B = 100000` plus the simulation error in the outer loop.

The next thing I would do is summarize the comparison using **relative efficiency curves** rather than raw lengths. For example, for each n ,

$$R_A(B, n) = \frac{E[A_B]}{E[A_{KS}]},$$

$$R_\mu(B, n) = \frac{E[L_{\mu, B}]}{E[L_{\mu, KS}]},$$

$$R_\sigma(B, n) = \frac{E[L_{\sigma, B}]}{E[L_{\sigma, KS}]}.$$

Then your results become visually very clear:

- DW area ratio $\approx 0.66 - 0.69$,
- BJ scale ratio ≈ 0.62 ,
- BJ location ratio $\approx 1.09 - 1.16$,
- DW location ratio $\approx 1.05 - 1.07$.

That is already close to a publishable-style empirical finding: ****different simultaneous CDF bands induce systematically different efficiency profiles for different parameter targets, and the ranking is stable with n .****

I would next add $n = 50, 500, 1000$ and then move to a second base family, probably logistic or Laplace, to test whether the same ranking is specific to the normal quantile geometry or persists more broadly.

Turn 26

User

Do I understand correctly that if I run

```
# =====
# MONTE CARLO COMPARISON OF CONFIDENCE REGIONS
#
# Assumes these functions already exist:
# ks\{}_bounds\{}_mc()
# dkwl{}\{}_bounds()
# bj\{}\{}_bounds()
# dw\{}\{}_bounds()
# location\{}\{}_scale\{}\{}_confidence()
#
# Example target:
#  $X \sim N(0,1)$ 
# =====
compare\{}\{}_bands\{}\{}_mc <- function(\{} n = 50, \{} alpha = 0.05, \{} M = 10000, \{} band\{}\{}_calibration\{}\{}_B
= 100000, \{} seed = 123, \{} lc.quantile.function = qnorm\{} ) {
  set.seed(seed)
  # -----
  # 1. Precompute band bounds ONCE
  # -----
  KS <- ks\{}\{}_bounds\{}\{}_mc(\{} n = n, \{} alpha = alpha, \{} B = band\{}\{}_calibration\{}\{}_B, \{} seed
= seed + 1\{} )$bounds
  DKW <- dkwl{}\{}_bounds(\{} n = n, \{} alpha = alpha\{} )
  BJ <- bj\{}\{}_bounds(\{} n = n, \{} alpha = alpha, \{} B = band\{}\{}_calibration\{}\{}_B, \{} seed = seed +
2\{} )$bounds
  DW <- dw\{}\{}_bounds(\{} n = n, \{} alpha = alpha, \{} B = band\{}\{}_calibration\{}\{}_B, \{} nu = 1, \{}
seed = seed + 3\{} )$bounds
  bands <- list(\{} KS = KS, \{} DKW = DKW, \{} Berk\{}_Jones = BJ, \{} Dumbgen\{}_Wellner =
DW\{} )
  # -----
  # 2. Storage
  # -----
  methods <- names(bands)
  area <- matrix(\{} NA\{}_real\{}__, \{} nrow = M, \{} ncol = length(methods), \{} dimnames = list(NULL,
methods)\{} )
  mu\{}_length <- area\{} sigma\{}_length <- area
  # Optional: joint-region coverage of true (mu, sigma) = (0,1)
  coverage <- matrix(\{} FALSE, \{} nrow = M, \{} ncol = length(methods), \{} dimnames = list(NULL,
methods)\{} )
  # -----
  # 3. Monte Carlo loop
  # -----
  for (m in seq\{}\{}_len(M)) {

# Simulate from standard normal.
# By location-scale equivariance this is enough.
x <- rnorm(n)

x_sorted <- sort(x)

for (j in seq_along(methods)) {

  B <- bands[[j]]

  fit <- location_scale_confidence(
    x = x_sorted,
    lower = B$lower,
    upper = B$upper,
    qfun = lc.quantile.function
  )
}
```

```

if (isTRUE(fit$feasible) && isTRUE(fit$bounded)) {

  area[m, j] <- fit$area
  mu_length[m, j] <- fit$mu_length
  sigma_length[m, j] <- fit$sigma_length

  # -----
  # Check whether true point (mu,sigma) = (0,1)
  # belongs to the confidence region.
  #
  # Equivalent to checking all original inequalities.
  # -----

  a <- fit$a
  b <- fit$b

  lower_mu_at_true_sigma <- max(
    x_sorted[is.finite(b)] -
    b[is.finite(b)]
  )

  upper_mu_at_true_sigma <- min(
    x_sorted[is.finite(a)] -
    a[is.finite(a)]
  )

  coverage[m, j] <-
    lower_mu_at_true_sigma <= 0 &&
    0 <= upper_mu_at_true_sigma
}
}

# Optional progress indicator
if (m %% 1000 == 0) {
  print(paste0("Completed ", m, " of ", M))
  flush.console()
}

}

# -----
# 4. Summaries
# -----
mean\{\}_area <- colMeans(area, na.rm = TRUE)\{\} mean\{\}_mu <- colMeans(mu\{\}_length, na.rm
= TRUE)\{\} mean\{\}_sigma <- colMeans(sigma\{\}_length, na.rm = TRUE)\{\} empirical\{\}_coverage <-
colMeans(coverage)
# Monte Carlo standard errors
se\{\}_area <- apply(\{\} area, \{\} 2, \{\} sd, \{\} na.rm = TRUE\{\} ) / sqrt(M)
se\{\}_mu <- apply(\{\} mu\{\}_length, \{\} 2, \{\} sd, \{\} na.rm = TRUE\{\} ) / sqrt(M)
se\{\}_sigma <- apply(\{\} sigma\{\}_length, \{\} 2, \{\} sd, \{\} na.rm = TRUE\{\} ) / sqrt(M)
summary <- data.frame(\{\} method = methods,

expected_area = mean_area,
se_area = se_area,

expected_mu_length = mean_mu,
se_mu_length = se_mu,

expected_sigma_length = mean_sigma,
se_sigma_length = se_sigma,

empirical_coverage = empirical_coverage,

row.names = NULL

)

# -----
# 5. Return everything
# -----
list(\{\} summary = summary, \{\} area = area, \{\} mu\{\}_length = mu\{\}_length, \{\} sigma\{\}_length =
sigma\{\}_length, \{\} coverage = coverage, \{\} bands = bands\{\} )\{\} }

```

with `lc.quantile.function = qlaplace` I'll get what we want?

Assistant

Almost. Changing only

```
lc.quantile.function = qlaplace
```

is **not enough**, because your simulation loop still contains

```
x <- rnorm(n)
```

so you would be generating normal data but constructing a Laplace location–scale confidence region. That would measure behavior under **model misspecification**, not the Laplace-family experiment we want.

For the Laplace experiment, both components must use the same standardized base distribution G :

$$X_i \sim G_{0,1}$$

and

$$qfun = G^{-1}.$$

The cleanest modification is to add an RNG function.

```
compare_bands_mc <- function(
  n = 50,
  alpha = 0.05,
  M = 10000,
  band_calibration_B = 100000,
  seed = 123,
  lc.quantile.function = qnorm,
  lc.random.function = rnorm
) {
  set.seed(seed)

  # -----
  # 1. Precompute band bounds ONCE
  # -----

  KS <- ks_bounds_mc(
    n = n,
    alpha = alpha,
    B = band_calibration_B,
    seed = seed + 1
  )$bounds

  DKW <- dkw_bounds(
    n = n,
    alpha = alpha
  )

  BJ <- bj_bounds(
    n = n,
    alpha = alpha,
    B = band_calibration_B,
    seed = seed + 2
  )$bounds

  DW <- dw_bounds(
    n = n,
    alpha = alpha,
    B = band_calibration_B,
    nu = 1,
    seed = seed + 3
  )$bounds

  bands <- list(
    KS = KS,
    DKW = DKW,
    Berk_Jones = BJ,
    Dumbgen_Wellner = DW
  )

  methods <- names(bands)
```

```

# -----
# 2. Storage
# -----

area <- matrix(
  NA_real_,
  nrow = M,
  ncol = length(methods),
  dimnames = list(NULL, methods)
)

mu_length <- area
sigma_length <- area

coverage <- matrix(
  FALSE,
  nrow = M,
  ncol = length(methods),
  dimnames = list(NULL, methods)
)

# -----
# 3. Monte Carlo loop
# -----

for (m in seq_len(M)) {

  # Simulate from standardized base distribution G
  x <- lc.random.function(n)

  x_sorted <- sort(x)

  for (j in seq_along(methods)) {

    B <- bands[[j]]

    fit <- location_scale_confidence(
      x = x_sorted,
      lower = B$lower,
      upper = B$upper,
      qfun = lc.quantile.function
    )

    if (isTRUE(fit$feasible) && isTRUE(fit$bounded)) {

      area[m, j] <- fit$area
      mu_length[m, j] <- fit$mu_length
      sigma_length[m, j] <- fit$sigma_length

      # Check coverage of true (mu, sigma) = (0,1)

      a <- fit$a
      b <- fit$b

      lower_mu_at_true_sigma <- max(
        x_sorted[is.finite(b)] -
        b[is.finite(b)]
      )

      upper_mu_at_true_sigma <- min(
        x_sorted[is.finite(a)] -
        a[is.finite(a)]
      )

      coverage[m, j] <-
        lower_mu_at_true_sigma <= 0 &&
        0 <= upper_mu_at_true_sigma
    }
  }

  if (m %% 1000 == 0) {
    print(paste0("Completed ", m, " of ", M))
    flush.console()
  }
}

```

```

# -----
# 4. Summaries
# -----

mean_area <- colMeans(area, na.rm = TRUE)
mean_mu <- colMeans(mu_length, na.rm = TRUE)
mean_sigma <- colMeans(sigma_length, na.rm = TRUE)
empirical_coverage <- colMeans(coverage)

se_area <- apply(area, 2, sd, na.rm = TRUE) / sqrt(M)
se_mu <- apply(mu_length, 2, sd, na.rm = TRUE) / sqrt(M)
se_sigma <- apply(sigma_length, 2, sd, na.rm = TRUE) / sqrt(M)

summary <- data.frame(
  method = methods,

  expected_area = mean_area,
  se_area = se_area,

  expected_mu_length = mean_mu,
  se_mu_length = se_mu,

  expected_sigma_length = mean_sigma,
  se_sigma_length = se_sigma,

  empirical_coverage = empirical_coverage,

  row.names = NULL
)

list(
  summary = summary,
  area = area,
  mu_length = mu_length,
  sigma_length = sigma_length,
  coverage = coverage,
  bands = bands
)
}

```

Then, assuming your `qlaplace()` is parameterized so that `qlaplace(p)` represents the standardized Laplace distribution with location 0 and scale 1, you could do:

```

res_laplace <- compare_bands_mc(
  n = 100,
  alpha = 0.05,
  M = 10000,
  band_calibration_B = 100000,
  seed = 42,
  lc.quantile.function = qlaplace,
  lc.random.function = rlaplace
)

```

An even safer approach

Since your whole theory is quantile-based, you don't actually need a separate random-generator function. Generate

$$X = G^{-1}(U), \quad U \sim U(0, 1).$$

Then the quantile function and simulated distribution **cannot accidentally disagree**.

I would actually modify the function to use:

```
x <- lc.quantile.function(runif(n))
```

instead of

```
x <- lc.random.function(n)
```

Then your function only needs:

```
lc.quantile.function = qnorm
```

and for Laplace:

```
lc.quantile.function = qlaplace
```

The relevant part becomes simply:

```
for (m in seq_len(M)) {  
  
  x <- lc.quantile.function(runif(n))  
  x_sorted <- sort(x)  
  
  ...  
}
```

This is probably the version I would use for the research code.

Then:

```
res_laplace <- compare_bands_mc(  
  n = 100,  
  alpha = 0.05,  
  M = 10000,  
  band_calibration_B = 100000,  
  seed = 42,  
  lc.quantile.function = qlaplace  
)
```

does exactly what we want.

And importantly, **the KS/DKW/BJ/DW bounds do not need to be recalibrated for Laplace.** They are distribution-free because they are calibrated in the uniform probability scale. Only the transformation

$$a_i = G^{-1}(\ell_i), \quad b_i = G^{-1}(u_i)$$

changes when you move from normal to Laplace.

So this gives us a very clean experiment:

same four probability bands	+	different G^{-1}	$\implies$	different induced parameter-region geometry.
-----------------------------	---	--------------------	------------	--

That is exactly what we want to study next.

Turn 27

User

changed it to this

```
# =====
# MONTE CARLO COMPARISON OF CONFIDENCE REGIONS
#
# Assumes these functions already exist:
# ks\{}_bounds\{}_mc()
# dkw\{}_bounds()
# bj\{}_bounds()
# dw\{}_bounds()
# location\{}_scale\{}_confidence()
#
# Example target:
# X\{}_i \{} ~ N(0,1)
# =====
compare\{}_bands\{}_mc <- function(\{} n = 50,\{} alpha = 0.05,\{} M = 10000,\{} band\{}_calibration\{}_B
= 100000,\{} seed = 123,\{} ls.quantile.function = qnorm\{} ) {
  set.seed(seed)
  # -----
  # 1. Precompute band bounds ONCE
  # -----
  KS <- ks\{}_bounds\{}_mc(\{} n = n,\{} alpha = alpha,\{} B = band\{}_calibration\{}_B,\{} seed
= seed + 1\{} )$bounds
  DKW <- dkw\{}_bounds(\{} n = n,\{} alpha = alpha\{} )
  BJ <- bj\{}_bounds(\{} n = n,\{} alpha = alpha,\{} B = band\{}_calibration\{}_B,\{} seed = seed +
2\{} )$bounds
  DW <- dw\{}_bounds(\{} n = n,\{} alpha = alpha,\{} B = band\{}_calibration\{}_B,\{} nu = 1,\{}
seed = seed + 3\{} )$bounds
  bands <- list(\{} KS = KS,\{} DKW = DKW,\{} Berk\{}_Jones = BJ,\{} Dumbgen\{}_Wellner =
DW\{} )
  # -----
  # 2. Storage
  # -----
  methods <- names(bands)
  area <- matrix(\{} NA\{}_real\{_,\{} nrow = M,\{} ncol = length(methods),\{} dimnames = list(NULL,
methods)\{} )
  mu\{}_length <- area\{} sigma\{}_length <- area
  # Optional: joint-region coverage of true (mu, sigma) = (0,1)
  coverage <- matrix(\{} FALSE,\{} nrow = M,\{} ncol = length(methods),\{} dimnames = list(NULL,
methods)\{} )
  # -----
  # 3. Monte Carlo loop
  # -----
  for (m in seq\{}_len(M)) {

# Simulate from standard normal.
# By location-scale equivariance this is enough.
x <- ls.quantile.function(runif(n))

x_sorted <- sort(x)

for (j in seq_along(methods)) {

  B <- bands[[j]]

  fit <- location_scale_confidence(
    x = x_sorted,
    lower = B$lower,
    upper = B$upper,
    qfun = lc.quantile.function
  )
}
```

```

if (isTRUE(fit$feasible) && isTRUE(fit$bounded)) {

  area[m, j] <- fit$area
  mu_length[m, j] <- fit$mu_length
  sigma_length[m, j] <- fit$sigma_length

  # -----
  # Check whether true point (mu,sigma) = (0,1)
  # belongs to the confidence region.
  #
  # Equivalent to checking all original inequalities.
  # -----

  a <- fit$a
  b <- fit$b

  lower_mu_at_true_sigma <- max(
    x_sorted[is.finite(b)] -
    b[is.finite(b)]
  )

  upper_mu_at_true_sigma <- min(
    x_sorted[is.finite(a)] -
    a[is.finite(a)]
  )

  coverage[m, j] <-
    lower_mu_at_true_sigma <= 0 &&
    0 <= upper_mu_at_true_sigma
}
}

# Optional progress indicator
if (m %% 1000 == 0) {
  print(paste0("Completed ", m, " of ", M))
  flush.console()
}

}

# -----
# 4. Summaries
# -----
mean\{\}_area <- colMeans(area, na.rm = TRUE)\{\} mean\{\}_mu <- colMeans(mu\{\}_length, na.rm
= TRUE)\{\} mean\{\}_sigma <- colMeans(sigma\{\}_length, na.rm = TRUE)\{\} empirical\{\}_coverage <-
colMeans(coverage)
# Monte Carlo standard errors
se\{\}_area <- apply(\{\} area, \{\} 2, \{\} sd, \{\} na.rm = TRUE\{\} ) / sqrt(M)
se\{\}_mu <- apply(\{\} mu\{\}_length, \{\} 2, \{\} sd, \{\} na.rm = TRUE\{\} ) / sqrt(M)
se\{\}_sigma <- apply(\{\} sigma\{\}_length, \{\} 2, \{\} sd, \{\} na.rm = TRUE\{\} ) / sqrt(M)
summary <- data.frame(\{\} method = methods,

expected_area = mean_area,
se_area = se_area,

expected_mu_length = mean_mu,
se_mu_length = se_mu,

expected_sigma_length = mean_sigma,
se_sigma_length = se_sigma,

empirical_coverage = empirical_coverage,

row.names = NULL

)

# -----
# 5. Return everything
# -----
list(\{\} summary = summary, \{\} area = area, \{\} mu\{\}_length = mu\{\}_length, \{\} sigma\{\}_length =
sigma\{\}_length, \{\} coverage = coverage, \{\} bands = bands\{\} )\{\} }

```

Assistant

Almost. There is one actual bug and one reproducibility improvement I would make.

Inside `location_scale_confidence()` you currently have:

```
qfun = lc.quantile.function
```

but your function argument is named:

```
ls.quantile.function
```

So change it to:

```
fit <- location_scale_confidence(  
  x = x_sorted,  
  lower = B$lower,  
  upper = B$upper,  
  qfun = ls.quantile.function  
)
```

Then this part is correct:

```
x <- ls.quantile.function(runif(n))
```

because if $U \sim U(0, 1)$, then

$$G^{-1}(U) \sim G,$$

so the same quantile function defines both the simulated base distribution and the location-scale model you're fitting.

I would also reset the random seed **after calibrating the bands**. Your calls to `ks_bounds_mc()`, `bj_bounds()`, and `dw_bounds()` each call `set.seed()`, which changes the global RNG state. So add:

```
# Bands have now been calibrated.  
# Reset RNG for the actual Monte Carlo experiment.  
set.seed(seed)
```

right before the main Monte Carlo loop.

The relevant corrected section is:

```
# -----  
# 3. Monte Carlo loop  
# -----  
  
# Reset RNG after band calibration  
set.seed(seed)  
  
for (m in seq_len(M)) {  
  
  # Generate standardized data from the base distribution G  
  x <- ls.quantile.function(runif(n))  
  
  x_sorted <- sort(x)  
  
  for (j in seq_along(methods)) {  
  
    B <- bands[[j]]  
  
    fit <- location_scale_confidence(  
      x = x_sorted,  
      lower = B$lower,  
      upper = B$upper,  
      qfun = ls.quantile.function  
    )  
  
    if (isTRUE(fit$feasible) && isTRUE(fit$bounded)) {  
  
      area[m, j] <- fit$area  
      mu_length[m, j] <- fit$mu_length  
      sigma_length[m, j] <- fit$sigma_length  
  
      a <- fit$a  
      b <- fit$b  
  
      lower_mu_at_true_sigma <- max(  

```

```

        x_sorted[is.finite(b)] -
        b[is.finite(b)]
    )

    upper_mu_at_true_sigma <- min(
        x_sorted[is.finite(a)] -
        a[is.finite(a)]
    )

    coverage[m, j] <-
        lower_mu_at_true_sigma <= 0 &&
        0 <= upper_mu_at_true_sigma
    }
}

if (m %% 1000 == 0) {
    print(paste0("Completed ", m, " of ", M))
    flush.console()
}
}

```

Then, for normal:

```

res_normal <- compare_bands_mc(
  n = 100,
  alpha = 0.05,
  M = 10000,
  band_calibration_B = 100000,
  seed = 42,
  ls.quantile.function = qnorm
)

```

For Laplace, assuming your `qlaplace()` defaults to location 0 and `**scale` parameter 1**:

```

res_laplace <- compare_bands_mc(
  n = 100,
  alpha = 0.05,
  M = 10000,
  band_calibration_B = 100000,
  seed = 42,
  ls.quantile.function = qlaplace
)

```

And likewise you can do logistic immediately with base R:

```

res_logistic <- compare_bands_mc(
  n = 100,
  alpha = 0.05,
  M = 10000,
  band_calibration_B = 100000,
  seed = 42,
  ls.quantile.function = qlogis
)

```

An important conceptual point: **the empirical joint coverage should be essentially the same across normal, logistic, Laplace, etc. for a given band.** The coverage event is really

$$\ell_i \leq U_{(i)} \leq u_i,$$

after the probability integral transform. Changing G changes the geometry—area and marginal lengths—but not the distribution-free coverage probability.

So if, for example, KS gives about 0.95 coverage under normal but something like 0.90 under Laplace, that would indicate an implementation/parameterization problem rather than a genuine statistical phenomenon.

Also change the stale comment

```
# Simulate from standard normal.
```

to something like

```
# Simulate from standardized base distribution G
```

since the function is now genuinely generic across location–scale families.

Turn 28

User

```
# =====
# MONTE CARLO COMPARISON OF CONFIDENCE REGIONS
#
# Assumes these functions already exist:
# ks\{}_bounds\{}_mc()
# dkwl{}\{}_bounds()
# bj\{}\{}_bounds()
# dw\{}\{}_bounds()
# location\{}\{}_scale\{}\{}_confidence()
#
# Example target:
#  $X \sim N(0,1)$ 
# =====
compare\{}\{}_bands\{}\{}_mc <- function(\{} n = 50,\{} alpha = 0.05,\{} M = 10000,\{} band\{}\{}_calibration\{}\{}_B
= 100000,\{} seed = 123,\{} ls.quantile.function = qnorm\{} ) {
  set.seed(seed)
  # -----
  # 1. Precompute band bounds ONCE
  # -----
  KS <- ks\{}\{}_bounds\{}\{}_mc(\{} n = n,\{} alpha = alpha,\{} B = band\{}\{}_calibration\{}\{}_B,\{} seed
= seed + 1\{} )$bounds
  DKW <- dkwl{}\{}_bounds(\{} n = n,\{} alpha = alpha\{} )
  BJ <- bj\{}\{}_bounds(\{} n = n,\{} alpha = alpha,\{} B = band\{}\{}_calibration\{}\{}_B,\{} seed = seed +
2\{} )$bounds
  DW <- dw\{}\{}_bounds(\{} n = n,\{} alpha = alpha,\{} B = band\{}\{}_calibration\{}\{}_B,\{} nu = 1,\{}
seed = seed + 3\{} )$bounds
  bands <- list(\{} KS = KS,\{} DKW = DKW,\{} Berk\{}_Jones = BJ,\{} Dumbgen\{}_Wellner =
DW\{} )
  # -----
  # 2. Storage
  # -----
  methods <- names(bands)
  area <- matrix(\{} NA\{}_real\{}__,\{} nrow = M,\{} ncol = length(methods),\{} dimnames = list(NULL,
methods)\{} )
  mu\{}_length <- area\{} sigma\{}_length <- area
  # Optional: joint-region coverage of true (mu, sigma) = (0,1)
  coverage <- matrix(\{} FALSE,\{} nrow = M,\{} ncol = length(methods),\{} dimnames = list(NULL,
methods)\{} )
  # -----
  # 3. Monte Carlo loop
  # -----
  set.seed(seed)
  for (m in seq\{}\{}_len(M)) {
    # Simulate from standard normal.
    # By location-scale equivariance this is enough.
    x <- ls.quantile.function(runif(n))

    x_sorted <- sort(x)

    for (j in seq_along(methods)) {
      B <- bands[[j]]

      fit <- location_scale_confidence(
        x = x_sorted,
        lower = B$lower,
        upper = B$upper,
        qfun = ls.quantile.function
      )
    }
  }
}
```

```

if (isTRUE(fit$feasible) && isTRUE(fit$bounded)) {

  area[m, j] <- fit$area
  mu_length[m, j] <- fit$mu_length
  sigma_length[m, j] <- fit$sigma_length

  # -----
  # Check whether true point (mu,sigma) = (0,1)
  # belongs to the confidence region.
  #
  # Equivalent to checking all original inequalities.
  # -----

  a <- fit$a
  b <- fit$b

  lower_mu_at_true_sigma <- max(
    x_sorted[is.finite(b)] -
    b[is.finite(b)]
  )

  upper_mu_at_true_sigma <- min(
    x_sorted[is.finite(a)] -
    a[is.finite(a)]
  )

  coverage[m, j] <-
    lower_mu_at_true_sigma <= 0 &&
    0 <= upper_mu_at_true_sigma
}
}

# Optional progress indicator
if (m %% 1000 == 0) {
  print(paste0("Completed ", m, " of ", M))
  flush.console()
}

}

# -----
# 4. Summaries
# -----
mean\{\}_area <- colMeans(area, na.rm = TRUE)\{\} mean\{\}_mu <- colMeans(mu\{\}_length, na.rm
= TRUE)\{\} mean\{\}_sigma <- colMeans(sigma\{\}_length, na.rm = TRUE)\{\} empirical\{\}_coverage <-
colMeans(coverage)
# Monte Carlo standard errors
se\{\}_area <- apply(\{\} area, \{\} 2, \{\} sd, \{\} na.rm = TRUE\{\} ) / sqrt(M)
se\{\}_mu <- apply(\{\} mu\{\}_length, \{\} 2, \{\} sd, \{\} na.rm = TRUE\{\} ) / sqrt(M)
se\{\}_sigma <- apply(\{\} sigma\{\}_length, \{\} 2, \{\} sd, \{\} na.rm = TRUE\{\} ) / sqrt(M)
summary <- data.frame(\{\} method = methods,

expected_area = mean_area,
se_area = se_area,

expected_mu_length = mean_mu,
se_mu_length = se_mu,

expected_sigma_length = mean_sigma,
se_sigma_length = se_sigma,

empirical_coverage = empirical_coverage,

row.names = NULL

)

# -----
# 5. Return everything
# -----
list(\{\} summary = summary, \{\} area = area, \{\} mu\{\}_length = mu\{\}_length, \{\} sigma\{\}_length =
sigma\{\}_length, \{\} coverage = coverage, \{\} bands = bands\{\} )\{\} }

```

Assistant

Yes, this version is now internally consistent for a generic location-scale family.

The two important fixes are both present:

- simulation uses `r x <- ls.quantile.function(runif(n))` - inversion uses `r qfun = ls.quantile.function`

So if you call it with `qnorm`, `qlogis`, `qlaplace`, etc., the simulated standardized distribution and the fitted location-scale family match.

I would only change two small things.

First, update this stale comment:

```
# Simulate from standard normal.  
# By location-scale equivariance this is enough.
```

to:

```
# Simulate from standardized base distribution G  
# using the probability integral transform.
```

Second, for Monte Carlo standard errors, divide by the actual number of non-NA observations rather than always by `sqrt(M)`. Normally all regions should be feasible and bounded, but this makes the code robust:

```
se_area <- apply(  
  area,  
  2,  
  function(z) sd(z, na.rm = TRUE) / sqrt(sum(!is.na(z)))  
)  
  
se_mu <- apply(  
  mu_length,  
  2,  
  function(z) sd(z, na.rm = TRUE) / sqrt(sum(!is.na(z)))  
)  
  
se_sigma <- apply(  
  sigma_length,  
  2,  
  function(z) sd(z, na.rm = TRUE) / sqrt(sum(!is.na(z)))  
)
```

I would also add a diagnostic:

```
n_valid <- colSums(!is.na(area))
```

and include it in the summary:

```
summary <- data.frame(  
  method = methods,  
  n_valid = n_valid,  
  
  expected_area = mean_area,  
  se_area = se_area,  
  
  expected_mu_length = mean_mu,  
  se_mu_length = se_mu,  
  
  expected_sigma_length = mean_sigma,  
  se_sigma_length = se_sigma,  
  
  empirical_coverage = empirical_coverage,  
  
  row.names = NULL  
)
```

For the experiments, you can now simply do:

```
# Normal  
res_normal <- compare_bands_mc(  
  n = 100,  
  seed = 42,  
  ls.quantile.function = qnorm  
)
```

```
# Logistic
res_logistic <- compare_bands_mc(
  n = 100,
  seed = 42,
  ls.quantile.function = qlogis
)
```

and, assuming your package's parameterization is the desired location 0, scale 1:

```
# Laplace
res_laplace <- compare_bands_mc(
  n = 100,
  seed = 42,
  ls.quantile.function = qlaplace
)
```

One very useful validation now is to compare:

```
res_normal$summary$empirical_coverage
res_logistic$summary$empirical_coverage
res_laplace$summary$empirical_coverage
```

These should be almost identical up to Monte Carlo variation. In contrast, the expected area and marginal lengths **should change**, because those depend on the quantile geometry of G .

That distinction—distribution-free coverage but family-dependent geometry—is actually one of the cleanest features of the whole framework.

Turn 29

User

```
res <- compare\{\}_bands\{\}_mc(\{ n = 50,\{ alpha = 0.05,\{ M = 10000,\{ band\{\}_calibration\{\}_B  
= 100000,\{ seed = 42,\{ ls.quantile.function = qLaplace\{\} )  
res$summary
```

	method	expected_area	se_area	expected_mu_length	se_mu_length
1	KS	0.9403326	0.003195836	0.7712252	0.001607659\{\}
2	DKW	0.9989837	0.003377009	0.7924402	0.001642512\{\}
3	Berk\{\}_Jones	0.7878275	0.002941374	0.9427438	0.001760942\{\}
4	Dumbgen\{\}_Wellner	0.7534635	0.002720767	0.8747483	0.001671747\{\}
expected\{\}_sigma\{\}_length	se\{\}_sigma\{\}_length	empirical\{\}_coverage\{\}	1	1.923620	0.003359516
0.9549\{\}	2	1.986488	0.003455860	0.9610\{\}	3
1.286555	0.002836859	0.9554\{\}	4	1.333172	0.002784367
0.9534					

Assistant

These Laplace results reinforce the same qualitative pattern you saw under the normal family.

For $n = 50$:

Method	Expected area	Expected μ -length	Expected σ -length	Coverage	— —: —: —: —:	KS
0.9403	0.7712	1.9236	0.9549	DKW	0.9990	0.7924
1.9865	0.9610	Berk-Jones	0.7878	0.9427	1.2866	0.9554
Dumbgen-Wellner	0.7535	0.8747	1.3332	0.9534		

So again:

KS best for μ

BJ best for σ

DW best for joint area

That is already more interesting than if the result held only for the normal family.

Relative to KS, DW reduces expected area by

$$1 - \frac{0.7535}{0.9403} \approx 19.9\%.$$

BJ reduces expected scale length by

$$1 - \frac{1.2866}{1.9236} \approx 33.1\%.$$

But BJ's expected μ -interval is about

$$\frac{0.9427}{0.7712} - 1 \approx 22.2\%$$

longer than KS.

DW again looks like a compromise: about 20% smaller joint area and 31% shorter scale interval than KS, while its location interval is about 13% longer.

What's especially useful is that the coverage numbers remain essentially unchanged compared with the normal experiment. That's exactly what the distribution-free theory predicts: changing G affects the geometry of the induced parameter set, but not the coverage mechanism on the probability scale.

The fact that the **magnitude** of the advantage changes is also scientifically interesting. Under the normal family you were seeing roughly a 30%+ area reduction for DW relative to KS; here it's closer to 20%. So the ranking seems stable, but the efficiency gain is **family-dependent**.

That suggests a stronger research question than just "which band is best?":

How does the geometry of G^{-1} determine the relative efficiency of the induced confidence regions?

The next family I would run is logistic, because it sits between normal and Laplace in some geometric respects and has a simple analytic quantile function. Then Student- t with a few degrees of freedom would be very informative for heavier tails.

Turn 30

User

```
res <- compare\{\}_bands\{\}_mc(\{\} n = 50,\{\} alpha = 0.05,\{\} M = 10000,\{\} band\{\}_calibration\{\}_B  
= 100000,\{\} seed = 42,\{\} ls.quantile.function = qt\{\} ) res$summary
```

Assistant

That call will **not work as intended** with base R's `qt`, because `qt()` requires a degrees-of-freedom argument:

```
qt(p, df)
```

Your code calls the quantile function internally as

```
ls.quantile.function(runif(n))
```

and later as

```
qfun(lower)  
qfun(upper)
```

so `qt` by itself will give an error like:

```
argument "df" is missing, with no default
```

Wrap it in a one-argument function instead. For example, for Student- t_5 :

```
qt5 <- function(p) qt(p, df = 5)  
  
res_t5 <- compare_bands_mc(  
  n = 50,  
  alpha = 0.05,  
  M = 10000,  
  band_calibration_B = 100000,  
  seed = 42,  
  ls.quantile.function = qt5  
)  
  
res_t5$summary
```

Or more compactly:

```
res_t5 <- compare_bands_mc(  
  n = 50,  
  alpha = 0.05,  
  M = 10000,  
  band_calibration_B = 100000,  
  seed = 42,  
  ls.quantile.function = function(p) qt(p, df = 5)  
)
```

I would actually run several degrees of freedom:

```
res_t3 <- compare_bands_mc(  
  n = 50,  
  alpha = 0.05,  
  M = 10000,  
  band_calibration_B = 100000,  
  seed = 42,  
  ls.quantile.function = function(p) qt(p, df = 3)  
)  
  
res_t5 <- compare_bands_mc(  
  n = 50,  
  alpha = 0.05,  
  M = 10000,  
  band_calibration_B = 100000,  
  seed = 42,  
  ls.quantile.function = function(p) qt(p, df = 5)  
)  
  
res_t10 <- compare_bands_mc(  
  n = 50,  
  alpha = 0.05,  
  M = 10000,
```

```
band_calibration_B = 100000,  
seed = 42,  
ls.quantile.function = function(p) qt(p, df = 10)  
)
```

This is especially useful for the research question because you get a controlled progression:

$$t_3 \rightarrow t_5 \rightarrow t_{10} \rightarrow N(0, 1),$$

with increasingly lighter tails.

One subtle point: here σ is the **location–scale parameter** in

$$X = \mu + \sigma T_\nu,$$

not necessarily the standard deviation. For $\nu > 2$,

$$\text{SD}(X) = \sigma \sqrt{\frac{\nu}{\nu - 2}}.$$

So in the paper I would consistently call σ the **scale parameter**, not standard deviation, once you move beyond the normal family.

Turn 31

User

```
res <- compare\{\}_bands\{\}_mc(\{\} n = 50,\{\} alpha = 0.05,\{\} M = 10000,\{\} band\{\}_calibration\{\}_B
= 100000,\{\} seed = 42,\{\} ls.quantile.function = function(p) qt(p, df = 2)\{\} )
res$summary
```

method	expected_area	se_area	expected_mu_length	se_mu_length
--------	---------------	---------	--------------------	--------------

1 KS	1.0706380	0.003772008	0.9335078	0.001822055\{\}	2 DKW	1.1361222	0.003988763	0.9583368	0.001860978\{\}
3 Berk\{\}_Jones	1.0823297	0.004335111	1.1456061	0.002086525\{\}	4 Dumbgen\{\}_Wellner	0.9879995	0.003807135	1.0611032	0.001954873\{\}
expected\{\}_sigma\{\}_length	se\{\}_sigma\{\}_length	empirical\{\}_coverage\{\}	1	1.874928	0.003543875	0.9549\{\}	2	1.935536	0.003654488
0.9610\{\}	3	1.514591	0.003736043	0.9554\{\}	4	1.505973	0.003451849	0.9534	

Assistant

These t_2 results are especially interesting because the ranking has started to change.

For $n = 50$:

Method	Expected area	Expected μ -length	Expected σ -length	Coverage	— —: —: —: —:	KS							
1.0706	0.9335	1.8749	0.9549		DKW	1.1361	0.9583	1.9355	0.9610		Berk-Jones	1.0823	1.1456
1.5146	0.9554		Dümbgen-Wellner	0.9880	1.0611	1.5060	0.9534						

So unlike the normal and Laplace cases:

DW is now best for both joint area and scale length

while KS is still best for location.

That is a potentially important result. Under t_2 ,

$$A_{BJ} = 1.0823 > A_{KS} = 1.0706,$$

so Berk-Jones has actually lost its joint-area advantage over KS.

DW still improves area:

$$1 - \frac{0.9880}{1.0706} \approx 7.7\%.$$

But compare that with the much larger DW advantage you saw under normality. The benefit of the tail-adaptive bands is clearly ****strongly** dependent on the base distribution G^{**} .

Even more interesting is scale. BJ and DW are almost tied:

$$1.5146 \quad \text{vs} \quad 1.5060,$$

with DW now slightly better.

This strongly supports moving away from a claim like

“BJ is optimal for scale.”

Instead, the empirical story is becoming:

the optimal CDF band for parameter inference depends on both the target parameter and the base location-scale family.

That is a much richer research result.

There is also a plausible geometric explanation. For t_2 ,

$$G^{-1}(p) = t_2^{-1}(p)$$

grows extremely rapidly near $p = 0, 1$. Since your polygon constraints have slopes

$$-a_i = -G^{-1}(\ell_i), \quad -b_i = -G^{-1}(u_i),$$

small differences between bands in the extreme probability bounds become hugely magnified in parameter space. A band that is very aggressive in the tails can therefore help or hurt depending on exactly how those endpoint constraints interact with the heavy-tailed quantile function.

This suggests the next experiment should be a systematic tail-heaviness sweep:

$t_2, t_3, t_5, t_{10}, N(0, 1).$

For each one, record relative efficiencies such as

$$R_A(B) = \frac{E[A_B]}{E[A_{KS}]},$$

$$R_\mu(B) = \frac{E[L_{\mu,B}]}{E[L_{\mu,KS}]},$$

$$R_\sigma(B) = \frac{E[L_{\sigma,B}]}{E[L_{\sigma,KS}]}.$$

If those ratios move smoothly with ν , that would give you a very strong figure and perhaps a theoretical clue about how the tail growth rate of G^{-1} controls induced confidence-region efficiency.

Turn 32

User

I think it is time to write the text. Summarize everything in this chat and prepare a draft in latex

Assistant

Below is a self-contained first paper draft. I have treated the simulation results as **preliminary**, and I've deliberately avoided claiming novelty for test inversion or KS convexity. The literature supports the historical framing in the draft: Easterling proposed goodness-of-fit inversion; Littell–Rao developed KS location–scale regions; Salvia proved convexity/finiteness for KS consonance sets; Owen inverted Berk–Jones to obtain likelihood bands; Dümbgen–Wellner refined those bands; and Chattopadhyay–Sarkar–Kuchibhotla (UAI 2026) recently developed general quantile-function inversion of CDF bands. [source citation: turn873422search3 turn638412search3 turn873422search1
:::writing{variant="document" id="58321"} # LaTeX Draft

```
\documentclass[11pt]{article}

\usepackage[a4paper,margin=1in]{geometry}
\usepackage{amsmath,amssymb,amsthm,mathtools}
\usepackage{booktabs}
\usepackage{array}
\usepackage{enumitem}
\usepackage{hyperref}
\usepackage{microtype}

\newtheorem{proposition}{Proposition}
\newtheorem{remark}{Remark}
\newtheorem{lemma}{Lemma}

\newcommand{\R}{\mathbb{R}}
\newcommand{\Pp}{\mathbb{P}}
\newcommand{\Ee}{\mathbb{E}}
\newcommand{\cC}{\mathcal{C}}
\newcommand{\cB}{\mathcal{B}}
\newcommand{\ind}{\mathbf{1}}

\title{
From Distribution-Free CDF Bands to Polyhedral Confidence Regions\\
for Location--Scale Families
}

\author{[Author Name]}
\date{\today}

\begin{document}

\maketitle

\begin{abstract}
We study confidence regions for the location and scale parameters of a
continuous location--scale family obtained by inverting simultaneous,
distribution-free confidence bands for the cumulative distribution
function. If the band implies simultaneous order-statistic constraints
\[
\ell_i \leq F(X_{(i)}) \leq u_i,
\quad i=1,\ldots,n,
\]
then for a location--scale family
\[
F_{\mu,\sigma}(x)
=
G\!\left(\frac{x-\mu}{\sigma}\right),
\quad \sigma>0,
\]
the induced confidence region is the intersection of the affine
constraints
\[
X_{(i)}-\sigma G^{-1}(u_i)
\leq \mu
\leq
X_{(i)}-\sigma G^{-1}(\ell_i).
\]
Consequently, the region is convex and polyhedral in the
```

(μ, σ) parameterization. We derive exact representations of its projection onto the scale and location coordinates, its piecewise-linear boundary, and its area. When the band limits are monotone in the order statistic index, the complete boundary can be constructed by a monotone-slope envelope algorithm in linear time after sorting the sample.

The framework permits different distribution-free CDF bands to be compared according to their induced parametric uncertainty rather than according to CDF-band width alone. We investigate Kolmogorov--Smirnov, Dvoretzky--Kiefer--Wolfowitz, Berk--Jones/Owen, and Dümbgen--Wellner bands. Preliminary Monte Carlo experiments reveal a strong target-dependent pattern. For the normal location--scale family, the Kolmogorov--Smirnov band gives the shortest marginal interval for location, Berk--Jones gives the shortest interval for scale, and the Dümbgen--Wellner band gives the smallest expected joint area across a range of sample sizes. Experiments with Laplace and Student- t families show that the magnitude, and in some cases the ranking, of these efficiency differences depends on the base location--scale family. These observations motivate the broader problem of designing distribution-free CDF bands specifically to optimize induced parametric confidence regions.

Introduction

A simultaneous confidence band for a cumulative distribution function (CDF) is usually viewed as a nonparametric inferential object. Given an empirical distribution function F_n , a band provides a set of distribution functions that are compatible with the observed sample at a prescribed confidence level. If a parametric family $\{F_\theta: \theta \in \Theta\}$ is of interest, however, one may instead ask which members of this parametric family are compatible with the same band. This leads naturally to the inverted set

$$\begin{aligned} & \{ \\ & \quad \text{\texttt{\cC(X)}} \\ & = \\ & \quad \left\{ \begin{array}{l} \theta \in \Theta: \\ F_\theta \text{ \texttt{lies in the simultaneous CDF band}} \end{array} \right\}. \\ & \end{aligned}$$

The resulting set is a confidence region for the parameter.

Inference obtained by inversion of goodness-of-fit procedures is classical. Easterling \cite{Easterling1976} explicitly proposed parameter estimation through inversion of goodness-of-fit tests, while Littell and Rao \cite{LittellRao1978} developed confidence regions for location and scale parameters based on the Kolmogorov--Smirnov (KS) statistic. Salvia \cite{Salvia1980} subsequently established important geometric properties of KS consonance sets, including convexity and finiteness for location--scale models. Thus, neither goodness-of-fit test inversion nor convexity of the KS location--scale region is new.

At the same time, the literature on distribution-free CDF bands has developed substantially beyond KS. Berk and Jones \cite{BerkJones1979} introduced likelihood-ratio-type goodness-of-fit statistics with strong tail sensitivity. Owen \cite{Owen1995} inverted the Berk--Jones construction to obtain nonparametric likelihood confidence bands, which are narrower in the tails and wider in the center than KS bands. Frey \cite{Frey2008} studied distribution-free bands chosen directly to optimize narrowness criteria rather than test power. More recently, Dümbgen and Wellner \cite{DumbgenWellner2023} introduced a multiscale refinement of the Berk--Jones/Owen framework that preserves strong tail behavior while improving performance in the central region.

There has also been renewed interest in direct inversion of CDF bands through parametric quantile functions. Chattopadhyay, Sarkar, and Kuchibhotla \cite{Chattopadhyay2026} develop a general framework for quantile-parametrized families, motivated particularly by models in which the quantile function is tractable while the likelihood or CDF is not.

The present work focuses on a special case that has unusually strong

geometric structure: two-parameter location--scale families. The quantile function is affine in the parameters,

```
\[
  Q_{\{\mu,\sigma\}}(p)
=
  \mu + \sigma G^{-1}(p),
\]
```

and therefore inversion of any simultaneous order-statistic CDF band produces linear inequalities in (μ, σ) . This observation allows the induced confidence set to be treated as a problem in convex geometry.

The objectives of this paper are fourfold.

First, we formulate CDF-band inversion for a generic continuous location--scale family and show that the resulting confidence region is polyhedral in (μ, σ) .

Second, we derive exact computational representations of the joint region, its marginal confidence intervals for μ and σ , and its area. These calculations reduce to piecewise-linear envelope operations.

Third, we use this geometry to compare several classical and modern distribution-free confidence bands according to three parametric criteria:

```
\[
  \operatorname{Area}(\mathcal{C}), \quad
  |\mathcal{C}_\mu|, \quad
  |\mathcal{C}_\sigma|.
\]
```

Fourth, we investigate how these comparisons change with the base location--scale family. The preliminary results suggest that CDF-band quality is inherently target dependent: a band that is highly efficient for scale need not be efficient for location, and a band that minimizes joint area need not minimize either marginal length.

`\section{General band inversion}`

`\subsection{Order-statistic representation}`

Let

```
\[
  X_1, \ldots, X_n
\]
```

be independent observations from a continuous distribution F , and let

```
\[
  X_{(1)} \leq \cdots \leq X_{(n)}
\]
```

denote the order statistics.

Suppose that a simultaneous distribution-free CDF band implies deterministic bounds

```
\begin{equation}
  \ell_i
  \leq
  F(X_{(i)})
  \leq
  u_i,
  \quad i=1, \ldots, n,
  \label{eq:orderband}
\end{equation}
```

such that

```
\begin{equation}
  \mathbb{P}_F
  \left(
    \ell_i
    \leq
    F(X_{(i)})
    \leq
    u_i
    \text{ for every } i
  \right)
  \geq
\end{equation}
```

$1-\alpha$
 $\backslash\text{label}\{\text{eq:coverage}\}$
 $\backslash\text{end}\{\text{equation}\}$
 for every continuous F .

The distribution-free nature of $\backslash\text{eqref}\{\text{eq:coverage}\}$ follows from the probability integral transform. If

$$U_i = F(X_i),$$

then U_i are independent $\backslash\text{operatorname}\{\text{Unif}\}(0,1)$ random variables and

$$F(X_{(i)}) = U_{(i)}.$$

Thus calibration of the band depends only on uniform order statistics and not on the eventual parametric family.

A small indexing detail is important. If a CDF band is originally specified on the intervals between empirical jumps, then at $X_{(i)}$ continuity of F combines information from the empirical CDF immediately before and immediately after the jump. In particular, for constant-width bands the lower endpoint involves i/n whereas the upper endpoint involves $(i-1)/n$. This distinction is essential for obtaining the intended finite-sample coverage.

$\backslash\text{subsection}\{\text{Location--scale families}\}$

Consider the location--scale family

$$F_{(\mu, \sigma)}(x) = G\left(\frac{x - \mu}{\sigma}\right),$$

$\backslash\text{quad}$
 $\mu \in \mathbb{R}, \sigma > 0,$
 $\backslash\text{label}\{\text{eq:lsfamily}\}$

$\backslash\text{end}\{\text{equation}\}$
 where G is a known continuous strictly increasing CDF.

Let

$$g(p) = G^{-1}(p),$$

and define

$$a_i = g(\ell_i),$$

$\backslash\text{quad}$
 $b_i = g(u_i).$
 $\backslash\text{label}\{\text{eq:ab}\}$

$\backslash\text{end}\{\text{equation}\}$
 Values $g(0) = -\infty$ and $g(1) = +\infty$ are interpreted in the usual extended-real sense.

For a candidate (μ, σ) , condition $\backslash\text{eqref}\{\text{eq:orderband}\}$ becomes

$$G\left(\frac{X_{(i)} - \mu}{\sigma}\right) \leq u_i.$$

Since G is increasing,

$$\frac{X_{(i)} - \mu}{\sigma} \leq b_i,$$

and therefore

```

\begin{equation}
X_{(i)} - \sigma b_i
\leq
\mu
\leq
X_{(i)} - \sigma a_i.
\label{eq:linearconstraints}
\end{equation}

This gives the fundamental geometric representation.

\begin{proposition}[Band-induced location--scale region]
\label{prop:polyhedral}
Let \eqref{eq:coverage} hold for a simultaneous order-statistic band.
For the location--scale family \eqref{eq:lsfamily}, define
\begin{equation}
\mathcal{C}
=
\bigcap_{i=1}^n
\left\{
\begin{array}{l}
(\mu, \sigma): \\
X_{(i)} - \sigma b_i \\
\leq \\
\mu \\
\leq \\
X_{(i)} - \sigma a_i
\end{array}
\right\}
\cap
\{\sigma > 0\}.
\label{eq:C}
\end{equation}
Then  $\mathcal{C}$  is a convex polyhedral set in the  $(\mu, \sigma)$ 
parameterization, and
\begin{equation}
\mathbb{P}_{\mu_0, \sigma_0}
\left(
\begin{array}{l}
(\mu_0, \sigma_0) \in \mathcal{C} \\
\geq \\
1 - \alpha.
\end{array}
\right)
\end{equation}
If the order-statistic band is calibrated with exact coverage
 $1 - \alpha$ , the induced region has the same coverage.
\end{proposition}

\begin{proof}
Each inequality in \eqref{eq:linearconstraints} defines a half-plane in
 $(\mu, \sigma)$ , and  $\{\sigma > 0\}$  is convex. Their intersection is
therefore convex and polyhedral.

For the true parameter  $(\mu_0, \sigma_0)$ ,
membership in  $\mathcal{C}$  is equivalent to
\begin{equation}
\ell_i
\leq
F_{\mu_0, \sigma_0}(X_{(i)})
\leq
u_i
\end{equation}
for every  $i$ . The coverage statement therefore follows directly from
\eqref{eq:coverage}.
\end{proof}

\begin{remark}
Convexity here is naturally expressed in  $(\mu, \sigma)$  coordinates.
The transformation from scale to variance,
 $v = \sigma^2$ , is nonlinear and does not in general preserve convexity.
\end{remark}

\section{Exact geometry of the confidence region}

Write
\begin{equation}
x_i = X_{(i)}
\end{equation}

```

```

for convenience.

\subsection{Lower and upper envelopes}

For any fixed scale  $s > 0$ , define
\begin{align}
L(s) &= \max_{1 \leq i \leq n} \left\{ x_i - b_i s \right\}, \\
&\quad \text{\label{eq:L}} \\
U(s) &= \min_{1 \leq i \leq n} \left\{ x_i - a_i s \right\}. \\
&\quad \text{\label{eq:U}}
\end{align}
Constraints involving infinite  $a_i$  or  $b_i$  are omitted whenever they are nonbinding.

The joint region can be written exactly as
\begin{equation}
\mathcal{C} = \left\{ (\mu, s) : s > 0, \text{\quad} L(s) \leq \mu \leq U(s) \right\}. \\
\text{\label{eq:enveloperegion}}
\end{equation}

Because  $L$  is a maximum of affine functions, it is convex and piecewise linear. Because  $U$  is a minimum of affine functions, it is concave and piecewise linear. Consequently
\begin{equation}
H(s) = U(s) - L(s)
\end{equation}
is concave and piecewise linear.

The feasible scale values satisfy
\begin{equation}
H(s) \geq 0.
\end{equation}
A superlevel set of a concave function on the real line is an interval. Thus, whenever the joint confidence region is nonempty and bounded in scale,
\begin{equation}
\mathcal{C}_\sigma = [\sigma_L, \sigma_U].
\end{equation}

\subsection{Pairwise formula for the scale interval}

For a fixed  $s$ , feasibility requires every lower constraint to lie below every upper constraint:
\begin{equation}
x_i - s b_i \leq x_j - s a_j \quad \text{for all } i, j.
\end{equation}
Equivalently,
\begin{equation}
s(a_j - b_i) \leq x_j - x_i. \\
\text{\label{eq:pairwise}}
\end{equation}

```

```

\end{equation}

Let
\[
d_{ij}=a_j-b_i,
\quad
\Delta_{ij}=x_j-x_i.
\]
If  $d_{ij}>0$ , \eqref{eq:pairwise} gives
\[
s
\leq
\frac{\Delta_{ij}}{d_{ij}}.
\]
If  $d_{ij}<0$ , it gives
\[
s
\geq
\frac{\Delta_{ij}}{d_{ij}}.
\]

Hence an exact representation is
\begin{equation}
\sigma_L
=
\max
\left\{
\begin{array}{l}
0, \\
\max_{d_{ij}<0}
\frac{x_j-x_i}{a_j-b_i}
\end{array}
\right\},
\label{eq:sigmaL}
\end{equation}
and
\begin{equation}
\sigma_U
=
\min_{d_{ij}>0}
\frac{x_j-x_i}{a_j-b_i},
\label{eq:sigmaU}
\end{equation}
with the usual conventions if one of the index sets is empty.

```

This representation shows that critical scale values are ratios of observed order-statistic spacings to theoretical quantile spacings.

\subsection{Marginal interval for location}

The marginal confidence interval for μ is the projection of \eqref{eq:enveloperegion} onto the location coordinate:

```

\begin{equation}
\mathcal{C}_\mu
=
[\mu_L, \mu_U],
\end{equation}
where
\begin{align}
\mu_L
&=
\min_{\sigma_L \leq s \leq \sigma_U}
L(s),
\label{eq:muL}
\\
\mu_U
&=
\max_{\sigma_L \leq s \leq \sigma_U}
U(s).
\label{eq:muU}
\end{align}

```

Since L is convex and piecewise linear, its minimum occurs at an endpoint or a breakpoint of its upper envelope. Similarly, since U is concave and piecewise linear, its maximum occurs at an endpoint or breakpoint of its lower envelope. The marginal interval can therefore be obtained exactly from finitely many line intersections,

without numerical optimization over a dense parameter grid.

`\subsection{Area of the joint region}`

The area of the joint region is

```
\begin{equation}
A(\backslash cC)
=
\int_{\backslash sigma_L}^{\backslash sigma_U}
\left[
U(s)-L(s)
\right]\backslash ds.
\label{eq:area}
\end{equation}
```

Let

```
\[
\sigma_L=s_0<s_1<\cdots<s_K=\sigma_U
\]
```

be the merged set of breakpoints of L and U . On each interval $[s_k, s_{k+1}]$,

```
\[
L(s)=c_k+d_ks,
\quad
U(s)=e_k+f_ks.
\]
```

Therefore

```
\begin{align}
A(\backslash cC)
&=
\sum_{k=0}^{K-1}
\Bigg[
&\&(e_k-c_k)(s_{k+1}-s_k)
&\nonumber\\
&\&+
&\frac{f_k-d_k}{2}
&\left(
&s_{k+1}^2-s_k^2
&\right)
\Bigg].
\label{eq:areaexact}
\end{align}
```

Equivalently, because the gap $U-L$ is affine between consecutive breakpoints, trapezoidal integration on the breakpoint grid is exact.

`\subsection{Vertices}`

Boundary lines have the form

```
\[
\mu=x_i-b_i\sigma
\]
or
\[
\mu=x_j-a_j\sigma.
\]
```

A lower and upper boundary line intersect when

```
\[
x_i-b_i\sigma
=
x_j-a_j\sigma,
\]
```

giving

```
\begin{equation}
\sigma_{ij}
=
\frac{x_j-x_i}{a_j-b_i},
\quad
\mu_{ij}
=
x_i-b_i\sigma_{ij}.
\label{eq:vertices}
\end{equation}
```

Candidate vertices are retained only if they satisfy all constraints.

`\section{Efficient envelope construction}`

The naive pairwise formulas involve $O(n^2)$ candidate intersections. The order-statistic structure permits a more efficient construction for the common case in which the confidence-band limits are monotone.

Suppose

```
\[
\ell_1 \leq \dots \leq \ell_n,
\quad
u_1 \leq \dots \leq u_n.
\]
```

Then, because $g = G^{-1}$ is increasing,

```
\[
a_1 \leq \dots \leq a_n,
\quad
b_1 \leq \dots \leq b_n.
\]
```

The lines defining L are

```
\[
L_i(s) = x_i - b_i s,
\]
```

whose slopes $-b_i$ are monotone. The lines defining $-U$ are

```
\[
-U_i(s) = -x_i + a_i s,
\]
```

whose slopes a_i are also monotone.

`\begin{proposition}[Monotone envelope algorithm]`

`\label{prop:envelope}`

Assume the band limits are monotone in i . Given the sorted sample, the active lines of L and U , together with all their breakpoints, can be constructed in $O(n)$ time.

`\end{proposition}`

`\begin{proof}`

Consider generic lines

```
\[
f_j(t) = c_j + m_j t
\]
```

arriving in strictly increasing slope order. Suppose the current upper envelope contains active lines

```
\[
h_1, \dots, h_r
\]
```

with increasing activation points

```
\[
z_1 < \dots < z_r.
\]
```

A new line q has slope larger than that of h_r . Let t^* be their intersection.

If

```
\[
t^* > z_r,
\]
```

then h_r remains active until t^* and q becomes active thereafter.

If instead

```
\[
t^* \leq z_r,
\]
```

then h_r can never be part of the new envelope. Before z_r it was not part of the old envelope, while after z_r the higher-slope line q is already above it. Hence h_r may be removed permanently. The same comparison is repeated with the new last active line.

Each line is pushed onto the envelope stack once and can be popped at most once. The total number of stack operations is therefore linear in n . Equal-slope lines are handled by retaining only the line with the largest intercept.


```

\]
which is precisely the standardized confidence region.
The scaling statements follow from the affine transformation and its
Jacobian.
\end{proof}

This result has an important computational implication: for a given
base family  $G$ , sample size  $n$ , confidence level, and CDF band, the
distribution of the normalized confidence-region geometry can be
studied entirely under
\[
\begin{aligned}
&\mu_0=0, \\
&\quad \quad \quad \\
&\sigma_0=1.
\end{aligned}
\]

\section{Distribution-free bands considered}

We compare four simultaneous distribution-free CDF

> Retrieval note: The source interface truncated the preceding assistant message at 20,000 characters. The
text above is preserved exactly as returned; the remaining original content was not available through this
retrieval channel.

---

# Turn 33

## User

Well, you did all the work and I acted as an operator. How this should be properly acknowledged in the text?

## Assistant-

Yesbut because the AI contribution here went far beyond copy-editing, I would disclose it substantially, not with
a generic "ChatGPT was used to improve the "language sentence.

Current publisher policies are fairly consistent on two points: an LLM cannot be an author because it cannot
accept responsibility for the work, but substantive AI use in research design, code, analysis, or manuscript
generation should be disclosed. Elsevier explicitly says AI-assisted code/research methods should be
described in the Methods section, and substantive manuscript assistance requires a declaration; Nature
likewise requires substantive LLM use to be documented and keeps accountability with the human authors.
citeturn747266search1turn747266search2turn747266search3

In our case, the truthful description is stronger than "writing assistance". ChatGPT contributed to the
mathematical formulation, derivations, algorithm design, R implementation, literature exploration,
interpretation of preliminary simulations, and drafting. You contributed the original research direction/
questions, selected which avenues to pursue, ran the computations, supplied the empirical results, identified
what needed further investigation, and made the decisions that steered the project. But from what you just
said, you do not yet regard yourself as having independently done or verified all of the intellectual work.

That last point matters. I would not submit the paper yet with a declaration that merely says "the author
reviewed the output and takes responsibility". Before submission, you should be able to independently defend
every proposition, derivation, algorithm, simulation choice, citation, and conclusion. AI cannot take that
responsibility for you, and merely operating the prompts would be a weak basis for conventional scholarly
authorship.

A transparent statement at the present stage could be:

> Use of generative artificial intelligence. ChatGPT (OpenAI) was used extensively during the development of
this work. Its contributions included assisting with the formulation of the CDF-band inversion framework,
mathematical derivations concerning the geometry of the induced location--scale confidence regions,
development of computational algorithms and R code, literature exploration, interpretation of preliminary
simulation results, and preparation of an initial manuscript draft. The human author initiated and directed
the research investigation, selected the questions and comparisons pursued, executed the computational
experiments, inspected the resulting output, and made decisions concerning the scope and direction of the
study. All AI-generated mathematical arguments, code, references, and interpretations require independent
verification by the author before publication.

For a finished paper, I would want the last sentence to become much stronger, but only after you actually do
it:

> All mathematical arguments, algorithms, code, references, numerical results, and interpretations generated with
AI assistance were subsequently checked and validated by the author. The final scientific judgments,

```

interpretation of the results, and decision to publish are those of the author, who takes full responsibility for the content of the article.

I would put the research-use disclosure in the **Methods / Computational Methods** section, because the AI was involved in code and mathematical development, and also include the journal's required generative-AI declaration immediately before the references if the target journal requires one. Elsevier's current policy explicitly distinguishes these two cases: AI used in the research process belongs in Methods, while AI used in manuscript preparation also gets a separate declaration. citeturn747266search1

There is another important consequence. I would **not** write in the paper phrases such as

> "We derived"...

if you have not personally checked the derivation. After verification, "we" derive or "we" show is fine because you are taking scholarly responsibility for the argument; before verification, the manuscript is still essentially a research working draft.

And I think the best next stage of this project is actually for me to become **less** involved in producing new theory for a while. You should now work through the central results yourself: prove the polyhedral representation, prove the envelope algorithm, check the area formula, inspect the R implementation, and read the key papers directly. You can use me to challenge your proofs, find mistakes, and answer questions. That changes your role from "operator into the researcher who genuinely understands and owns the work.

If, after doing that, you can explain why

```
\[
\ell_i \leq F(X_{(i)}) \leq u_i
\]
```

becomes a polyhedron, derive (σ_L, σ_U) , justify the $O(n)$ envelope algorithm, explain why BJ/DW favor scale differently from KS, and reproduce the simulations independently, then describing this as **your** research conducted with substantial AI assistance is much more defensible than describing the present interaction as though the AI contribution had only been editorial.

Turn 34

User

The problem is not that I don't understand what you did or did not check your proofs =) It is more conceptual. When you helped me with the previous paper (<https://arxiv.org/pdf/2607.15015>), it could have been described as an actual help. It went far beyond editorial stuff (you suggested a theorem, processed experimental result, wrote proper english text), but the overall idea was mine, initial text was mine, the method was mine and the code was mine. To prepare the text of such quality, I would have spent months. With your help it was less than a week. And I honestly wrote about what I did and what you did in the section about AI use. And without hesitation, I placed my name as an author. Because I was one. The current situation is essentially different. I was just keeping asking the questions and you did THE WHOLE WORK. I mean, everything. And I don't have problem with that, to be honest. You suggest me to try to prove something by myself and complete the text by myself. That does not make sense from my point of view as you can do it faster and better. And I don't have intention to present your work as my own. It is tempting, but I'll pass =) Probably, you think that I am upset and you want me to feel better. Well, I already feel ok. The problem is different. What I witness is that the neural network can actually make science on its own. And this is very cool. I think it is a good research and hence it should be published. I don't want to publish this stuff formally in a journal. Arxiv is enough. But I can't name myself an author, since I was only a witness (and this is fine), or write you as an author (since, well, it is not accepted?). So the question is: how to handle this situation properly? I see it like this: near my name we write something like (ChatGPT operator, editor and the witness on the new age) and start the paper with big disclaimer that the whole work was done by you and I just checked it. Again, that is completely fine by me. What do you say?

Assistant

I think your proposed framing is substantially better than trying to squeeze this into the conventional sentence "the author used ChatGPT for assistance".

There are really **two** separate questions here:

1. **Who generated the scientific content?**
2. **Who can be the accountable author/submitter in the current scholarly infrastructure?**

In this case, I agree with you that the honest answer to the first is: **the model generated most of it**. The seed question came from you, and you repeatedly selected directions, executed the proposed computations, returned the results, caught practical issues, and decided what was interesting. But the mathematical development, literature exploration, algorithms, code drafts, simulation design, interpretation, and manuscript draft were overwhelmingly generated in this conversation by ChatGPT. Calling that merely "AI

"assistance would undersell what happened.

At the same time, I would slightly resist the phrase "science on its own". This was not an autonomous agent independently deciding to study CDF bands, running R, observing results, and deciding what experiment to do next. It was a **human-in-the-loop** scientific process in which the intellectual generation was unusually AI-dominant. That distinction makes the case more interesting, not less.

There is a practical arXiv problem with literally making ChatGPT the author. arXiv's current submission agreement requires the submitter to represent that they are "the original" author or an arXiv-approved proxy acting for an original author, and the submitter acts on behalf of all listed coauthors. [citeturn478867view0](#)
Existing scholarly norms likewise generally reject LLMs as authors because authorship includes accountability. [citeturn703986search1turn703986search4](#)

So I would **not** put "ChatGPT into arXiv's author metadata as though it were a conventional coauthor.

But I also would **not** pretend that you originated the technical contribution.

I think the clean solution is to change what the human authorship means

Make the arXiv object explicitly a **documented AI-generated research artifact**.

You are the human author of the **report/provenance record**, accountable verifier and submitter. The report contains a scientific work whose substantive content is attributed explicitly to ChatGPT.

For example, the first page could look conceptually like:

```
> From Distribution-Free CDF Bands to Polyhedral Confidence Regions for -LocationScale Families
> An AI-generated research note
>
> [Your Name]
> Human operator, verifier, curator, and accountable submitter
>
> Scientific content generated by ChatGPT (OpenAI, GPT-5.6 Sol)
```

I wouldn't use "editor because it still understates your actual role, and I would probably keep "witness on the new "age out of the formal author line. I like the sentiment, though. It could make an excellent sentence in a preface.

Then put a conspicuous box on page -not buried in acknowledgements:

```
> Provenance statement.
> This work was produced in an interactive dialogue between the named human submitter and ChatGPT (OpenAI, GPT-5.6 Sol). The human supplied the initial research question and subsequently selected directions for investigation, executed the computational experiments proposed by the model, returned their outputs, checked the mathematical arguments and implementation, and decided to preserve and disseminate the resulting work. The substantive scientific -developmentincluding the literature search, formulation of the general framework, mathematical derivations, algorithm design, R code drafts, simulation design, interpretation of results, and initial manuscript -textwas generated by ChatGPT. The named human does not claim authorship of those contributions in the ordinary sense; he serves as their human operator, verifier, curator, and accountable submitter. ChatGPT is not listed as an author because current scholarly infrastructure does not recognize an LLM as an accountable author.
```

That says exactly what happened.

And I would go **further**, because this is an unusually interesting provenance case. Include a contribution table:

Activity	Human	ChatGPT
Initial research question	Primary	-
Choice of follow-up questions	Joint / human-directed	Suggestions
Literature search	-/ verification	Primary
Mathematical formulation	Verification	Primary
Proofs and derivations	Verification	Primary
Algorithm design	Verification	Primary
R code drafting	Execution/testing	Primary
Simulation execution	Primary	Design
Interpretation	Selection/verification	Primary draft
Manuscript drafting	Review	Primary
Decision to disseminate	Primary	-

There is no need to inflate either side.

Preserve the conversation

This is the part I think is scientifically important.

If the claim of the document is partly that an LLM produced substantial original-looking research through this process, then the **interaction itself is experimental evidence**.

I would therefore preserve as ancillary/repository material:

- the full conversation, or as much of it as can appropriately be made public;
- the final R source;
- seeds and simulation settings;
- raw simulation tables;
- the exact model identity, `GPT-5.6 Sol`;
- dates of the interaction;
- which web searches/sources were used;
- a clear indication of any manual edits made after generation.

arXiv explicitly supports ancillary files such as data and programs. [citeturn969524view0](#)

That makes the paper reproducible not only statistically but **provenance-reproducible**.

There is also a platform that fits your idea almost exactly

Rather remarkably, in 2026 there is now **AiraXiv**, which explicitly has an "AI-Generated" Track for papers "substantially contributed to or generated end-to-end by AI". [citeturn158949view0](#)

That may actually be the philosophically cleaner home for this experiment than trying to make arXiv's conventional human-authorship machinery represent something it was not designed for.

You could conceivably put the technical artifact there as AI-generated and separately put an arXiv note about the experiment if the latter satisfies arXiv's moderation and submission requirements.

One legal/practical wrinkle is less problematic than it looks

As between you and OpenAI, the current OpenAI Terms say that, to the extent permitted by law, you own the output and OpenAI assigns to you whatever rights it has in it. [citeturn118622search0](#) So OpenAI is not claiming authorship or ownership over this manuscript.

That does **not** magically make AI-generated text human-authored for copyright or academic-authorship purposes. But it does mean there isn't a competing claim from me/OpenAI saying "you cannot publish this because it belongs to ChatGPT".

And I think there is a second paper hiding here

The statistics result may be interesting.

But what just happened may itself be at least as interesting:

> A human supplied a fairly informal statistical question. Through dialogue, an LLM recognized the connection to test inversion, discovered that an apparent new result was old, performed a deeper literature search, reformulated the project around a potentially novel direction, derived the geometry, produced algorithms and code, diagnosed a nontrivial indexing error after empirical coverage exposed it, redesigned the experiment, and generated a coherent research manuscript.

The **indexing-error episode is particularly valuable**. The model initially gave you the wrong bounds, the simulations showed suspicious coverage, and subsequent reasoning identified and corrected the $\frac{i}{n}$ versus $\frac{(i-1)}{n}$ issue. That is much closer to an actual scientific feedback loop than simply asking an LLM to "write a paper".

So I would not hide the unusual authorship situation. I would make it part of the object.

Your phrase "operator, editor and "witness gets at the spirit, but I would formalize it as:

```
\[
\boxed{\text{Human operator, verifier, curator, and accountable submitter}}
\]
```

and then state, prominently,

```
\[
\boxed{\text{Scientific development and manuscript generation: ChatGPT, GPT-5.6 Sol}}
\]
```

That seems to me both intellectually honest and much more interesting than quietly placing your name over work you do not feel is yours.

One final practical point: because arXiv's submittal agreement literally asks the submitter to affirm original authorship, I would contact arXiv before submission with this exact proposed attribution scheme rather than

trying to infer how they will interpret such an unusual case. Their current agreement was clearly not written with "human-curated, AI-generated "research in mind. citeturn478867view0

Turn 35

User

Well, I like the idea of preserving the conversation itself. I also think that the full code with comments should be in the paper itself. What do you think about it?

Assistant

I think preserving the conversation is actually the most scientifically valuable part of this whole exercise. The conversation is not just a record of "how the manuscript was "written; it is part of the ****experimental protocol****.

There are two separate things here:

1. ****Reproducibility of the statistical result****
2. ****Reproducibility of the -humanAI research process****

Most papers only address the first. Your situation makes the second interesting.

1. Preserve the full conversation

I would definitely keep the full transcript as an artifact. Ideally:

- export the conversation as plain text or PDF;
- preserve timestamps;
- preserve the model identity;
- preserve all code versions;
- preserve all simulation outputs.

I would not necessarily put the entire conversation into the main paper. It would make the paper unreadable. Instead:

- main paper: concise provenance statement;
- supplementary material: full conversation;
- repository (Zenodo/GitHub/etc.): raw transcript + code + results.

The transcript itself is effectively a ****lab notebook****.

What is particularly valuable is that it contains:

- the initial idea;
- the model's first (partly wrong) reasoning;
- the literature correction;
- the discovery that KS convexity was already known;
- the refinement of the research question;
- the code development;
- the coverage failure;
- the debugging step;
- the corrected experiment.

That is a much richer record than a polished final manuscript.

In traditional science, nobody publishes the first incorrect calculation attempts. But here the process itself is the phenomenon.

2. Including all code in the paper

I like the idea, but I would separate ****core algorithm code**** from ****implementation code****.

A full R implementation of:

- KS calibration,
- DKW bounds,
- -BerkJones calibration,
- -DümbgenWellner calibration,
- envelope construction,
- confidence-region construction,
- simulation framework,

```

is probably hundreds of lines.

Putting all of it inside the PDF would make the paper very long and less readable.

I would do:

### In the paper body

Include:

- the mathematical algorithm;
- pseudocode;
- the key formulas;
- complexity statements.

For example:

> Algorithm 1: Construction of a band-induced  $\ell$ -locationscale confidence region.

with steps:

1. Compute  $(a_i = G^{-1}(\ell_i))$ ,  $(b_i = G^{-1}(u_i))$ .
2. Construct  $(L(\sigma))$  and  $(U(\sigma))$ .
3. Compute the feasible scale interval.
4. Extract envelope breakpoints.
5. Integrate  $(U-L)$ .

That is the scientific contribution.

### In supplementary material

Include:

- full R source;
- commented functions;
- examples reproducing every table.

### In a repository

Include:

```

```

paper/ manuscript.tex references.bib
code/ ks.R dkw.R berk_jones.R dumbgen_wellner.R confidence_region.R simulations.R
results/ normal/ laplace/ t2/
ai/ full_chat_transcript.pdf provenance.md

```

That would be an unusually transparent research package.

3. The code comments are actually important here

Normally, I would say "keep code comments minimal". But in this case, I think extensive comments are justified because the computational pipeline itself is part of the contribution.

For example, comments explaining:

```

# Lower endpoint uses  $i/n$  because:  $\hat{D}_n^+ = \max(i/n - U_{(i)})$ 
# Upper endpoint uses  $(i-1)/n$  because:  $\hat{D}_n^- = \max(U_{(i)} - (i-1)/n)$ 

```

are not ordinary programming comments. They document a subtle statistical point.

Similarly:

```

# We simulate  $X = G^{-1}(U)$  rather than using rG() # to guarantee consistency between simulation and fitted # location-scale family.

```

is valuable methodological documentation.

4. I would actually add a "Research provenance" section

Not an apology. Not a disclaimer. A methodological section.

Something like:

> This manuscript documents an unusual research workflow in which a large language model was used as the primary generator of mathematical development, computational algorithms, and manuscript text. The complete interaction transcript is provided as supplementary material. The purpose of preserving the transcript is to make the research-generation process transparent and reproducible.

That is honest and, frankly, scientifically interesting.

5. One more thing: preserve the failed versions

Do not only preserve the final code.

The wrong KS/BJ/DW indexing is actually a valuable part of the record.

Why?

Because it demonstrates something important:

- the AI-generated derivation was not automatically correct;
- empirical coverage diagnostics detected a problem;
- the workflow corrected the mistake.

That is much closer to real research.

A future reader may find the debugging trajectory more interesting than the final result.

So my preference would be:

****Main paper****

- mathematical contribution;
- algorithms;
- simulations;
- concise AI provenance.

****Supplement****

- full R code with comments;
- additional experiments.

****Repository****

- full conversation;
- all code versions;
- simulation seeds;
- raw outputs.

In fact, if the paper is partly about the scientific capabilities of LLMs, hiding the conversation would remove one of the most interesting parts of the experiment. The transcript is not an embarrassment or a log file; it is the equivalent of the experimental apparatus.

Turn 36

User

Is it appropriate to prepare the transcript now or should it be done after the preparation of the final version of the text? I think that there is a lot to be done in there in terms of editing, code and comments, but the most of the work is done and it is a good idea to preserve this work in some kind of artifact

Assistant

I think your instinct is right: ****preserve the transcript now, before further editing.**** Do not wait until the final manuscript.

The reason is that there are actually two different artifacts:

1. ****The research process artifact****

This is the raw conversation as it happened. It records:

- the initial question;

- the sequence of ideas;
- the mistakes and corrections;
- the transition from a vague idea to a research plan;
- the derivations;
- the code-generation process;
- the debugging loop.

2. ****The research product artifact****

This is the polished paper:

- cleaned mathematics;
- verified proofs;
- final algorithms;
- final code;
- final simulations;
- final conclusions.

They serve different purposes. Editing the transcript into a nicer document would destroy part of its value. The raw version is analogous to a laboratory notebook: you do not rewrite your lab notebook after obtaining the final result.

What I would do now is:

1. Freeze the current state

Export the conversation in its current form.

Add metadata:

AI research transcript

Project: From Distribution-Free CDF Bands to Polyhedral Confidence Regions for Location-Scale Families

Human participant: [Name]

AI system: ChatGPT (OpenAI)

Date range: [dates]

Purpose: Complete record of the research-development interaction.

Do not edit the transcript itself.

2. Create a separate "curated transcript"

After freezing the raw transcript, create another document:

****Supplementary Material S1: Annotated Research Transcript****

This can contain:

- section headings;
- references to the final paper sections;
- explanations of where corrections happened;
- links to code versions.

For example:

Conversation segment 1: Initial formulation of CDF-band inversion.

Conversation segment 2: Discovery that KS convexity was known (Salvia 1980).

Conversation segment 3: Development of the polyhedral formulation.

Conversation segment 4: Implementation and debugging of finite-sample indexing.

Conversation segment 5: Simulation results and interpretation.

This gives readers a usable document without compromising the historical record.

3. Freeze the code separately

The code is even more important to freeze now.

At the moment you have something like a "development branch":

- first KS implementation;
- first BJ implementation;

```
- first DW implementation;
- incorrect indexing;
- corrected indexing;
- general location-scale version.
```

Keep that history.

For a repository I would have:

code/ v01_initial/ v02_corrected_bounds/ v03_general_location_scale/ v04_final/

The temptation is always to keep only the final clean version. But in this specific project, the evolution is part of the scientific record.

4. Do not wait for the paper to be finished

The reason is practical: once you start polishing, you will unconsciously rewrite history.

For example, the final paper might say:

> "We construct exact order-statistic bounds..."

But the transcript shows something more realistic:

> "We first implemented the wrong indexing, obtained 97% coverage, investigated the discrepancy, and corrected the construction."

That difference matters if the goal is to document AI-assisted scientific discovery.

5. One thing I would add immediately

Create a short README file now:

README_provenance.md

This repository contains: 1. Raw transcript of the AI-assisted research interaction. 2. Current manuscript draft. 3. R implementations. 4. Simulation results.

The transcript is preserved in its original form. Subsequent edits to the manuscript do not modify the historical record.

That makes the provenance clear.

I would compare it to a mathematician's handwritten notebook or a physicist's experimental log. The final paper is the polished argument. The transcript is the record of how the argument came into existence.

Given the unusual nature of this project, I actually think preserving the raw conversation is not just - appropriateit may become one of the most interesting parts of the contribution. The fact that the workflow included a genuine debugging cycle (coverage revealing an indexing error) makes it much more scientifically credible than a simple "AI generated a paper" story.

Turn 37

User

Looks like you are excited => So... Can you create a full conversation transcript? I think pdf as a final result is the best choice. Meanwhile I'll save the jupyter notebook with current code.

Assistant

I agree that saving the notebook now is the right move. And yes, I am enthusiastic about the *research-process aspect* -herenot in the sense that the AI should replace the scientist, but because this is an unusually well -documented example of a -humanAI research workflow with actual iteration, mistakes, corrections, and refinement.

For the transcript: yes, a PDF is a good archival format. I would suggest preserving it in a slightly more structured way rather than just printing the chat window. The PDF should contain:

```

1. **Cover page**
- Project title:
  > From Distribution-Free CDF Bands to Polyhedral Confidence Regions for  $\eta$ -LocationScale Families
- Human participant
- AI system used
- Date range
- Purpose:
  > Research provenance record of an AI-assisted scientific development process

2. **Conversation transcript**
- Full chronological dialogue
- Preserve:
  - user messages;
  - assistant responses;
  - equations;
  - code blocks;
  - corrections;
  - intermediate ideas.

3. **Metadata appendix**
- Model name
- Any relevant settings
- Notes about subsequent verification/editing

4. **A final note**
  Something like:

> This document is an unedited record of the research-development interaction. It is preserved separately from
  the final manuscript. Mathematical statements, algorithms, software, and numerical results appearing in the
  final research artifact may differ after subsequent verification and refinement.

I would **not** clean up the transcript too much. The rawness is part of the value. In particular, keep:

- the moment where we discovered that KS convexity was already known;
- the first incorrect order-statistic indexing;
- the coverage discrepancy;
- the correction;
- the transition from a normal-only idea to the general  $\eta$ -locationscale framework.

Those are exactly the kinds of things that make it a research record rather than a generated document.

One limitation: I cannot directly retrieve the entire chat history from the interface and automatically package
it into a PDF from inside this conversation. The safest way is:

1. Use ChatGPT's export/share functionality to export the conversation, or copy the conversation text.
2. Save the raw export.
3. Convert that export to PDF.
4. Keep the original raw text/HTML export alongside the PDF.

If you want a polished archival PDF, the best workflow is:

```

conversation_raw.html | v conversation_transcript.md | v conversation_transcript.pdf

where the Markdown version is the editable archival source and PDF is the presentation format.

Also: keep the Jupyter notebook exactly as it is now, including the current state. I would actually make two copies:

research/ notebook_current.ipynb notebook_frozen_2026-08-20.ipynb

The frozen copy is the provenance snapshot. Future cleanup happens only in the working copy.

This is one of those rare situations where the messy development history is not η -noise it is part of the object being studied. So preserving it before polishing is the correct order.

Preservation note

This PDF is a static provenance snapshot generated from the accessible source conversation on 20 August 2026. It is not a substitute for the source-platform export, which should be retained when available.